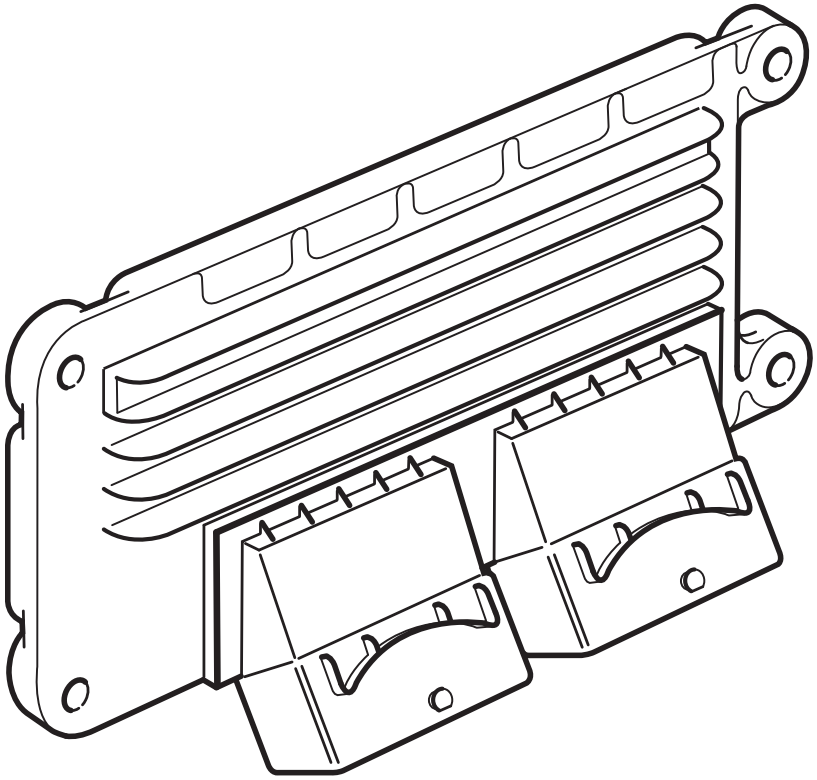
Lotus T4e ECU

Engine Management & Tuning Guide

Elise / Exige / 211 (Toyota 1ZZ-FE / 2ZZ-GE)



Reverse Engineered by J.F.

Version 1.1 — June 25, 2026

1 Table of contents

Contents

1 Table of contents **2**

2 List of figures **5**

3 List of tables **6**

4 Change Log **7**

5 Introduction **8**
5.1 Audience . . . 8
5.2 Workflow . . . 8

6 RomRaider Definition **9**
6.1 Compatibility . . . 9
6.2 Opening CRP file . . . 9
6.3 Terminology . . . 10

7 Hardware **12**
7.1 Overview . . . 12
7.2 Safety CPU (MC68HC908JK8) . . . 13
7.3 Time Processor Units (TPU) . . . 14
7.4 Pinout . . . 16

8 Load Calculation **17**
8.1 Overview . . . 17
8.2 Air Load: MAF vs Alpha-N . . . 18
8.3 Alpha-N Learned Adjustment . . . 19
8.4 Tuning Tips . . . 20

9 Fuel Injection **21**
9.1 Overview . . . 21
9.2 Running Mode . . . 23
9.3 Fuel Load Detail . . . 24
9.4 Cranking Mode . . . 25
9.5 DFSO (Decel Fuel Shut-Off) . . . 26
9.6 Rev Limiter . . . 27
9.7 Tuning Tips . . . 28

10 Transient Fueling **30**
10.1 Overview . . . 30
10.2 Tip-In Enrichment . . . 31
10.3 Tip-Out Enrichment . . . 32
10.4 DFSO Reactive Enrichment . . . 33

11 Closed-Loop Fuel Control **34**
11.1 Overview . . . 34
11.2 Short-Term Fuel Trim (STFT) . . . 35
11.3 Long-Term Fuel Trim (LTFT) . . . 36
11.4 Tuning Tips . . . 37

12 Ignition **39**
12.1 Overview . . . 39
12.2 Running Mode . . . 40
12.3 Per-Cylinder Final Stage . . . 41
12.4 Idle Mode . . . 42
12.5 Cranking Mode . . . 43
12.6 DFSO Mode . . . 44
12.7 Traction Control Retard . . . 45
12.8 Tuning Tips . . . 46
12.9 Igniter Feedback (IGF) . . . 47
12.10 Combustion Misfire Detection . . . 47

13 Knock Detection **49**
13.1 Overview . . . 49
13.2 Knock Retard 1 — Immediate Retard . . . 51
13.3 Knock Retard 2 — Octane Learned Factor . . . 52
13.4 Tuning Tips . . . 53

14 Idle Control **55**
14.1 Overview . . . 55
14.2 Speed Target . . . 57
14.3 Flow Target . . . 58

15 Throttle Control **59**
15.1 Overview . . . 59
15.2 Throttle Body Startup Sequence . . . 59
15.3 Engine-Stopped Sweep Test . . . 60
15.4 Pedal Position Sensor (PPS) Calibration . . . 60
15.5 TPS Target . . . 62
15.6 Tuning Tips . . . 63

16 Variable Valve Timing & Lift **64**
16.1 Overview . . . 64
16.2 VVT Target . . . 65
16.3 VVL Engagement . . . 66

17 Traction Control **67**
17.1 Overview . . . 67
17.2 Slip . . . 68

18 Cooling System **69**
18.1 Overview . . . 69
18.2 Radiator Fan . . . 69
18.3 AC Compressor Clutch . . . 70
18.4 Coolant Recirculation Pump . . . 70
18.5 Idle Flow Adjustments . . . 70

19 Evaporative Emission Control **71**
19.1 Overview . . . 71
19.2 Federal Leak Test . . . 73

20 Data-Logging **74**
20.1 Live Tuning Access . . . 74

20.2 Snapshot Logging . . . 75
20.3 Tuning Tips . . . 75

21 Protocols **76**
21.1 TPMS . . . 76
21.2 Instrument Cluster . . . 77

22 OBD Diagnostics **79**
22.1 Mode $11 — Reset Learned Tables . . . 79
22.2 Mode $22 — Read Data by Identifier . . . 80
22.3 Mode $2F — Input/Output Control . . . 83
22.4 Mode $3B — VIN Programming . . . 84

2 List of figures

List of Figures

1 HC08 safety-processor state machine . . . 14
2 T4E Pinout . . . 16
3 Air load calculation — MAF and Alpha-N paths . . . 18
4 Alpha-N learned adjustment correction . . . 19
5 Load vs. MAP scatter plot, coloured by RPM (MegaLogViewer HD). . . . 20
6 Injection time — running mode . . . 23
7 Fuel load calculation detail . . . 24
8 Injection time — cranking mode . . . 25
9 DFSO enable/disable conditions . . . 26
10 Rev limiter — fuel cut . . . 27
11 Wideband AFR log — MegaLogViewer HD. . . . 29
12 Tip-in enrichment formula . . . 31
13 Tip-out enrichment formula . . . 32
14 DFSO reactive enrichment formula . . . 33
15 STFT formula . . . 35
16 LTFT formula . . . 36
17 AFR Target - Learning Zones (Lotus Exige S Cup 260) . . . 38
18 Ignition advance — running mode . . . 40
19 Ignition advance — per-cylinder final stage . . . 41
20 Ignition advance — idle mode . . . 42
21 Ignition advance — cranking mode . . . 43
22 Ignition advance — DFSO mode . . . 44
23 Ignition retard from traction control . . . 45
24 Live ignition map editing over CAN-Bus RAM access on a dyno. . . . 46
25 Knock retard 1 — immediate retard and recovery . . . 51
26 Knock retard 2 — octane learned factor . . . 52
27 Cylinder 3 noise floor baseline — MegaLogViewer HD (Lotus Exige S Cup 260). . . . 54
28 `CAL_knock_peak_threshold` — Lotus Exige S Cup 260 original calibration. . . . 54
29 Idle speed target calculation . . . 57
30 Idle flow target . . . 58
31 TPS target calculation . . . 62
32 VVT target advance angle . . . 65
33 VVL engagement conditions . . . 66
34 Traction control — slip calculation and target . . . 68

3 List of tables

List of Tables

1 Subsystem name prefixes . . . 10
2 Common name suffixes . . . 11
3 ECU board components . . . 12
4 TPU A channel assignments . . . 15
5 TPU B channel assignments . . . 15
6 Cluster CAN frame (ID 0x400) layout . . . 77
7 Cluster warning light bit encoding . . . 78
8 Mode $22 live data PIDs (0x02xx) . . . 81
9 Mode $22 performance log PIDs (0x03xx) . . . 82
10 Mode $2F actuator PIDs . . . 83
11 Mode $3B VIN programming index map . . . 84

4 Change Log

1.1 (2026-06-24)

- Added the Safety CPU (HC08) state-machine section with the MAF airflow cross-check and reprogramming note (Section 7.2).
- Fixed the forced-idle DTC (P2014 → P2104); engine shutdown disables the injectors, not the ignition coil.

1.0 (2026-03-03)

- Initial release of the tuning guide.

5 Introduction

This document describes the internal workings of the Lotus T4e ECU (MPC563-based) used in Lotus Elise, Exige, and 211 models equipped with the Toyota 1ZZ-FE and 2ZZ-GE engines. It is intended as both a technical reference and a tuning guide.

The ECU firmware has been reverse engineered from the binary, and the formulas presented here are derived directly from the decompiled code.

5.1 Audience

This guide is meant for Lotus owners who want full control over their car. It represents more than a hundred hours of reverse engineering work, shared freely because I believe every owner should be able to understand and tweak their own ECU.

However, I am not willing for professional tuners to use this work to develop tunes sold to customers without supporting the project, either financially or by contributing improvements. This is why the entire project, including this document, is released under the **Creative Commons Attribution-NonCommercial-ShareAlike (CC BY-NC-SA)** license. Any material and knowledge contained here cannot be used commercially (i.e. by professionals requesting money to tune cars) unless an agreement has been reached with the project author.

Professional tuners are welcome to use this material as long as they do so without commercial intent, meaning the resulting tune must be made freely available under the same **CC BY-NC-SA** license. Charging separately for hardware, installation, or other services unrelated to the tune itself remains perfectly acceptable.

If you are a professional without intent to support this project or publish your work freely, STOP READING HERE.

5.2 Workflow

This guide is the end product of a multi-step reverse engineering pipeline:

1. **Original firmware** — the binary is read from the ECU flash.
2. **Ghidra** — the binary is disassembled and decompiled into readable C code. Functions, variables, and data structures are identified and renamed.
3. **Identifying calibration data** — maps and parameters are tagged as `CAL_` (calibration) or `LEA_` (learned adaptive values).
4. **Export script** — a Ghidra script exports the identified symbols into RomRaider definition files.
5. **RomRaider** — the definition files allow editing the calibration maps in `calrom.bin`.
6. **Exported C code** — Ghidra's decompiled C output is exported for further analysis.
7. **Claude AI assistant** — the exported C code is analyzed with Claude to trace formulas, identify logic, and produce the descriptions and diagrams in this guide.
8. **Human review** — all AI-generated text and diagrams are thoroughly reviewed before publication.

Every symbol identified in the source code as `CAL_` or `LEA_` is automatically exported into a RomRaider definition. The prefix is removed, the first word becomes the category, and all underscores are replaced by spaces. For example, `CAL_ign_adv_low_cam_base` appears in RomRaider under the category *ign* as *adv low cam base*. The same naming convention is used in the formula diagrams throughout this guide.

6 RomRaider Definition

6.1 Compatibility

The `T4E090_def.xml` is a comprehensive definition that covers all `CAL_` parameters discussed in this guide, allowing them to be edited in RomRaider. It also enables visualisation of `LEA_` parameters; these can be edited too, but modifying learned adaptation values has little effect since the ECU will overwrite them sooner or later.

The following calibrations use the exact same software:

- A131E0018
- A121E0013
- A128E0032
- A129E0001
- B121E0014
- A128E0036
- A127E0062
- A127E0064
- A127E0066
- A127E0068
- A120E0034
- A128E0031
- A129E0002
- B120E0035

The following calibrations use similar software and are fully compatible with the definition:

- D129E6004H
- F121E0010H
- C131E6018H
- C128E6010H
- D120E0030H
- A128E0056
- A121E0023
- A121E0025
- A128E0046
- A128E0050
- A128E0052
- A121E0030
- A128E0064
- B121E0027
- B121E0030
- B128E0058
- B128E0061
- A121E0037
- A121E0039
- A128E0069
- A128E0071
- A128E0073
- A121E0041
- A121E0043
- A128E0078
- A128E0082
- A128E0086
- A127E0072
- A127E0076
- A127E0070
- A127E0074
- A120E0096
- A120E0102
- A128E0076
- A128E0080
- A128E0084
- A131E0022
- A128E0088
- A121E0046
- A121E0050
- A128E0092
- A128E0098
- A128E0102

The first letter of a calibration number is its revision (e.g. `A128E6010H` → `C128E6010H`). If your calibration matches one of the above except for that first letter, you can update your car to a listed revision before tuning, so that the definition tables align exactly with your calibration.

6.2 Opening CRP file

Lotus ECU updates are distributed as `.CRP` files — encrypted firmware packages that generally contain a calibration (`calrom.bin`) and a program (`prog.bin`). They are available from the Lotus VSIC portal or shipped with Lotus TechCentre and Lotus Scan 3.

RomRaider cannot open a `.CRP` file directly. Opening the `.CRP` with the *CRP08 Editor* tool decrypts it and produces two plain-binary `.cpt` files — for example, opening `A129E0002.CRP` yields:

`T4E_MY08_BIN.cpt` The program binary (equivalent to `prog.bin`).

`A129E0002_TAB.cpt` The calibration table (equivalent to `calrom.bin`).

Since `.cpt` files are plain binary, the calibration file can be opened directly in RomRaider using `File` → `Open Image`. Alternatively, a live ECU dump from the flasher tool produces `calrom.bin` directly.

Load the `T4E090_defs.xml` definition file if it is not already loaded (`ECU Definitions` → `ECU Definition Manager`).

To view the `LEA_` learned parameters, open `decram.bin` the same way — the same definition file covers both files.

After editing, save the modified `calrom.bin` and repack it into a `.CRP` file with the CRP editor before flashing back to the ECU.

6.3 Terminology

All parameter names follow the pattern `PREFIX_category_detail_suffix`. `CAL_` and `LEA_` are the two special prefixes; all other prefixes denote the subsystem and map to the RomRaider category.

`CAL_` Calibration constant stored in flash, editable in RomRaider. The prefix is stripped on export; the first word becomes the category and underscores become spaces.

`LEA_` Learned (adaptive) value stored in EEPROM, visible in RomRaider via `decram.bin`.

The subsystem prefixes used in this guide are:

Prefix	Subsystem
`ac_`	Air conditioning.
`dev_`	Development variable, writable via OBD for real-time tuning.
`evap_`	Evaporative emissions (purge canister) control.
`fan_`	Cooling fan control.
`idle_`	Idle speed control.
`ign_`	Ignition: advance angles, dwell, knock retard.
`inj_`	Fuel injection: pulse width, corrections, rev limiter.
`injtip_`	Transient fuelling: tip-in enrichment and tip-out enleanment.
`knock_`	Knock detection and retard.
`load_`	Air mass per intake stroke (engine load).
`ltft_`	Long-term fuel trim. Zones: `zone1` (idle), `zone2` (low load), `zone3` (high load).
`pps_`	Pedal position sensor (`_1` and `_2` for the two channels).
`stft_`	Short-term fuel trim: O2 feedback loop and enabling conditions.
`tc_`	Traction control.
`tps_`	Throttle position sensor (`_1` and `_2` for the two channels).
`vvl_`	Variable Valve Lift (high-lift cam engagement).
`vvt_`	Variable Valve Timing (cam phaser).

Table 1: Subsystem name prefixes

The following suffixes appear throughout this guide and the definition file:

Suffix	Meaning
_adj	Adjustment — multiplicative factor or additive offset.
_adv	Advance angle (ignition or VVT cam phaser).
_base	Base value before corrections are applied.
_disable	Threshold to deactivate a feature (hysteresis pair with _enable).
_enable	Threshold to activate a feature (hysteresis pair with _disable).
_fallback	Substitute value used when the sensor is confirmed faulted.
_high	Upper bound of a range pair.
_limit_h	High clamp: value is capped at this maximum.
_limit_l	Low clamp: value is floored at this minimum.
_low	Lower bound of a range pair.
_max	CAL_*_max: upper threshold for a condition.
_min	CAL_*_min: lower threshold for a condition.
_reactivity	Filter responsiveness: higher value = faster tracking.
_retard1	Knock-induced spark retard.
_retard2	Octane scaler applied on top of _retard1.
_smooth	Low-pass filtered value.
_step	Fixed increment/decrement applied equally in both directions.
_stop	Qualifier: applies when the engine is stopped — a value captured at engine-off, a table used at engine-off, or an action triggered at engine-off (e.g. coolant_stop = coolant temperature when the engine was last stopped).
_target	Setpoint or desired value.
_time_between_step	Minimum time between successive correction steps.

Table 2: Common name suffixes

7 Hardware

7.1 Overview

The T4e ECU is built around the Freescale MPC563MZP56, a 32-bit PowerPC microcontroller running at 40 MHz (4 MHz crystal, ×10 PLL). The board integrates dedicated peripheral ICs for power, actuator drive, signal conditioning, and diagnostics.

Component	Role	Description
MPC563MZP56	Main MCU	32-bit PowerPC, 40 MHz. Integrates CPU, 32 KB SRAM, flash controller, two TPUs, TouCAN, QSPI, ADC, and watchdog.
MC68HC908JK8	Safety CPU	Motorola 8-bit HC08. Independently monitors PPS/TPS plausibility; two shutdown mechanisms: forced idle and engine shutdown. See Section 7.2.
SC33394FDH	System basis chip	Freescale SBC: provides regulated 5 V/3.3 V supplies for the MCU, an integrated CAN transceiver, and a hardware watchdog for the main MCU.
VP251	CAN transceiver	Infineon industrial CAN transceiver for the second CAN bus used by the TPMS module.
L9613	K-Line driver	ST ISO 9141/14230 K-Line interface for legacy OBD-II diagnostics.
25160AN	SPI EEPROM	16 Kbit serial EEPROM. Stores all learned parameters (`LEA_` variables) across power cycles.
TLE6220GP	Quad low-side switch	Infineon quad low-side switch. Three devices daisy-chained via SPI for fault diagnostics; individual channel switching is driven directly by TPU B PWM outputs (injectors CH1–5, VVT CH7, VVL CH9).
L9822EPD	Octal solenoid driver	ST 8-channel serial low-side switch. Drives auxiliary outputs: purge solenoid, AC clutch relay, start relay, and others.
TLE6209R	H-Bridge motor driver	Infineon full-bridge for the electronic throttle body DC motor. Provides bidirectional position control with current limiting.
76407D	Power MOSFET	N-channel power MOSFET used to drive the O2 sensor heater elements. Output current is monitored by measuring the voltage drop across a shunt resistor (R10).
MPX4001A	Pressure sensor	Freescale absolute pressure sensor mounted inside the ECU. Used as the barometric pressure reference.
L9119D	Knock amplifier	ST knock sensor signal filter and variable-gain amplifier. Conditions the piezoelectric signal before ADC sampling.
SM5A27	TVS diodes	Bidirectional transient voltage suppressors (27 V clamp) protecting sensor and actuator lines from voltage spikes.

Table 3: ECU board components

7.2 Safety CPU (MC68HC908JK8)

The MC68HC908JK8 is a Motorola 8-bit HC08 microcontroller that operates independently of the main MCU. Its sole function is to monitor the plausibility of the accelerator pedal (PPS) and throttle position (TPS) sensors to detect faults in the electronic throttle system. It communicates with the main MCU via the SCI-2 serial interface.

If an implausible PPS/TPS relationship is detected, the HC08 can trigger one of two shutdown mechanisms:

Forced idle (PTD4) Inhibits the TLE6209R H-Bridge, removing motor drive and allowing the throttle plate to return to its rest position under spring force. The main MCU is simultaneously flagged via serial (`flags_to_mpc` bit 4), which triggers OBD fault P2104.

Engine shutdown (PTD3) Cut the injector drive path. The main MCU is flagged via serial (`flags_to_mpc` bit 5), which triggers OBD fault P2105.

Internally the HC08 runs a six-state machine that *only escalates*: a fault state (forced idle or engine shutdown) latches and can be left only by a hardware reset (Figure 1).

Plausibility monitoring In the normal monitoring state the HC08 samples PPS and TPS every 1 ms and, from a 16-point lookup table, derives the maximum throttle opening allowed for the current pedal position. The table floors at ≈10%, so the idle air opening is permitted without a false trip, and the comparison is bypassed above ≈80% pedal where a wide-open throttle is expected. If the measured throttle exceeds the target for 100 ms, forced idle (P2104) is triggered.

Smoothing suppression While the main MCU runs its throttle-smoothing algorithm it raises a serial flag (`flags_from_mpc` bit 2) that drops the HC08 into a *check-suppressed* state, pinning the fault timers and tolerating an open throttle for up to 400 ms.

Airflow cross-check Forced idle is not the last line of defence. Once idle is forced, the HC08 watches an *independent* airflow signal — the MAF sensor, tapped on its `PTB1` analog input. If airflow remains above 2.2 V for 100 ms despite the forced idle, the throttle has clearly not closed regardless of what the TPS reports, and the HC08 escalates to engine shutdown (P2105). Cross-checking against airflow rather than re-reading the suspect throttle sensor is what makes the shutdown robust.

Reprogramming The bootloader reflashes the HC08 image when the marker at `0x021FE8` is set to `0xAAAAAAAA` (together with the `"HC08CODE"` signature and a firmware pointer). **Before reprogramming, tie the TPS1 input (ECU pin RD3) to 5 V.** RD3 is wired to the HC08's `PTB3` pin, which doubles as the monitor-mode baud-rate strap latched at reset: it must be high to select the clock divisor that yields 9600 baud. Left at the normal sensor level — or merely unplugged and floating — it sits at the wrong level, the monitor link runs at the wrong baud, and programming fails at the passphrase step (HC08 authentication, error `0x90`).

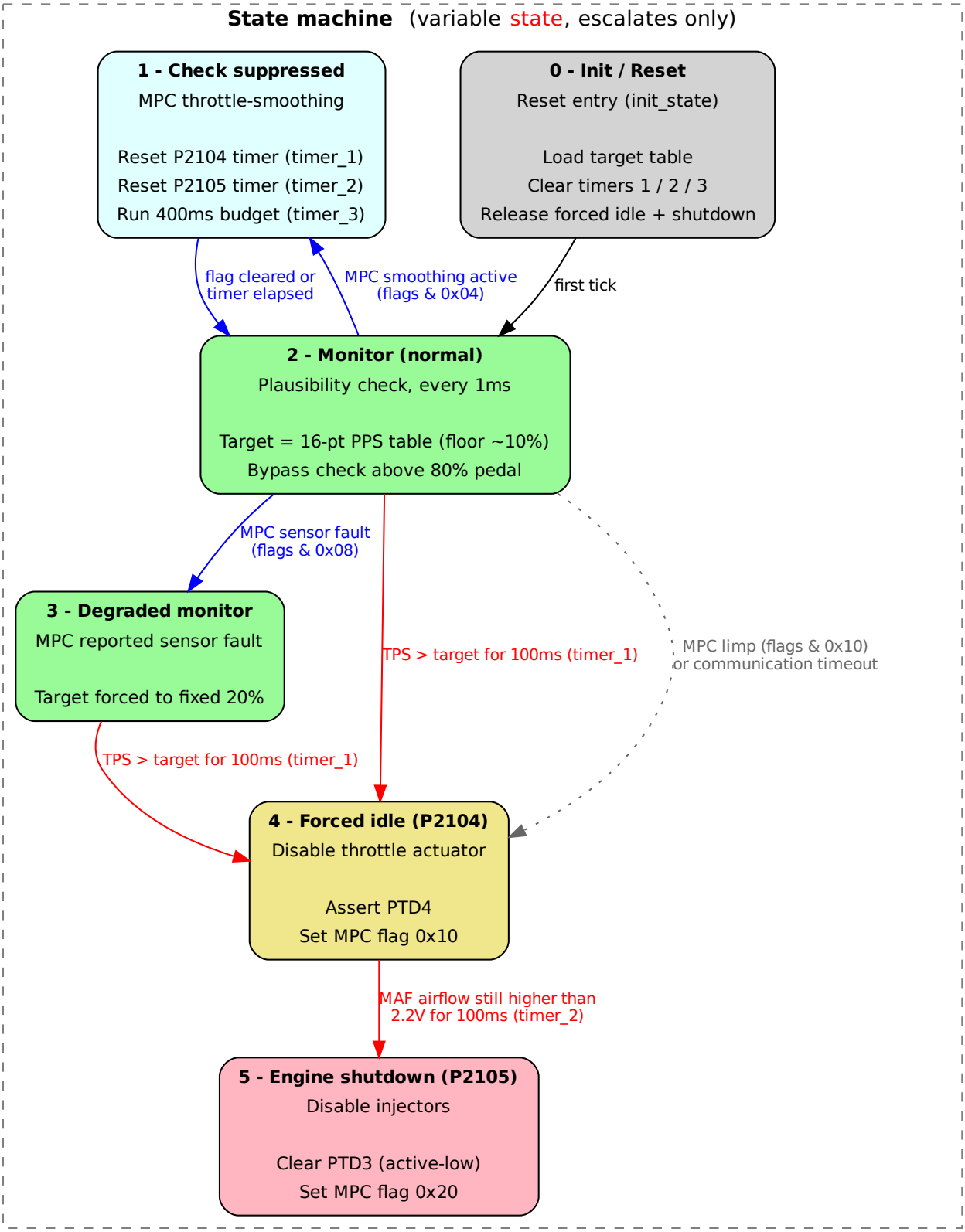


Figure 1: HC08 safety-processor state machine

7.3 Time Processor Units (TPU)

The MPC563 contains two independent Time Processor Units, TPU A and TPU B, each with 16 channels. A TPU is a dedicated co-processor that handles precise time-critical I/O (pulse measurement, PWM generation, period counting) without burdening the main CPU. Each channel can be configured independently

to capture input edges, generate output pulses, or trigger an interrupt at a programmed time.

TPU A — Inputs and Timing

CH	Pin	Function
0	RE1	Crank sync reference (shared with CH7/9/15)
1	RG1	Ignition coil 1 output
2	RG2	Ignition coil 4 output
3	RG4	Ignition coil 2 output
4	RG3	Ignition coil 3 output
5–6	—	Unused
7	—	Knock sensor window gate (crank-angle timed)
8	LF1	ABS wheel speed — rear right
9	RE1	Crank position input (36-1 reluctor)
10	RC1	Cam position input (cylinder ID, VVT feedback)
11	LA4	ABS wheel speed — front left
12	LF4	ABS wheel speed — front right
13	LB4	ABS wheel speed — rear left
14	—	Periodic MAF/MAP ADC sampling trigger
15	RE1	Misfire detection (crank inter-tooth timing)

Table 4: TPU A channel assignments

Ignition coils fire in the 1-3-4-2 order (CH1, 3, 2, 4). The crank signal on pin E1 is shared by CH0, CH7, CH9, and CH15.

TPU B — Outputs and Injection Sequencing

CH	Pin	Function
0	—	Sync reference (clear only)
1	RJ1	Injector 1 timing (TLE6220)
2	RK4	Injector 2 timing (TLE6220)
3	RK3	Injector 3 timing (TLE6220)
4	RK2	Injector 4 timing (TLE6220)
5	RK1	Secondary injector — 2-Eleven only (TLE6220)
6	RL3	Unused (TLE6220)
7	RJ3	VVT solenoid PWM (TLE6220)
8	RH2	Unused (TLE6220)
9	RH3	VVL solenoid PWM (TLE6220)
10	RJ2	Cylinder 1 injection/dwell sequencer
11	—	Cylinder 2 injection/dwell sequencer
12	—	Cylinder 3 injection/dwell sequencer
13	—	Cylinder 4 injection/dwell sequencer
14	—	Unused
15	RF3	Ignition coil feedback input (IGF)

Table 5: TPU B channel assignments

CH10–13 are per-cylinder state machines that coordinate the injector pulse with the ignition dwell window.

7.4 Pinout

BLACK (R) Connector				
PIN		Board Route	MPC563	Description
A	1	Pull-Up 3.3 kΩ	ADC (0x304A9A)	TMAP Sensor Temp Signal (Supercharged Only)
	2	Pull-Up 47 kΩ	ADC (0x304A8A)	Oil Pressure Switch
	3			
	4			Knock Sensor Ground
B	1	Pull-Up 3.3 kΩ	ADC (0x304A94)	MAP Sensor Signal (Supercharged Only)
	2	Pull-Up 47 kΩ	ADC (0x304A92)	Oil Pressure Switch VVTL
	3	Pull-Up 3.3 kΩ	ADC (0x304A90)	IAT Sensor Signal
	4	Pull-Up 47 kΩ	ADC (0x304A96)	MAF Sensor Signal
C	1		A_TPUCH10	Cam Sensor Signal
	2	Pull-Up 3.3 kΩ	ADC (0x304E84)	Traction Control Switch (Specific Models Only)
	3	Pull-Up 3.3 kΩ	ADC (0x304E82)	Coolant Temp Sensor Signal
	4		ADC (0x304E90)	Knock Sensor Signal
D	1			Throttle Body Sensor Ground
	2			MAF/IAT/MAP Sensor Ground
	3	Pull-Up 47 kΩ	ADC (0x304A80)	Throttle Body TPS1
	4	Pull-Up 47 kΩ	ADC (0x304E9E)	FREE INPUT
E	1		A_TPUCH9 / A_TPUCH15	Crank Sensor Signal
	2			Cam Sensor Ground
	3	Pull-Up 47 kΩ	ADC (0x304A82)	Throttle Body TPS2
	4			Crank Sensor Ground
F	1			Knock Sensor Cable Screen
	2			
	3		B_TPUCH15	Ignition Coil Feedback
	4	TLE6220 C OUT4	SPI PCS1 + MPWM1_MDO2	VSV Purge Valve Ground
G	1	R101 + EMD3	MDA11 / A_TPUCH1	Ignition Coil 1
	2	R101 + EMD3	MDA14 / A_TPUCH4	Ignition Coil 4
	3	R101 + EMD3	MDA13 / A_TPUCH3	Ignition Coil 3
	4	R101 + EMD3	MDA12 / A_TPUCH2	Ignition Coil 2
H	1			Coolant Temp Sensor Ground
	2	TLE6220 B OUT4	SPI PCS1 + B_TPUCH8	
	3	TLE6220 C OUT1	SPI PCS1 + B_TPUCH9	Oil Control Valve VVL Ground
	4	L9822E OUT6	SPI PCS1	VSV ACIS Airbox
J	1	TLE6220 A OUT1	SPI PCS1 + B_TPUCH1	Injector 1
	2	TLE6220 C OUT2	SPI PCS1 + B_TPUCH10	
	3	TLE6220 B OUT3	SPI PCS1 + B_TPUCH7	Oil Control Valve VVT Ground
	4	L9822E OUT5	SPI PCS1	Traction Control LED Ground (Specific Models Only)
K	1	TLE6220 B OUT1	SPI PCS1 + B_TPUCH5	Secondary Injector (2-Eleven Only)
	2	TLE6220 A OUT2	SPI PCS1 + B_TPUCH4	Injector 4
	3	TLE6220 A OUT4	SPI PCS1 + B_TPUCH3	Injector 3
	4	TLE6220 A OUT3	SPI PCS1 + B_TPUCH2	Injector 2
L	1	TLE6209 OUT1	SPI PCS3	Throttle Body M+
	2	TLE6209 OUT2	SPI PCS3	Throttle Body M-
	3	TLE6220 B OUT2	SPI PCS1 + B_TPUCH6	
	4			MAP Sensor +5V (Supercharged Only)
M	1			Chassis Ground
	2			Chassis Ground
	3			Chassis Ground
	4			Throttle Body +5V

BROWN (L) Connector				
PIN		Board Route	MPC563	Description
A	1	SC33394FDH P3	TouCAN A	Diagnostic Connector CAN High
	2	VP251 P7 CANH	TouCAN B	TPMS R1 (Specific Models Only)
	3		ADC (0x304E86)	Pre O2 Sensor Signal
	4		A_TPUCH11	ABS Unit Left Front
B	1	SC33394FDH P2	TouCAN A	Diagnostic Connector CAN Low
	2	VP251 P6 CANL	TouCAN B	TPMS R2 (Specific Models Only)
	3		ADC (0x304E8A)	Post O2 Sensor Signal (Except 2-Eleven)
	4		A_TPUCH13	ABS Unit Left Rear
C	1	Pull-Up 47 kΩ	ADC (0x304A8C)	Vap Pressure Signal/Diagnostic (2-Eleven Only)
	2	Pull-Up 47 kΩ	ADC (0x304A8E)	FREE INPUT
	3	Pull-Up 3.3 kΩ	ADC (0x304E98)	FREE INPUT
	4	L9613 PIN8 LI		Diagnostic Connector L Line
D	1	Pull-Down 47 kΩ	ADC (0x304A86)	Throttle Pedal S1 Signal
	2	Pull-Down 47 kΩ	ADC (0x304A88)	Throttle Pedal S2 Signal
	3	Pull-Up 3.3 kΩ	ADC (0x304A9C)	FREE INPUT
	4	Pull-Up 47 kΩ	ADC (0x304A98)	Variable TC Signal (Specific Models Only)
E	1	Pull-Up 47 kΩ	ADC (0x304E9A)	Trinary Switch AC Fan Request
	2		DIN (0x306100 Bit 15)	Trinary Switch AC Request
	3	Pull-Up 47 kΩ	ADC (0x304A9E)	Fuel Tank Sender Signal
	4	Pull-Up 47 kΩ	ADC (0x304E80)	FREE INPUT
F	1		A_TPUCH8	ABS Unit Right Rear
	2		DIN (0x306100 Bit 7)	Clutch Switch Air Bag Harness Canada Only
	3		DIN (0x306100 Bit 8)	FREE INPUT
	4			Pre/Post O2 Sensor Ground
G	1		MDA31 (0x3060FE)	FREE INPUT
	2	L9822E OUT7	SPI PCS1	Start Relay
	3		ADC (0x304E96)	Ignition ACC
	4	L9613 PIN6 K		Diagnostic Connector K Line
H	1			Throttle Pedal S1 Ground
	2	TLE6220 C OUT3	SPI PCS1 + MPWM0_MDO1	VSV CVCV Canister Vent Close Valve Ground
	3		DOUT (0x306100 Bit 4)	Post O2 Heater Ground (Except 2-Eleven)
	4		A_TPUCH12	ABS Unit Right Front
J	1	L9822E OUT4	SPI PCS1	AC Control Module Compressor Relay
	2	L9822E OUT2	SPI PCS1	AC Control Module Rad Fan Fast Relay 2
	3	L9822E OUT3	SPI PCS1	AC Control Module Rad Fan Slow Relay 3
	4	L9822E OUT0	SPI PCS1	Fuel Pump Relay
K	1	L9822E OUT1	SPI PCS1	Hot Soak Starter Internal Connection Relay
	2			Throttle Pedal S1 +5V
	3		DOUT (0x306100 Bit 1)	Pre O2 Sensor Heater Ground
	4			BATT
L	1			Throttle Pedal S2 +5V
	2			
	3			Throttle Pedal S2 Ground
	4			Vap Pressure/Fuel Tank Sender/VTC Ground
M	1			Main Power In
	2			Main Power In
	3			Main Relay
	4			Vap Pressure/Fuel Tank Sender/VTC 5V Ref

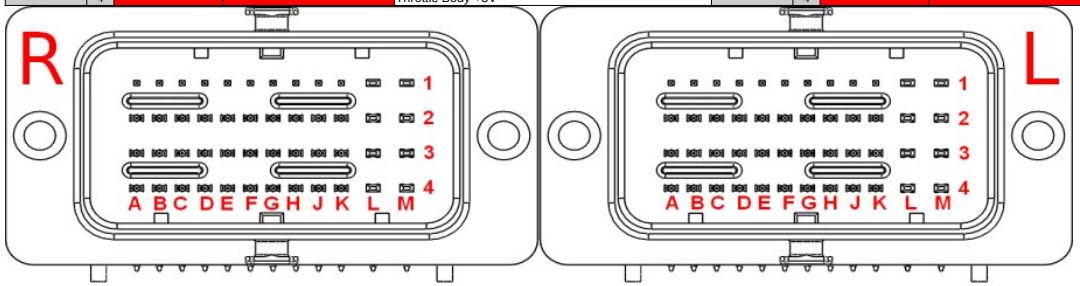


Figure 2: T4E Pinout

8 Load Calculation

8.1 Overview

Load is the air mass per intake stroke (mg/stroke). It is the primary input for fueling calculations.

Two parallel estimates are maintained every cycle:

- **MAF path** — the Mass Air Flow sensor readings (`maf_flow_1`, `maf_flow_2`) are averaged and scaled by engine speed period to convert from a flow rate to a per-stroke mass.
- **Alpha-N path** — a lookup-table estimate (`CAL_load_alphaN_base`, 3D vs RPM and TPS), multiplied by a learned correction factor (`LEA_load_alphaN_adj`), then compensated for atmospheric pressure (`atmo / 1013`) and intake air temperature (`298 / (air_temp + 273)`) using the ideal gas law.

Path Selection

The ECU uses the Alpha-N path when any of the following is true:

- The engine is not running (cranking uses a separate TPS-only table `CAL_load_alphaN_engine_stop`).
- A rapid TPS change is detected: `dt_tps_target_1` exceeds `CAL_load_use_alphaN_dt_tps_target_1_positive_min` or `CAL_load_use_alphaN_dt_tps_target_1_negative_min`.
- TPS is above a minimum threshold (`CAL_load_use_alphaN_tps_min`, 2D vs RPM) and the engine is not idling.
- The MAF sensor has a confirmed fault (P0101/P0102/P0103).

Once triggered, Alpha-N remains active for `CAL_load_use_alphaN_timer` cycles, provided engine speed is below `CAL_load_use_alphaN_engine_speed_max`. Otherwise the MAF path is used.

Alpha-N Learning

The learned correction (`LEA_load_alphaN_adj`) is a 16×16 table (RPM × TPS) that adjusts the Alpha-N base to match the MAF reading. When the MAF is trusted (steady-state conditions), the ECU compares the two estimates and slowly updates the learned table. The correction range is 50–150 %.

EVAP Purge Contribution

Fuel vapour entering the engine via the charcoal canister is added to the load. There is no sensor measuring canister flow directly, so the ECU estimates it from the purge valve duty cycle and the pressure differential. This estimated purge flow is converted from mg/s to mg/stroke and added to the selected load path.

8.2 Air Load: MAF vs Alpha-N

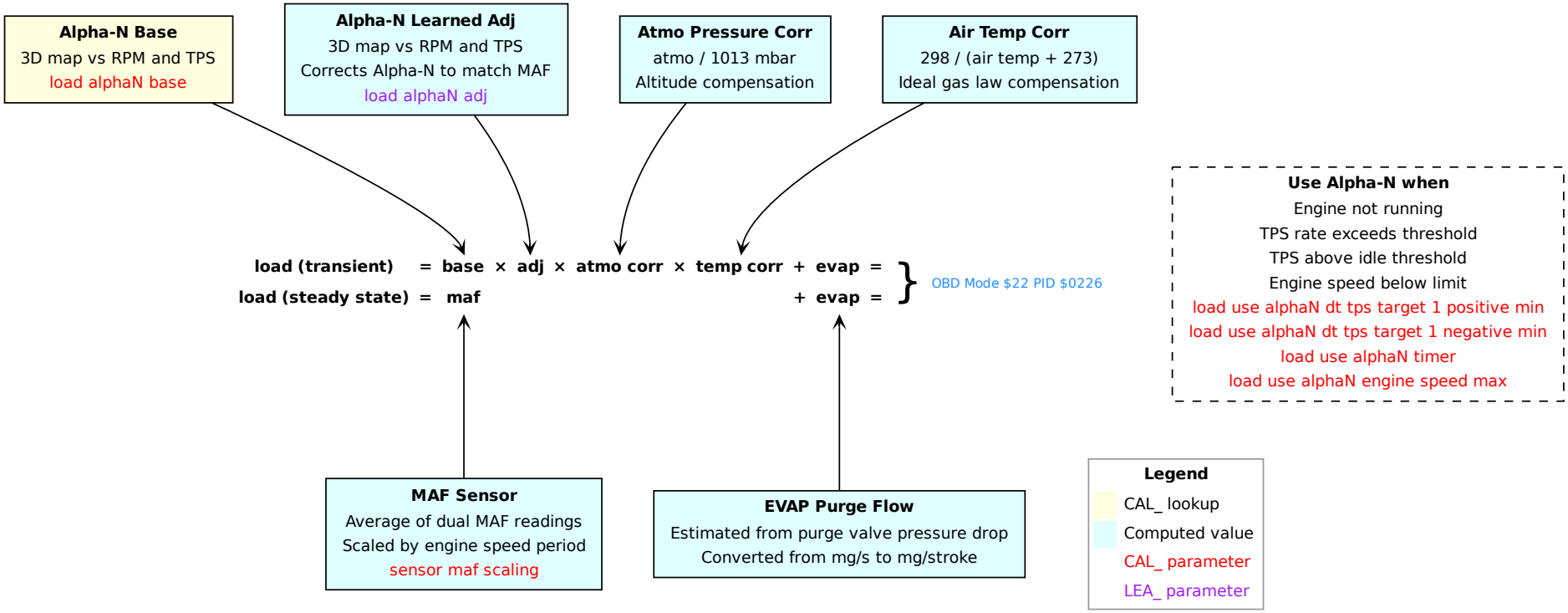


Figure 3: Air load calculation — MAF and Alpha-N paths

8.3 Alpha-N Learned Adjustment

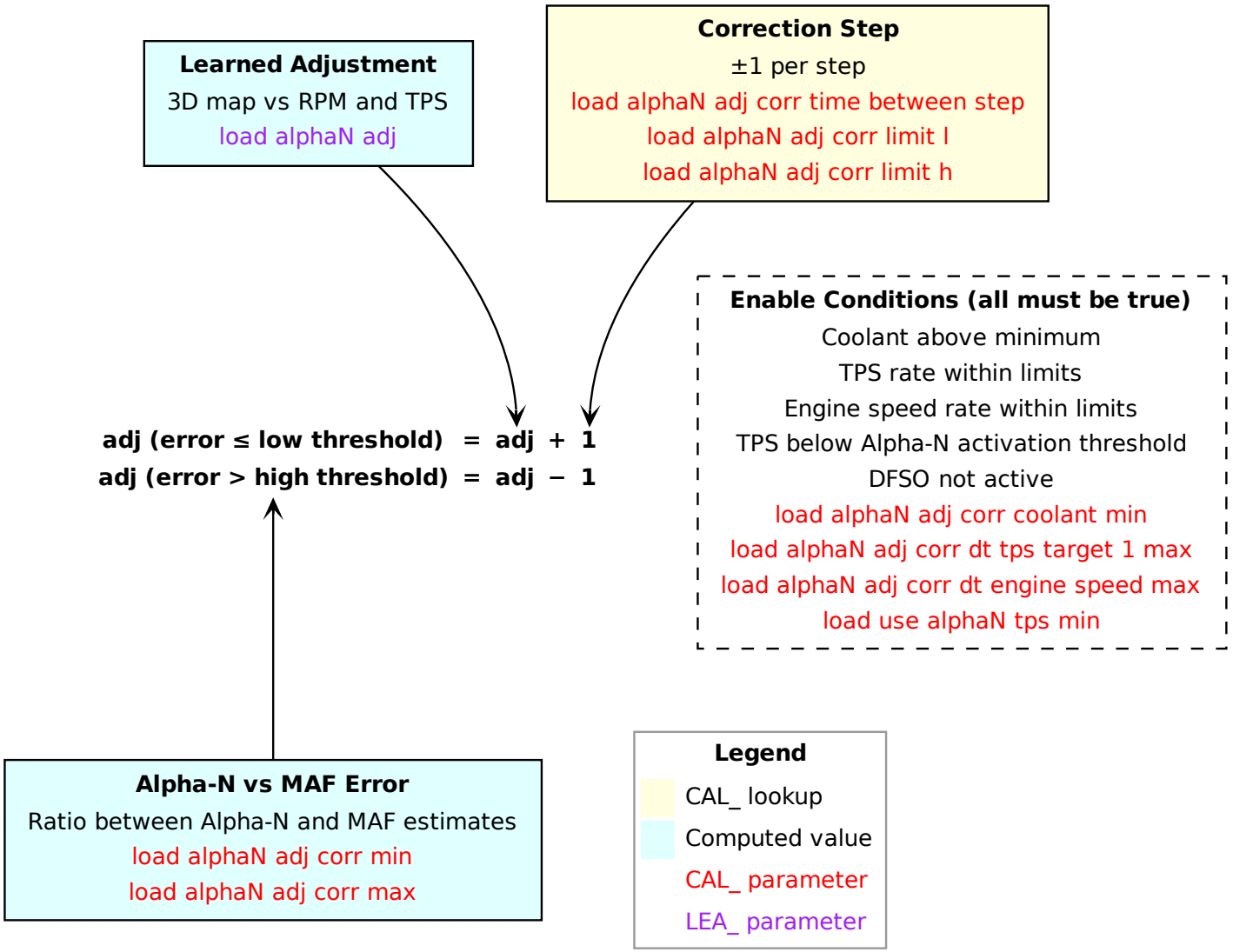


Figure 4: Alpha-N learned adjustment correction

8.4 Tuning Tips

A surprisingly common mistake is for tuners to manually edit `CAL_load_alphaN` — the base Alpha-N table — without first reading the learned correction stored in `LEA_load_alphaN_adj`. **This table is self-adjusted by the ECU.** After even a modest amount of driving, the ECU has already computed and stored corrections for the operating points it has visited. A tuner who edits the base table without consulting the learned correction is, at best, duplicating work the ECU has already done; at worst, they are fighting the ECU's own learning and introducing errors that the ECU will simply correct again after the next drive. Before touching `CAL_load_alphaN`, always read `LEA_load_alphaN_adj`: if the corrections are small and evenly distributed, the base table is already well calibrated. Large or structured corrections tell you something is wrong — but the answer is to investigate the root cause, not to bake the correction into the base table.

Compare the learned Alpha-N table (`LEA_load_alphaN_adj`) before and after any modification to the intake system. The ECU will eventually re-learn the corrections on its own, but a large shift in the table after an intake change is a warning sign worth investigating. The most common cause is not a change in true volumetric efficiency but an error in the MAF reading: intake modifications frequently introduce turbulence upstream of the MAF sensor, disrupting the laminar flow the sensor requires to measure accurately. A poorly positioned cone filter, a short intake pipe, or any geometry change close to the MAF element can all produce this effect and lead the ECU to build a correction that compensates for a false reading rather than a real fuelling error.

A useful sanity check is to plot load per stroke against boost pressure (map) as a scatter plot (Figure 5). On a well-calibrated car the points form a clean, tight line: MAP is a direct measure of the pressure differential driving air into the cylinder, so load should track it predictably. Scatter or a curved relationship suggests a calibration issue worth investigating. On 2ZZ-GE engines with VVL, the line visibly splits at higher load as the high-lift cam engages and increases volumetric efficiency — this is expected behaviour, not a fault. Colouring the points by VVL status rather than RPM would make this separation even clearer. Note that boost pressure (`map`) is not exposed on the standard OBD interface without a software patch — it must be read via the raw RAM access interface described in Section 20.

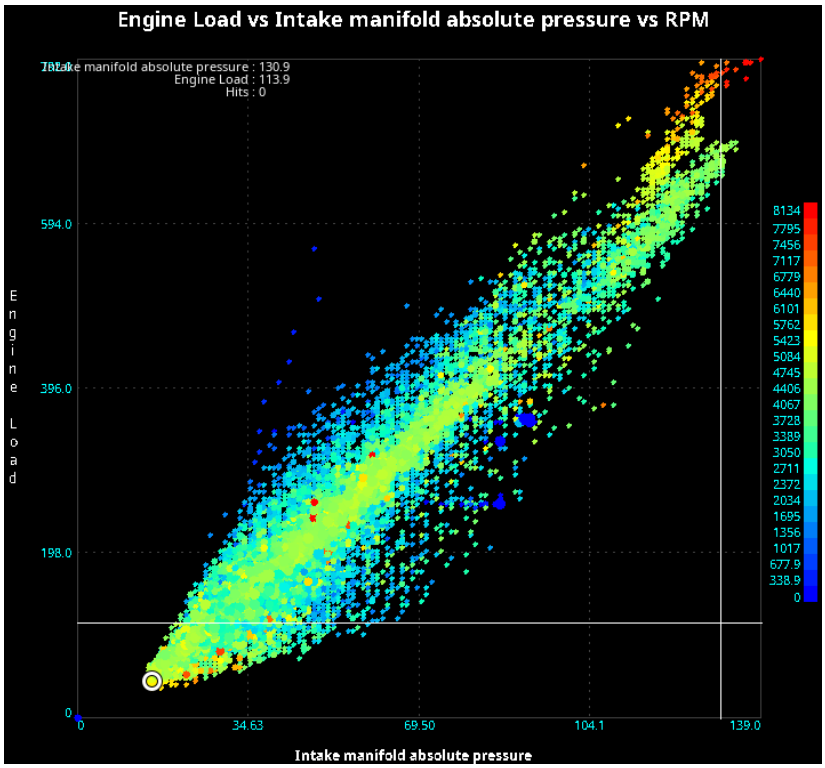


Figure 5: Load vs. MAP scatter plot, coloured by RPM (MegaLogViewer HD).

9 Fuel Injection

9.1 Overview

The injectors are driven by TPU-B channels 1–4 via a TLE6220GP quad low-side switch. The ECU computes a pulse width in microseconds for each combustion event.

Injector Dead Time

The injector opening dead time (`CAL_inj_time_base`, 2D vs battery voltage) is added to the fuel pulse width to compensate for the electrical delay before fuel actually flows.

Running Mode

The running injection time is built up from several components:

1. **Fuel load.** The air load (from MAF or Alpha-N) is divided by the target AFR to obtain the fuel mass needed. The target AFR comes from a 3D map (`CAL_inj_afr_base`, indexed by RPM and load). When the octane scaler (knock retard 2) is active, the mixture is enriched proportionally via `CAL_inj_afr_adj_per_adv_adj` to protect against detonation.
2. **Injector characteristics.** The fuel mass is converted to pulse width using the injector flow rate (`CAL_inj_flow_rate`) and an efficiency map (`CAL_inj_efficiency`, 3D vs RPM and load).
3. **Corrections.** Three multiplicative adjustment factors are applied:
   - adj1: start enrichment (3D vs stop-coolant and accumulated airflow),
   - adj2: intake air temperature correction (2D vs intake air temperature),
   - adj3: warm-up enrichment (3D vs load and coolant).
4. **Transient enrichment.** Tip-in, tip-out, and DFSO reactive enrichments are added (see Transient Fueling chapter).
5. **Fuel trims.** The STFT and LTFT multiplicative corrections are applied. The idle LTFT additive correction (`LEA_ltft_zone1_adj`, in microseconds) is added to the final pulse width.
6. **Evaporative fuel.** The estimated fuel vapour from the charcoal canister purge is subtracted from the required fuel load before computing the pulse width.

The result is clamped to a minimum of 160 μs. If the pulse width exceeds the available time between combustion events, the duty cycle is reported as 100 % and the excess is carried forward.

Cranking Mode

During cranking, a separate enrichment factor (`CAL_inj_time_adj_cranking`, 2D vs coolant temperature at engine stop) is applied on top of the dead time to provide the rich mixture needed for starting.

DFSO (Decel Fuel Shut-Off)

When the driver lifts off the throttle at speed, the ECU cuts fuel entirely. DFSO engages when all conditions are met: engine speed above `CAL_dfso_engine_speed_enable`, PPS below `CAL_dfso_pps_enable`, coolant above `CAL_dfso_coolant_enable`, and car speed above `CAL_dfso_car_speed_enable`. Each condition has a separate disable threshold with hysteresis for smooth re-engagement.

Rev Limiter

The rev limiter cuts fuel by setting the fuel-cut flag when engine speed exceeds a limit derived from `CAL_revlimit_speed_base` (3D vs coolant and a time-since-hit timer). When faults are present the limit is reduced by `CAL_revlimit_speed_adj_limp_reduced`. Launch control can override with a lower limit (`LEA_tc_launchcontrol_revlimit`) when the car is stationary.

Injection Angle

The injection start angle relative to TDC is looked up (`CAL_inj_angle`, 3D vs RPM and load). A development override (`dev_inj_angle`) can replace it for testing.

9.2 Running Mode

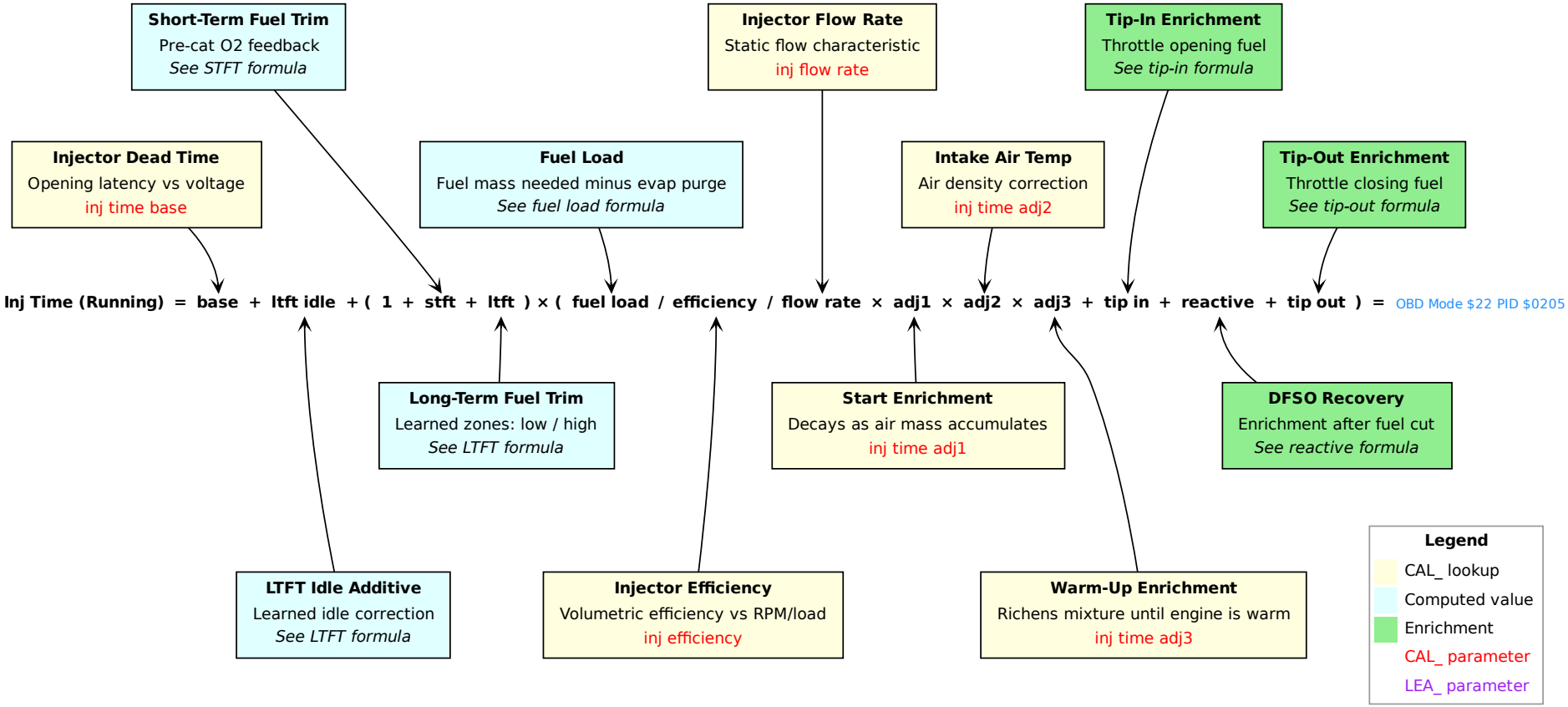


Figure 6: Injection time — running mode

9.3 Fuel Load Detail

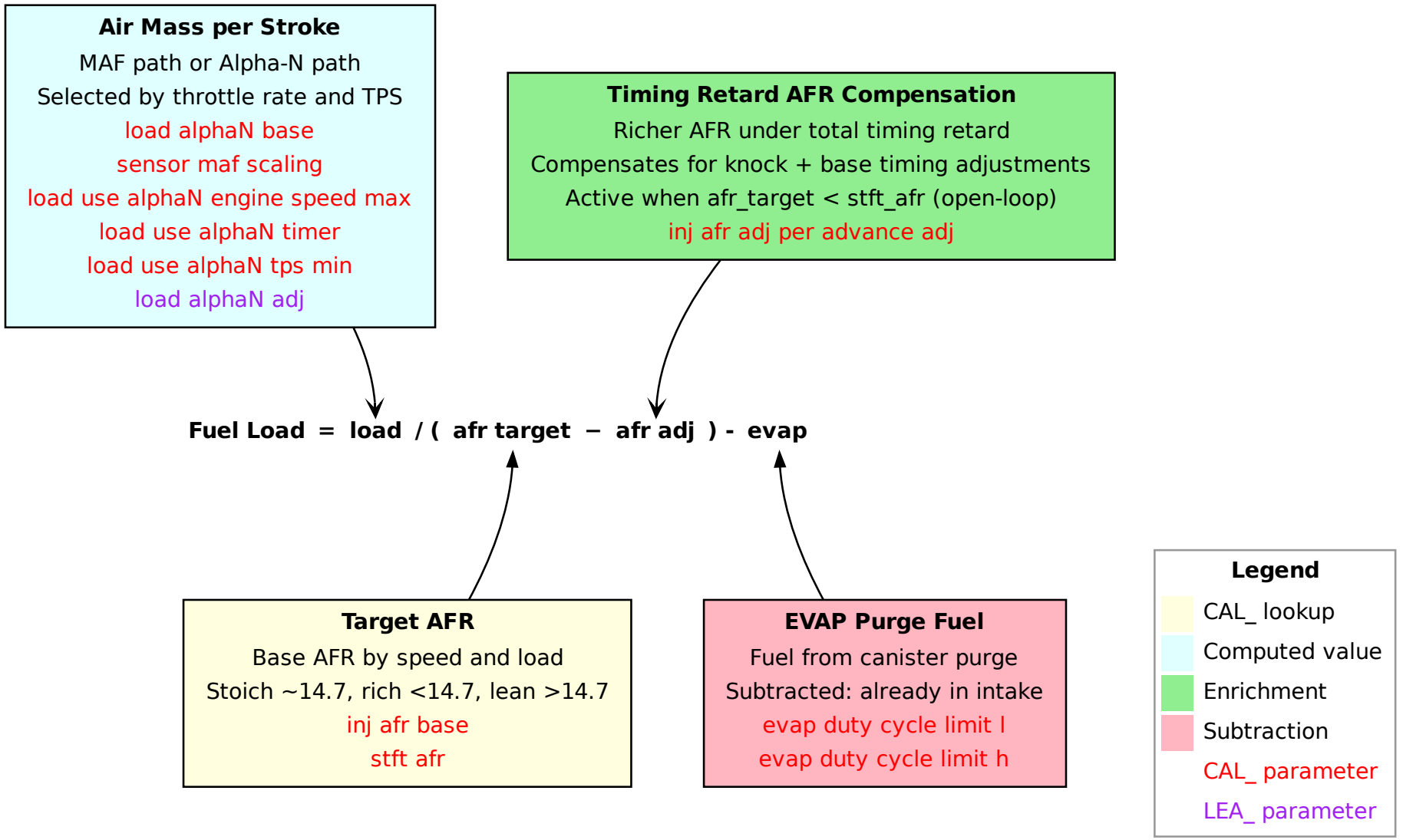


Figure 7: Fuel load calculation detail

9.4 Cranking Mode

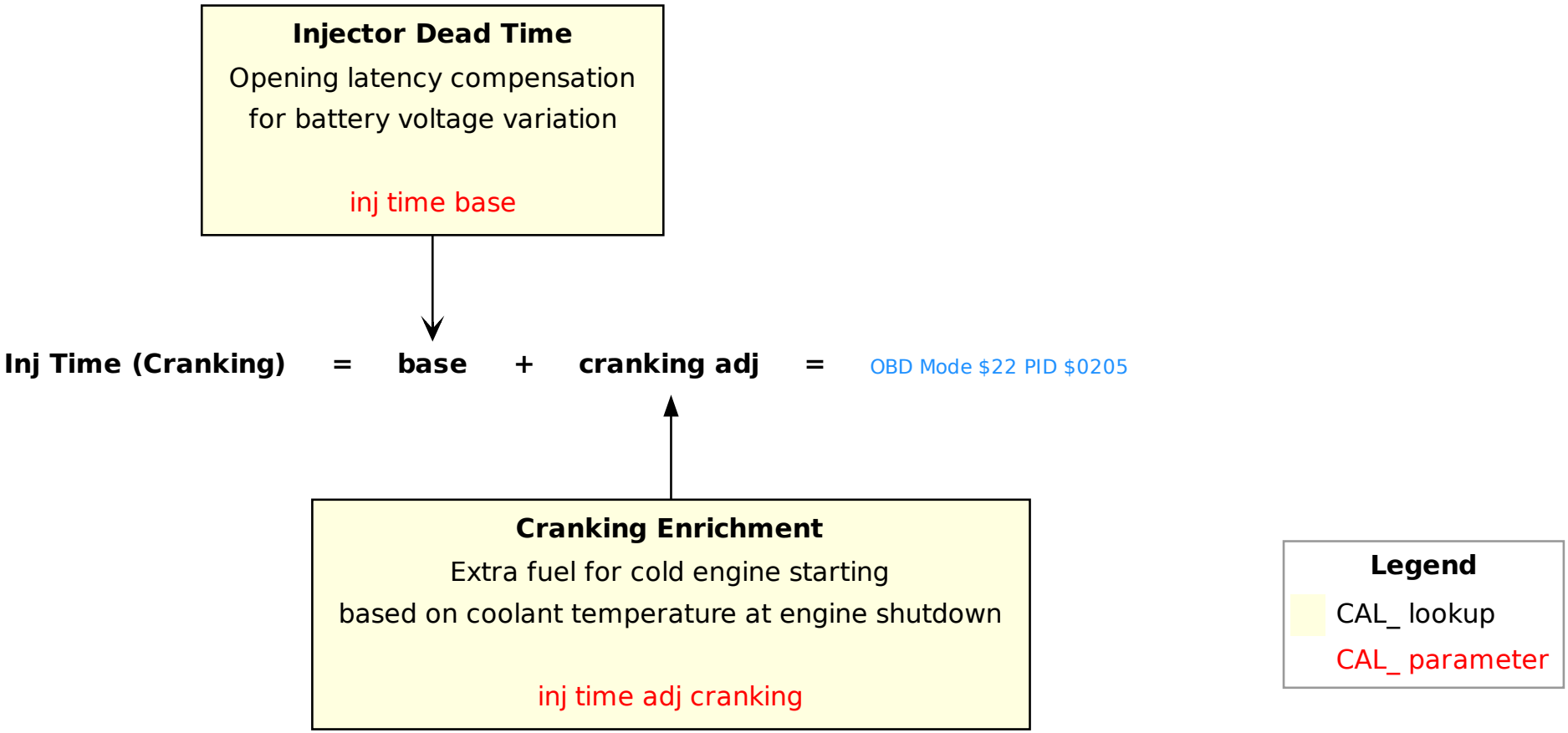


Figure 8: Injection time — cranking mode

9.5 DFSO (Decel Fuel Shut-Off)

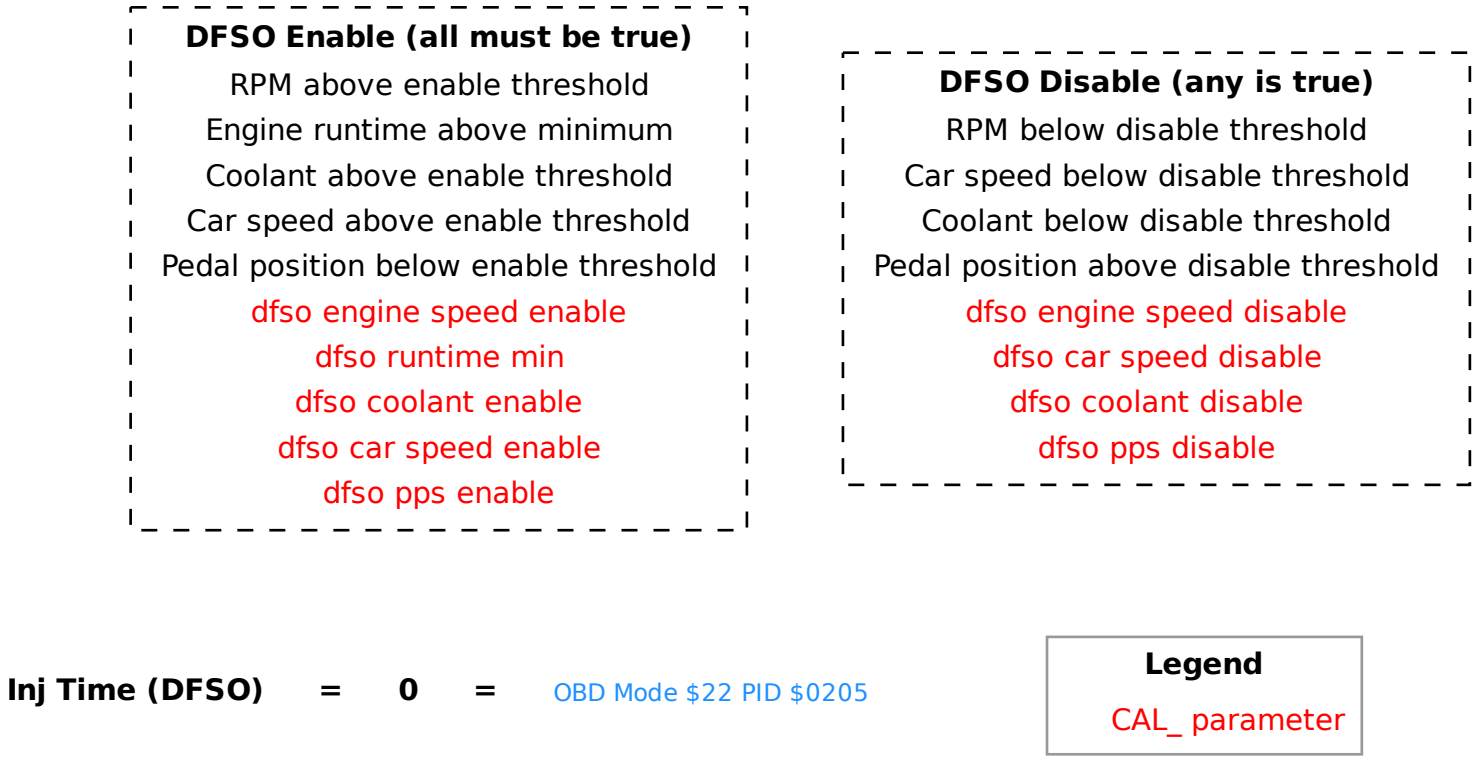


Figure 9: DFSO enable/disable conditions

9.6 Rev Limiter

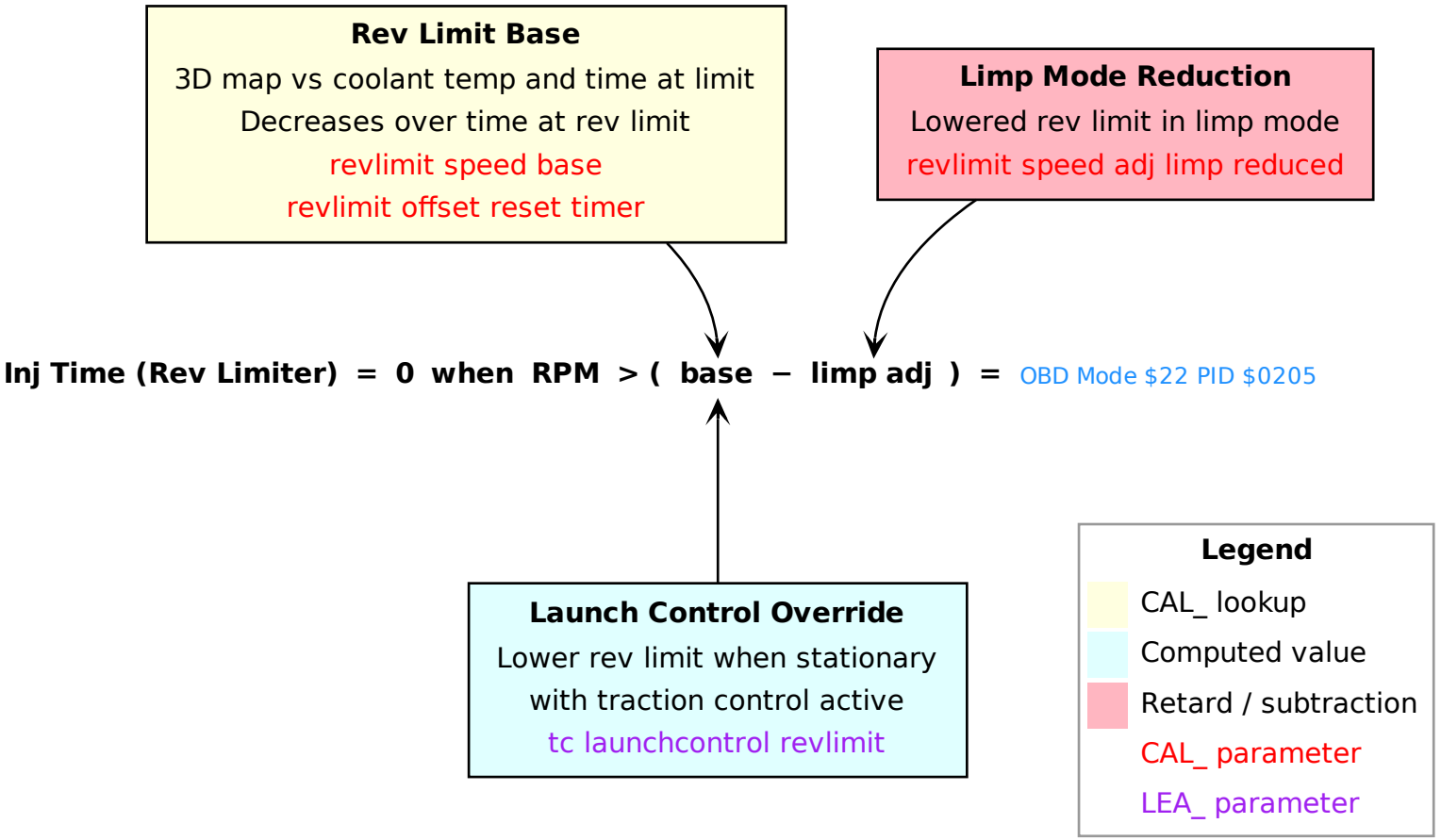


Figure 10: Rev limiter — fuel cut

9.7 Tuning Tips

The most direct way to tune `CAL_inj_efficiency` is to log actual versus target AFR over a representative drive and use MegaLogViewer HD's Histogram/Table generator to build a correction map, as shown in Figure 11. This requires a wideband AFR measurement — either via the wideband patch with a Spartan 2 controller exposing lambda over OBD, or any other valid data-logging method that captures measured AFR alongside the ECU channels.

Axis Setup

Set the X and Y axes in MegaLogViewer HD to engine speed and load with the same resolution as `CAL_inj_efficiency` (32 columns, 32 rows) and the same breakpoint values. The resulting table then maps one-to-one onto the calibration table, making corrections straightforward to transcribe.

Logging Strategy

Log both the measured AFR from the wideband and the target AFR from the ECU. Apply filters to exclude operating conditions where the base table should not be evaluated:

- **Transients**: exclude samples where `dt_tps` exceeds a small threshold — transient fuelling enrichments are handled separately and will pollute the steady-state map.
- **DFSO**: exclude samples during deceleration fuel cut-off.
- **Cold engine**: exclude samples below a minimum coolant temperature — cold-start enrichments are not part of the efficiency table.

Correction Formula

Work in lambda rather than AFR to keep the correction dimensionless and fuel independent. The correction factor for each cell is the ratio of measured lambda to target lambda: a value above 1 means the engine ran lean and the efficiency value must be increased proportionally. To include the long-term fuel trim already learned by the ECU, multiply by the active LTFT zone correction. This combined factor can be entered directly in MegaLogViewer HD as a custom formula, producing a table whose values can be reported straight into `CAL_inj_efficiency` without further arithmetic.

Injector Duty Cycle

The injectors must have enough headroom to deliver additional fuel when enrichment is needed — running close to 100 % duty cycle leaves no margin for correction. Monitoring the duty cycle live is therefore important. It can be computed from RPM (mode $01 PID 0x0C) and injection time (mode $22 PID $0205). Many logging tools (OBD Fusion, OBDLink, etc.) allow combining PIDs in a custom formula, so this value can be displayed as a live gauge without any post-processing.

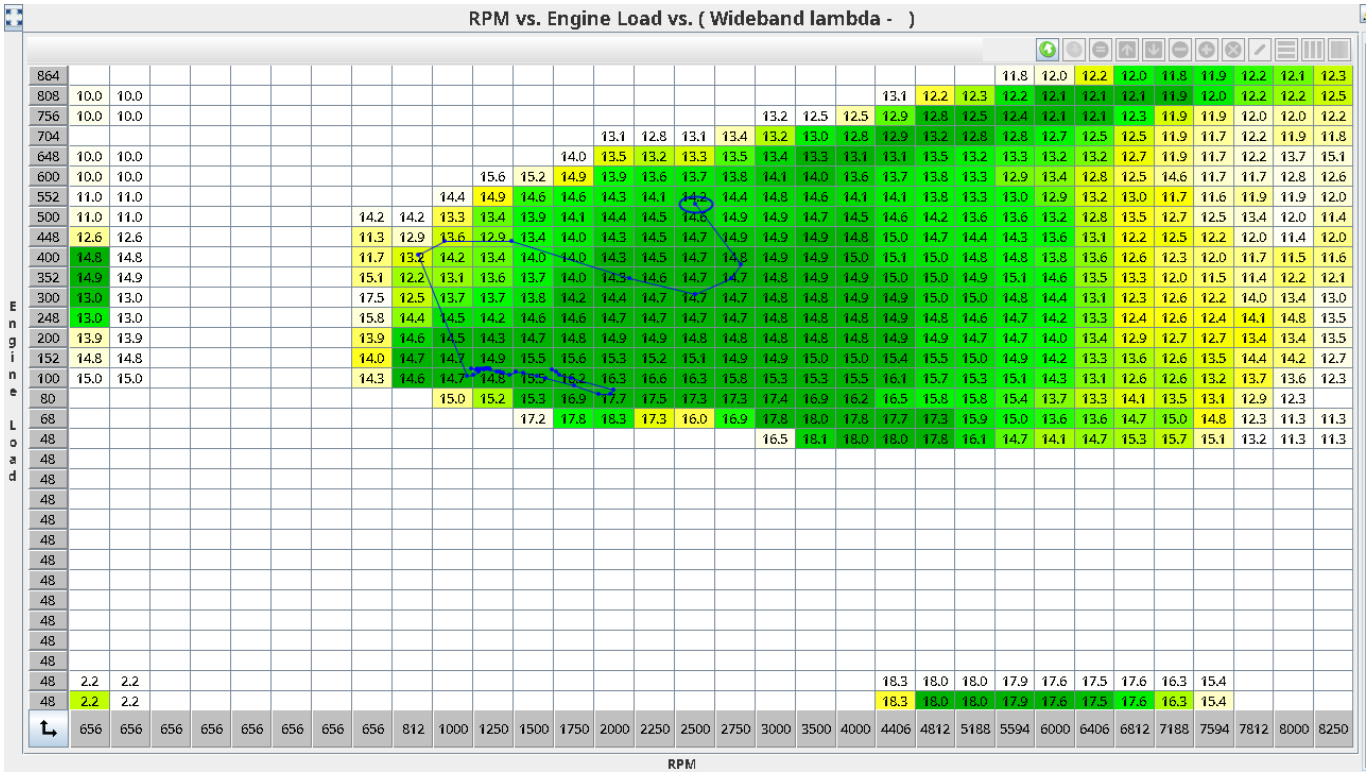


Figure 11: Wideband AFR log — MegaLogViewer HD.

10 Transient Fueling

10.1 Overview

The transient fueling system compensates for the fuel film dynamics that occur during rapid throttle changes. Three independent enrichment terms are computed and added directly to the injection pulse width (in microseconds). Closed-loop fuel control is suspended while any enrichment is active.

TPS Rate Signal

The core input is `dt_tps_injtip`, a peak-hold-with-decay version of the throttle rate of change. Unlike the raw TPS derivative, this signal captures the peak throttle movement and then decays gradually toward zero at a calibrated rate (`CAL_injtip_in_adj2` for positive, `CAL_injtip_out_adj2` for negative, both 2D vs RPM). A threshold (`CAL_injtip_{in,out}_dt_tps_target_2_min`) filters small movements and a limit (`CAL_injtip_{in,out}_dt_tps_target_2_limit`) caps the maximum magnitude. This signal is also used by the ignition system for its transient advance adjustment.

Tip-In Enrichment

When `dt_tps_injtip` is positive (throttle opening), enrichment is the product of the TPS rate, an engine speed adjustment (2D vs RPM), a coolant adjustment (2D vs coolant temperature — cold engines need more enrichment), a per-gear factor (drivetrain inertia varies with gear ratio), and `CAL_injtip_time_base`.

Tip-Out Enrichment

When `dt_tps_injtip` is negative (throttle closing), the magnitude is negated and enrichment is calculated similarly but without the gear factor. Despite the throttle closing, this is still an *enrichment* (added to the pulse width), maintaining combustion stability during the rapid transient.

DFSO and Reactive Enrichment

Deceleration Fuel Shut-Off (DFSO) cuts fuel entirely when the engine is decelerating with the throttle closed. Entry requires all of: engine speed above threshold, sufficient runtime, coolant above minimum, and pedal below threshold. A re-entry delay counter (looked up by PPS) prevents hunting at the boundary. DFSO exits when engine speed drops below a lower threshold, pedal is pressed, or coolant falls below minimum (all with hysteresis).

On DFSO exit, reactive enrichment protects the catalytic converter from the lean exhaust accumulated during fuel cut. The initial magnitude depends on how long DFSO was active (`dfso_count`), looked up from `CAL_injtip_catalyst_adj`. The enrichment then decays exponentially each combustion event, multiplied by `CAL_injtip_catalyst_adj_fadeout` (< 1.0) until it reaches zero. Only then does closed-loop fuel control resume.

10.2 Tip-In Enrichment

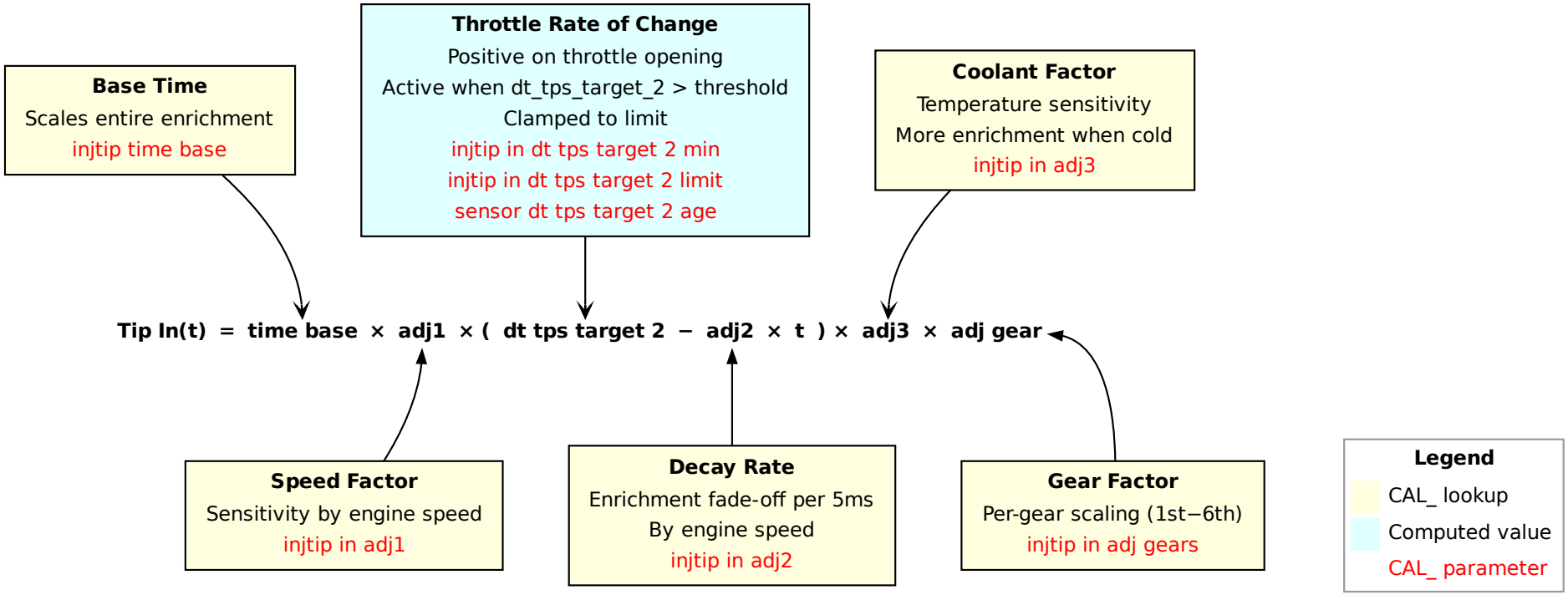


Figure 12: Tip-in enrichment formula

10.3 Tip-Out Enrichment

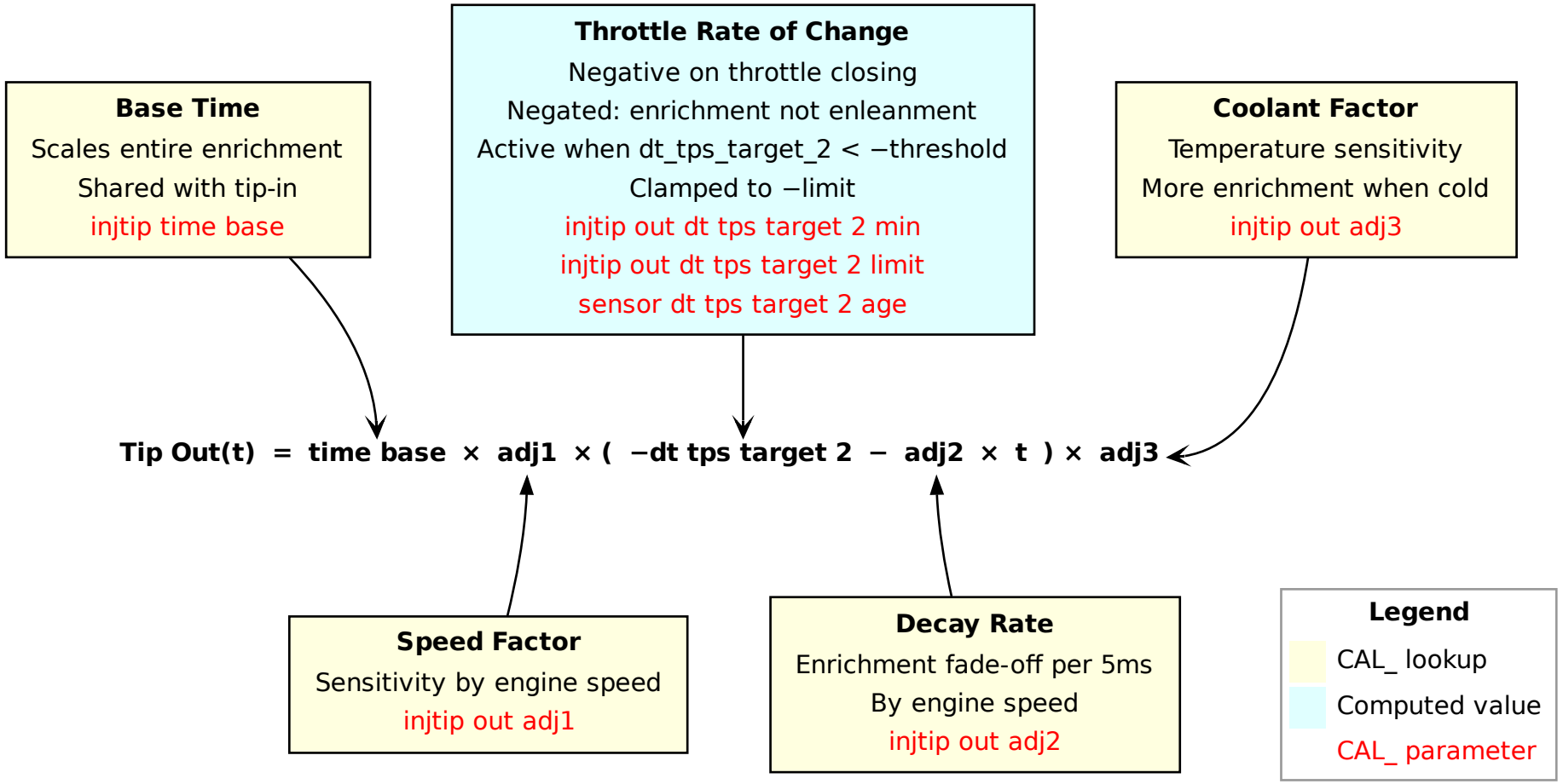


Figure 13: Tip-out enrichment formula

10.4 DFSO Reactive Enrichment

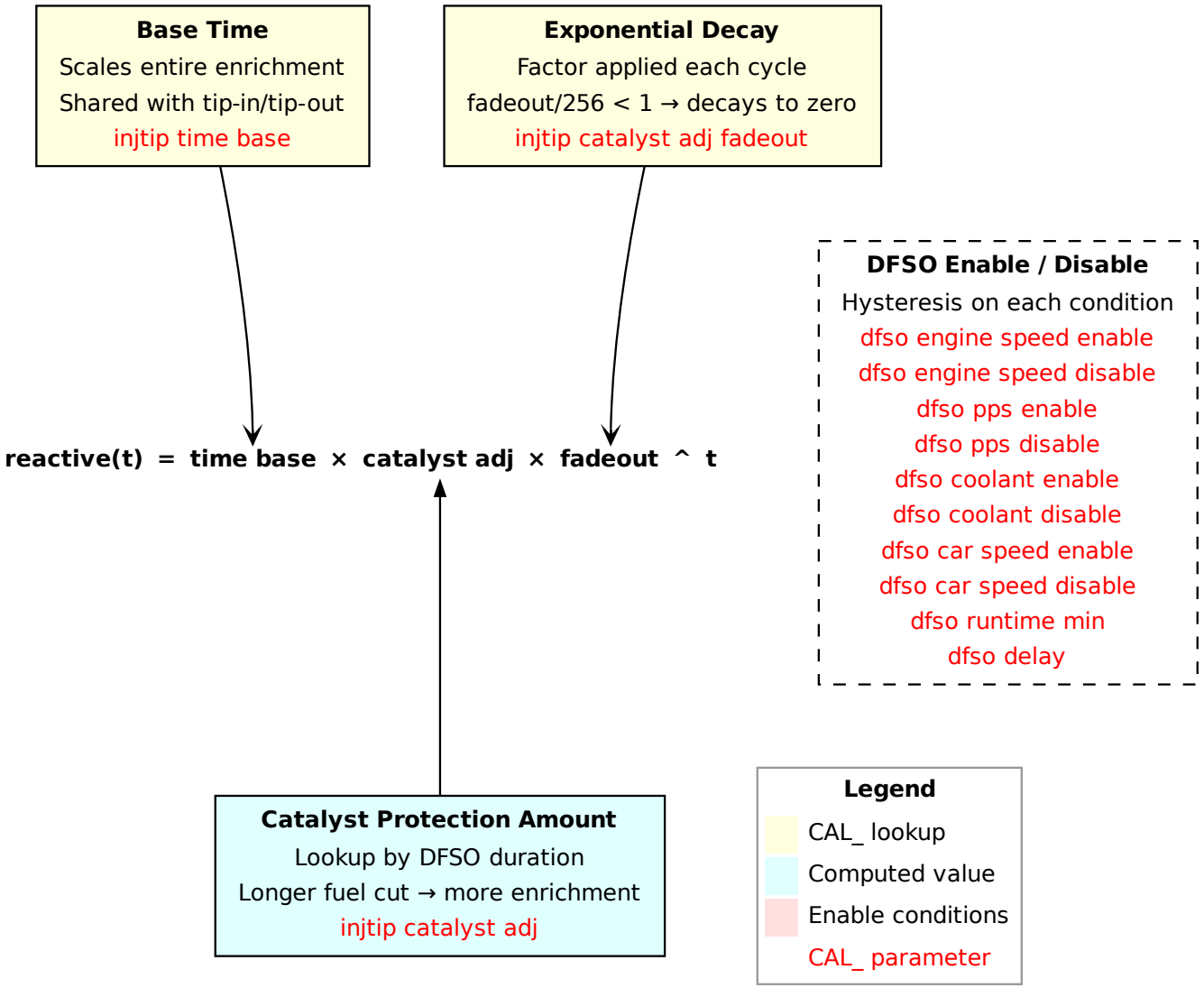


Figure 14: DFSO reactive enrichment formula

11 Closed-Loop Fuel Control

11.1 Overview

Closed-loop fuel control is active only when all of the following conditions are met: the engine has been running longer than `CAL_stft_runtime_min` (which varies with coolant temperature at stop), intake air temperature is above `CAL_stft_engine_air_min`, the target AFR equals stoichiometric (`CAL_stft_afr`), the pre-O2 sensor has been validated (voltage has left the mid-range delimited by `CAL_stft_o2_test_min` and `CAL_stft_o2_test_max` at least once), and there are no active DFSO, transient enrichment, or traction control interventions.

When any of these conditions is not met, the ECU operates in open loop and the STFT is held at zero.

Short-Term Fuel Trim (STFT)

The STFT oscillates around stoichiometric based on the pre-O2 sensor voltage. Two states exist: enriching (O2 below `CAL_stft_stoich_min`) and enleaning (O2 above `CAL_stft_stoich_max`).

On each O2 state transition, an initial jump is applied immediately, followed by a gradual step added at a timed interval. Both the initial jump and the step size come from 3D maps indexed by RPM and load, with separate maps for enrichment and enleanment. At idle, dedicated scalar values are used instead, and the step interval is scaled proportionally to engine speed so that the trim advances synchronously with combustion events.

After a configurable time (`CAL_stft_time_use_adj`), the step size is reduced by a factor (`CAL_stft_enrichment_step_adj` and `CAL_stft_enleanment_step_adj`) to slow the oscillation once the trim has had time to converge.

The STFT is clamped to ±`CAL_stft_limit`.

Long-Term Fuel Trim (LTFT)

The LTFT learns persistent fueling errors from a low-pass filtered version of the STFT (`inj_time_stft_smooth`, filtered with `CAL_stft_smooth_reactivity`). If the smoothed STFT stays consistently above or below calibrated thresholds, the LTFT is stepped in the corresponding direction.

Four zones exist, selected by MAF airflow:

Zone 1 Below `CAL_ltft_zone1_flow_max` and RPM below `CAL_ltft_zone1_engine_speed_max`. The correction (`LEA_ltft_zone1_adj`) is *additive* in microseconds, stepped by `CAL_ltft_zone1_step`.

Zone 2 Between the zone1 and zone2-max flow thresholds, with load within a defined window. The correction (`LEA_ltft_zone2_adj`) is *multiplicative*.

Mid Between the low-max and high-min flow thresholds. The multiplicative correction is linearly interpolated between the low and high learned values based on the current MAF position within the range.

Zone 3 Above `CAL_ltft_zone3_flow_min`. The correction (`LEA_ltft_zone3_adj`) is *multiplicative*.

Even when the ECU is in open loop (STFT inactive), the LTFT corrections are still applied to the injection pulse width — only the learning (stepping of the learned values) is suspended. This means the ECU always benefits from previously learned fuel corrections regardless of operating mode.

LTFT learning is further gated by: coolant and intake air temperature above minimum thresholds, atmospheric pressure above `CAL_ltft_atmo_min`, fuel level above `CAL_ltft_fuel_min`, and no evaporative system pressure drop in progress. The update interval is controlled by `CAL_ltft_time_between_step`. Each zone has independent limits.

11.2 Short-Term Fuel Trim (STFT)

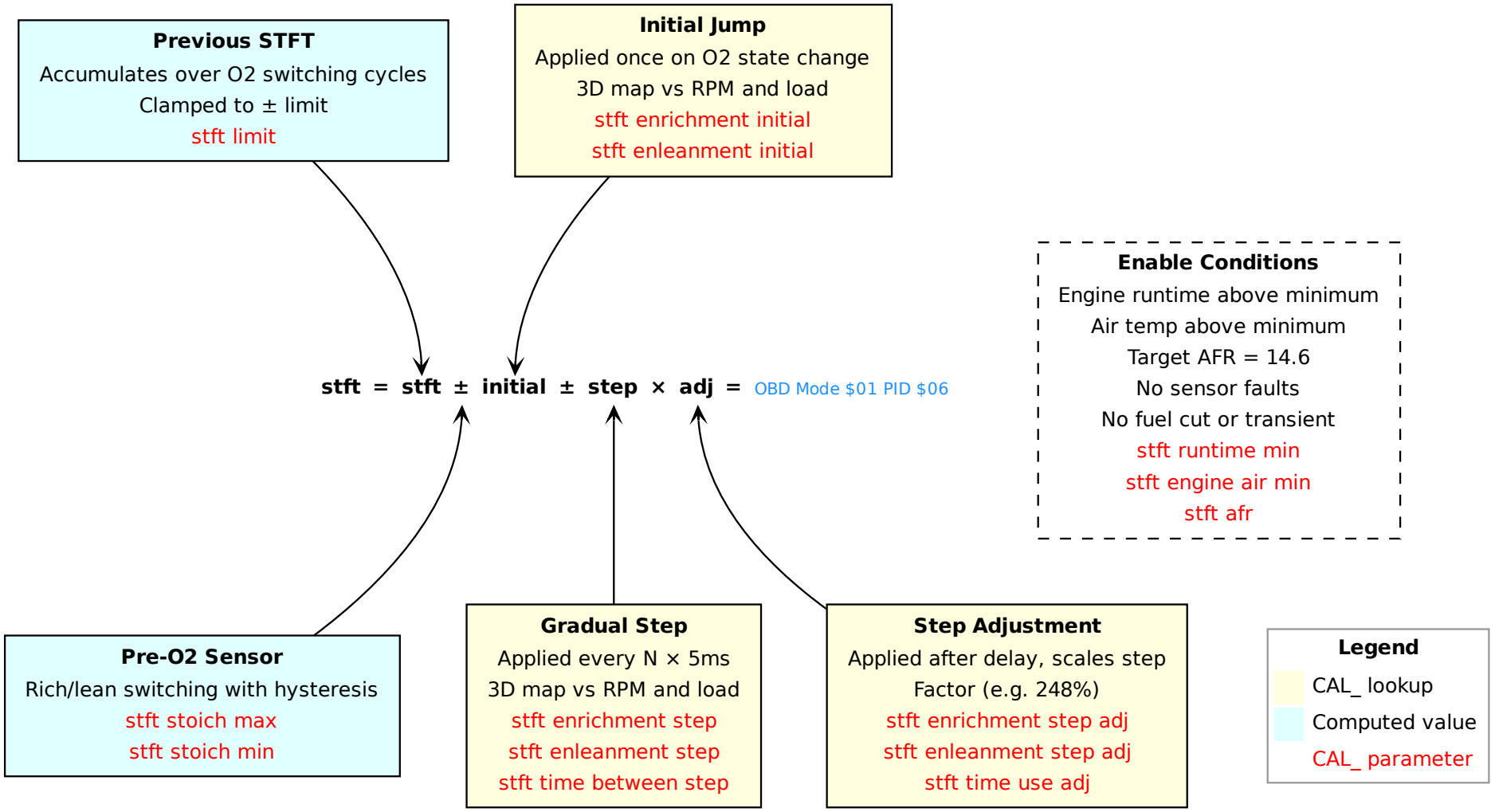


Figure 15: STFT formula

11.3 Long-Term Fuel Trim (LTFT)

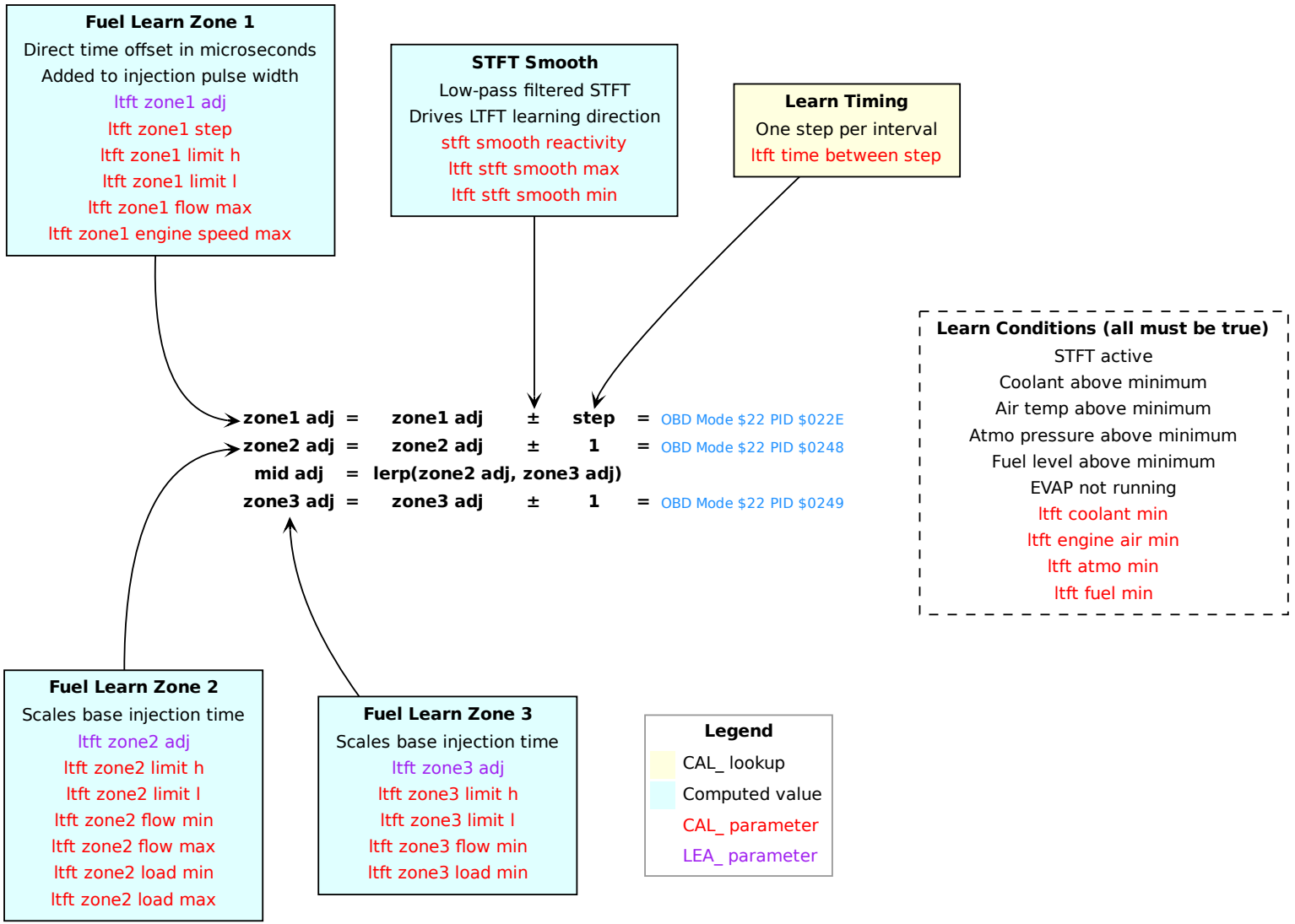


Figure 16: LTFT formula

11.4 Tuning Tips

Figure 17 shows the `CAL_inj_afr_base` table from a Lotus Exige S Cup 260 calibration, colorized by the active LTFT zone at each cell. The colour coding matches the zone boundaries defined by the flow and load thresholds described above: **blue** for Zone 1 (idle), **green** for Zone 2 (part-load), **pink** for Zone 3 (high-load), and **yellow** for the dead zone between Zone 2 and Zone 3 where the two corrections are interpolated. Bold values mark cells calibrated to the stoichiometric target (14.6 AFR), meaning the ECU will run in closed loop at those points and the STFT can drive LTFT learning.

A key design constraint is that *each zone must contain enough cells at 14.6* to give the ECU sufficient closed-loop operating time to build a reliable correction for that zone. A zone with too few stoichiometric cells will rarely enter closed loop there, leaving its learned correction at the reset default and therefore inaccurate. As the figure illustrates, the Cup 260 calibration provides wide stoichiometric coverage across all three zones: the entire idle region (Zone 1), a broad part-load band (Zone 2), and a high-load band before the enrichment ramp begins (Zone 3).

Zone 3 correction applies at WOT. At wide-open throttle the AFR target drops well below stoichiometric, the STFT is suspended and the ECU enters open loop — but `LEA_ltft_zone3_adj` is still applied multiplicatively to the base injection time. A miscalibrated Zone 3 LTFT therefore directly affects WOT fuelling, even though no learning takes place there. **It is essential that the Zone 3 stoichiometric cells in the AFR table are reachable under normal part-throttle driving so that the ECU can converge on the correct correction before it is relied upon at full load.**

Note: this is one of the most commonly overlooked aspects of closed-loop fuel tuning. A competent tuner should produce exactly this kind of Excel Sheet — overlaying the AFR targets with the LTFT zone boundaries — to verify that each zone, and Zone 3 above all, has sufficient stoichiometric coverage for learning to occur. In practice, many tuners focus exclusively on the WOT enrichment values and never verify Zone 3 LTFT convergence. Some are not even aware that the Lotus ECU operates with multiple independent LTFT zones at all.

Load \ RPM	< 1500	1500	1500	1500	2000	2500	3000	3500	4000	4812	5593	6000	6406	6812	7187	7593
< 224	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	13.5	13.5	13.5	13.0
224	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	13.5	13.5	13.5	13.0
224	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	13.5	13.5	13.5	13.0
224	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	13.5	13.5	13.5	13.0
224	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	13.5	13.5	13.5	13.0
224	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	13.5	13.5	13.5	13.0
224	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	13.5	13.5	13.5	13.0
304	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	13.5	13.5	13.5	13.0
352	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.3	13.5	13.5	13.2	13.0
400	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.3	14.0	13.5	13.5	13.2	13.0
448	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	12.8	12.8	12.8	12.7	12.5	12.5
476	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	14.6	12.8	12.8	12.8	12.7	12.5	12.5
500	14.6	14.6	14.6	14.6	14.0	14.0	14.0	14.0	13.5	12.8	12.8	12.8	12.8	12.7	12.5	12.5
552	14.6	14.6	14.6	14.6	13.8	13.8	13.8	13.8	13.5	12.8	12.8	12.8	12.8	12.7	12.5	12.5
756	14.6	14.6	14.6	14.6	13.5	13.5	13.5	13.5	13.3	12.6	12.6	12.6	12.6	12.4	12.4	12.4
864	14.6	14.6	14.6	14.6	13.0	13.0	13.0	13.0	13.0	12.5	12.5	12.5	12.5	12.4	12.0	11.9

	LTFT Learning / Authority Zones						
		Flow		Load		RPM	
	Zones	min	max	min	max	max	Remarks
	1 (Idle)		7			1100	
	2	10	24	162	378		Learning only in closed loop
	3	38		297			Learning only in closed loop
14.6	Closed loop						

	LTFT Authority Zones			
		Flow		
	Zones	min	max	Remarks
	1 (Idle)			Always applied
	2	10	24	
	3	38		
	2-3	24	38	Interpolated

Figure 17: AFR Target - Learning Zones (Lotus Exige S Cup 260)

12 Ignition

12.1 Overview

The engine uses four individual ignition coils (coil-on-plug). Dwell and fire timing are handled in hardware by TPU-A channels 1–4; the main CPU only computes advance angles and dwell duration.

Dwell Time

The dwell time is looked up from a 3D table indexed by engine speed and battery voltage (`CAL_ign_dwell_base`). At high RPM, if the required dwell would overlap the previous cylinder's fire event, the dwell is shortened automatically.

Operating Modes

The ECU selects one of four ignition modes based on engine state:

Cranking A simple 2D vs coolant temperature at the time the engine was stopped. Active for `CAL_ign_cranking_runtime_max` after the engine starts.

Running Full advance calculation: base map plus adjustments for intake air temperature, transient throttle, RPM, and accumulated airflow. Two base maps exist (low cam and high cam) and the correct one is selected by the current VVL state.

Idle A separate formula based on coolant temperature, idle speed error, and rate of change of engine speed. Includes an AC compressor offset.

DFSO A single fixed advance value (`CAL_ign_adv_dfso`), bypassing the normal calculation entirely.

Per-Cylinder Final Stage

After the mode-specific formula produces a final advance value, a 5 ms smoothing task ramps the advance in configurable steps (sized by RPM and PPS) to avoid abrupt timing changes. The smoothed value then passes through a per-cylinder stage that applies:

- A per-cylinder trim offset (`CAL_ign_adv_adj_cyl`).
- Knock retard 1: immediate timing retard from the knock detection system, recovered gradually in the post-fire ISR phase.
- Knock retard 2 (octane scaler): a long-term learned factor (0–100 %) per cylinder that blends the base ignition map with a conservative knock-safe map (low octane map).

The result is clamped to `CAL_ign_adv_limit_l` and loaded into the TPU parameters for hardware-timed execution.

12.2 Running Mode

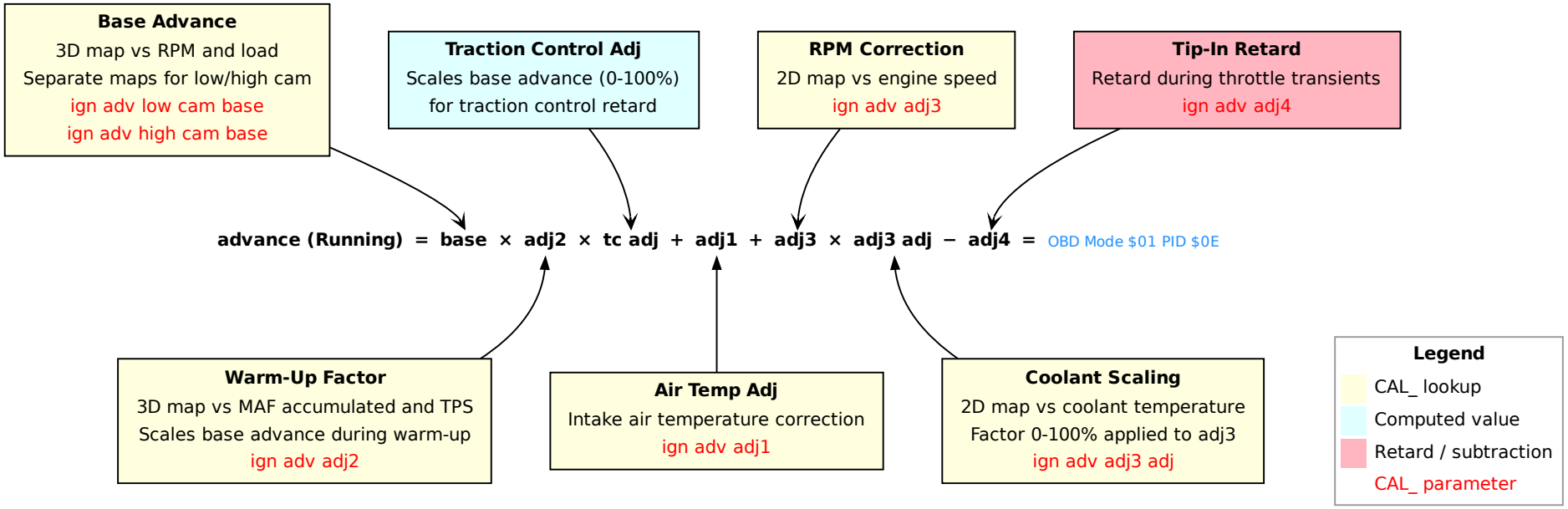


Figure 18: Ignition advance — running mode

12.3 Per-Cylinder Final Stage

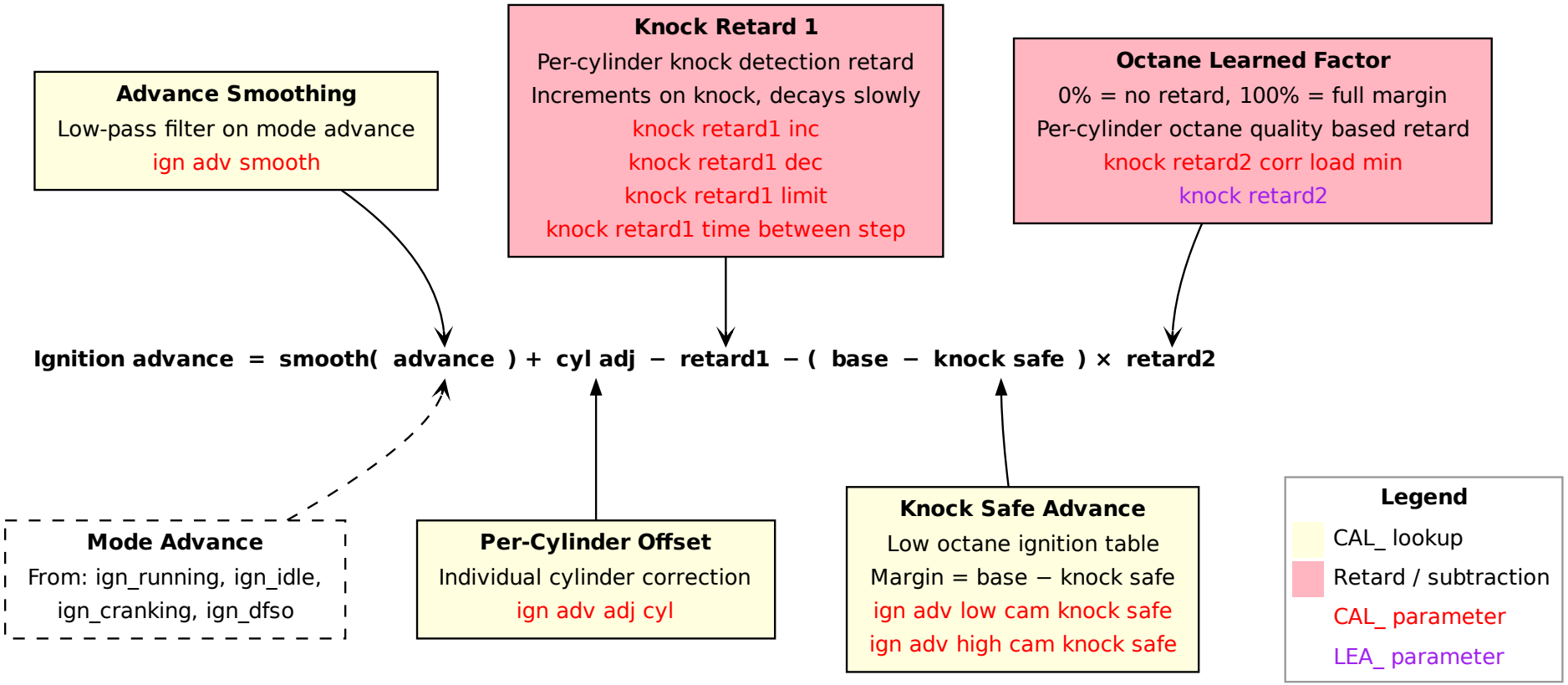


Figure 19: Ignition advance — per-cylinder final stage

12.4 Idle Mode

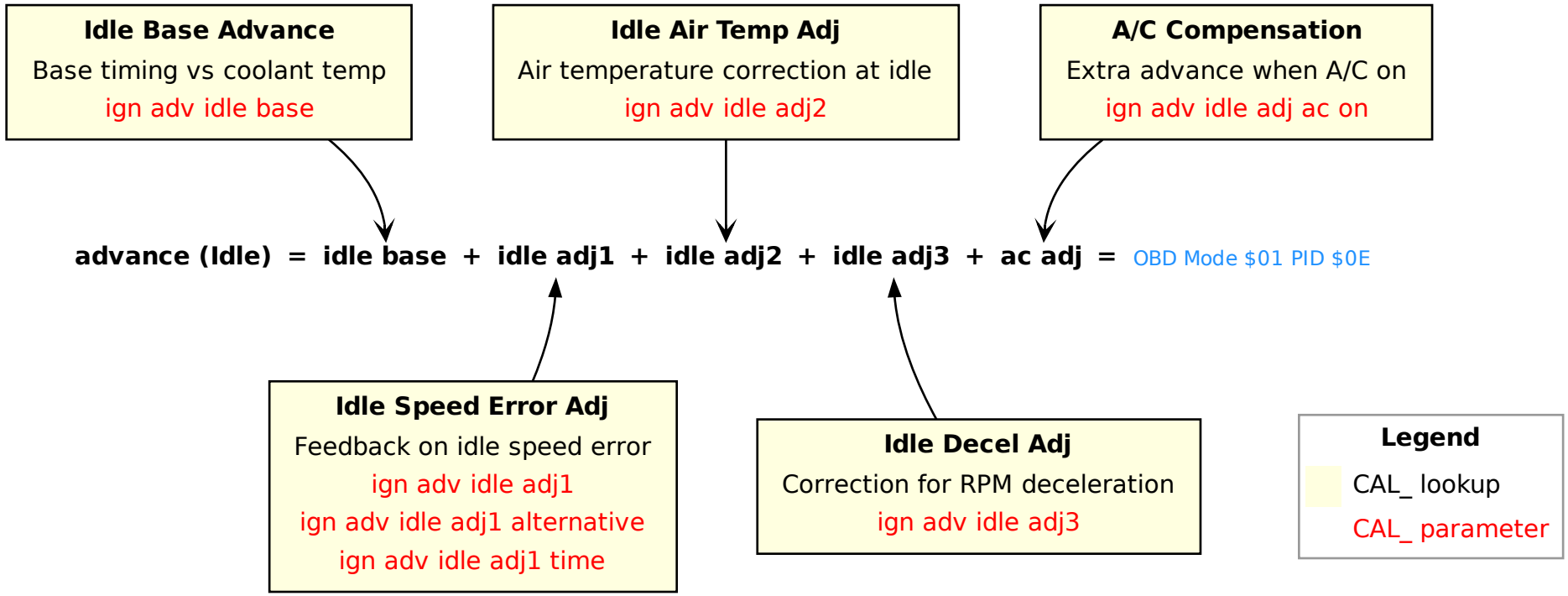


Figure 20: Ignition advance — idle mode

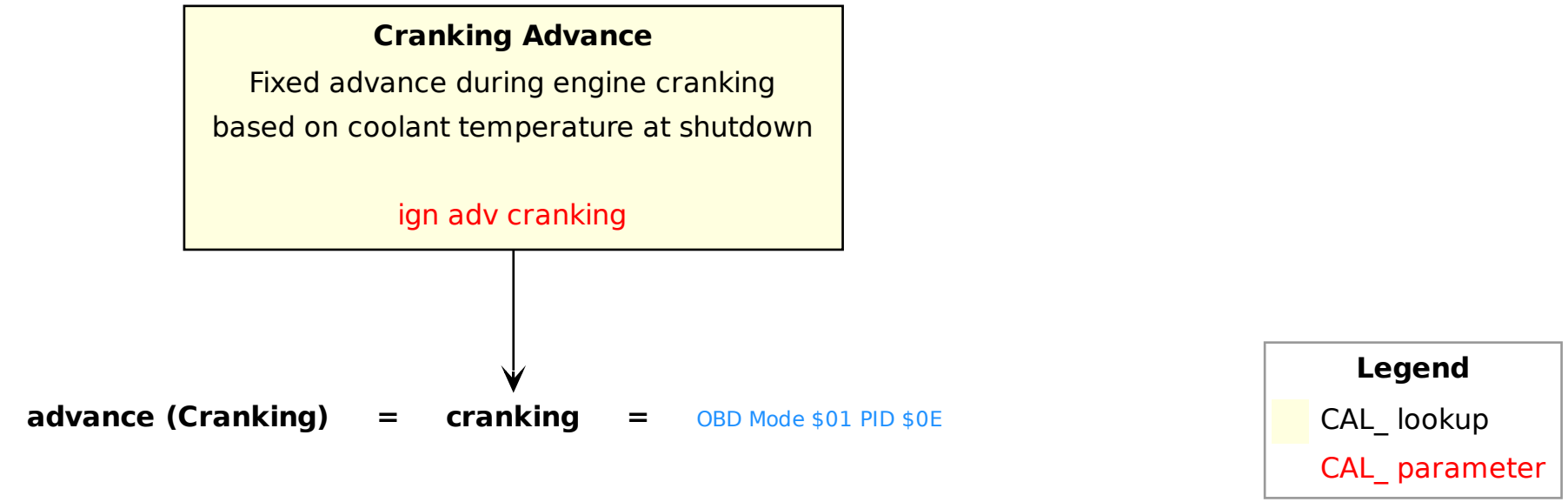


Figure 21: Ignition advance — cranking mode

12.6 DFSO Mode

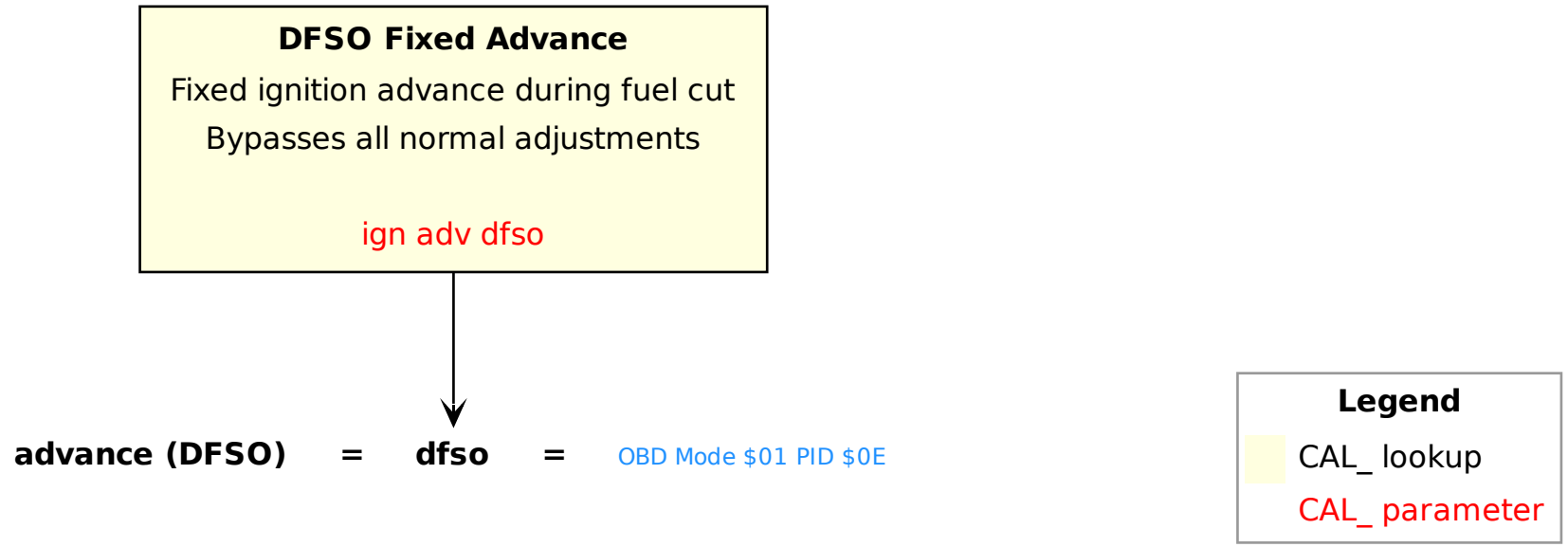


Figure 22: Ignition advance — DFSO mode

12.7 Traction Control Retard

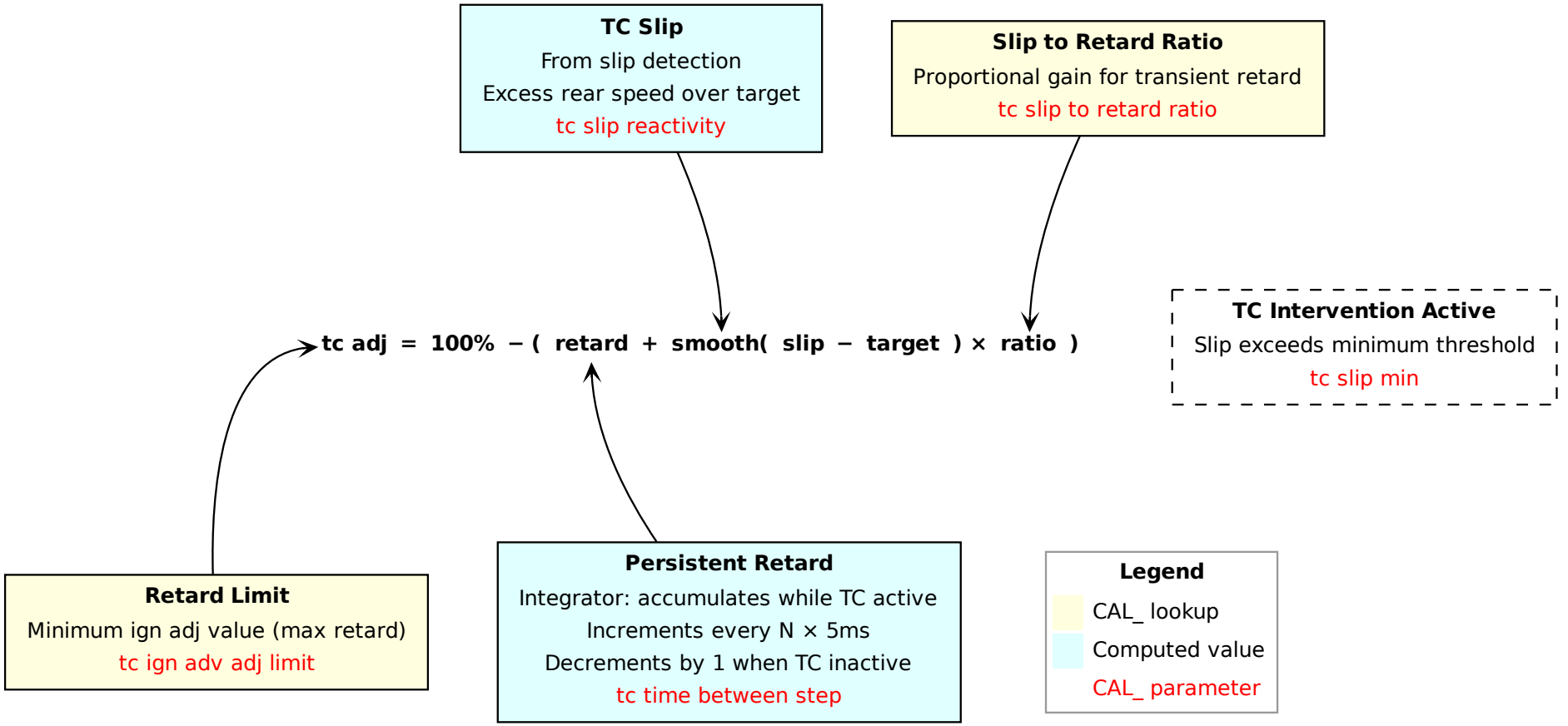


Figure 23: Ignition retard from traction control

12.8 Tuning Tips

The high-cam ignition map (3D vs RPM and load) is only used when VVL is engaged *and* RPM is below `CAL_vvl_rpm_enable` *and* load is above `CAL_vvl_high_load_enable`. In all other cases — including high cam at high RPM — the low-cam map (also 3D vs RPM and load) is used.

On the Elise, `CAL_vvl_rpm_high_load_enable` is set to the same value as `CAL_vvl_rpm_enable`, which means VVL can only engage at the normal RPM threshold and the high-cam ignition map is effectively never used. The same applies to its knock-safe variant. The original Elise calibration contains garbage values in both tables (`CAL_ign_adv_high_cam_base` and `CAL_ign_adv_high_cam_knock_safe`). If you lower `CAL_vvl_rpm_high_load_enable` to engage VVL earlier under high load, both high-cam maps *will* be used in the RPM range between `CAL_vvl_rpm_high_load_enable` and `CAL_vvl_rpm_enable` — populate them with correct values first. Garbage values in this range will cause a dramatic and sudden power loss that resembles a rev limiter firing between the two threshold RPMs.

The ignition advance reported over the standard OBD interface (PID $0E) is `ign_adv_final`, which is computed before the per-cylinder final stage. Per-cylinder trim (`CAL_ign_adv_adj_cyl`), knock retard 1 and knock retard 2 (octane scaler) are all applied *after* this value is set. This means that any knock-induced retard is invisible to a standard OBD logger — the displayed advance will look clean even while the ECU is actively pulling timing on individual cylinders.

Figure 24 shows a live-tuning tool in use on a dyno. The ignition advance table is displayed with the cell currently active highlighted in real time, allowing timing to be increased or decreased on the fly using the keyboard. The octane scalers are visible alongside, giving an immediate view of knock activity on each cylinder. AFR can also be monitored precisely when the wideband patch is applied and a Spartan 2 wideband controller is connected.

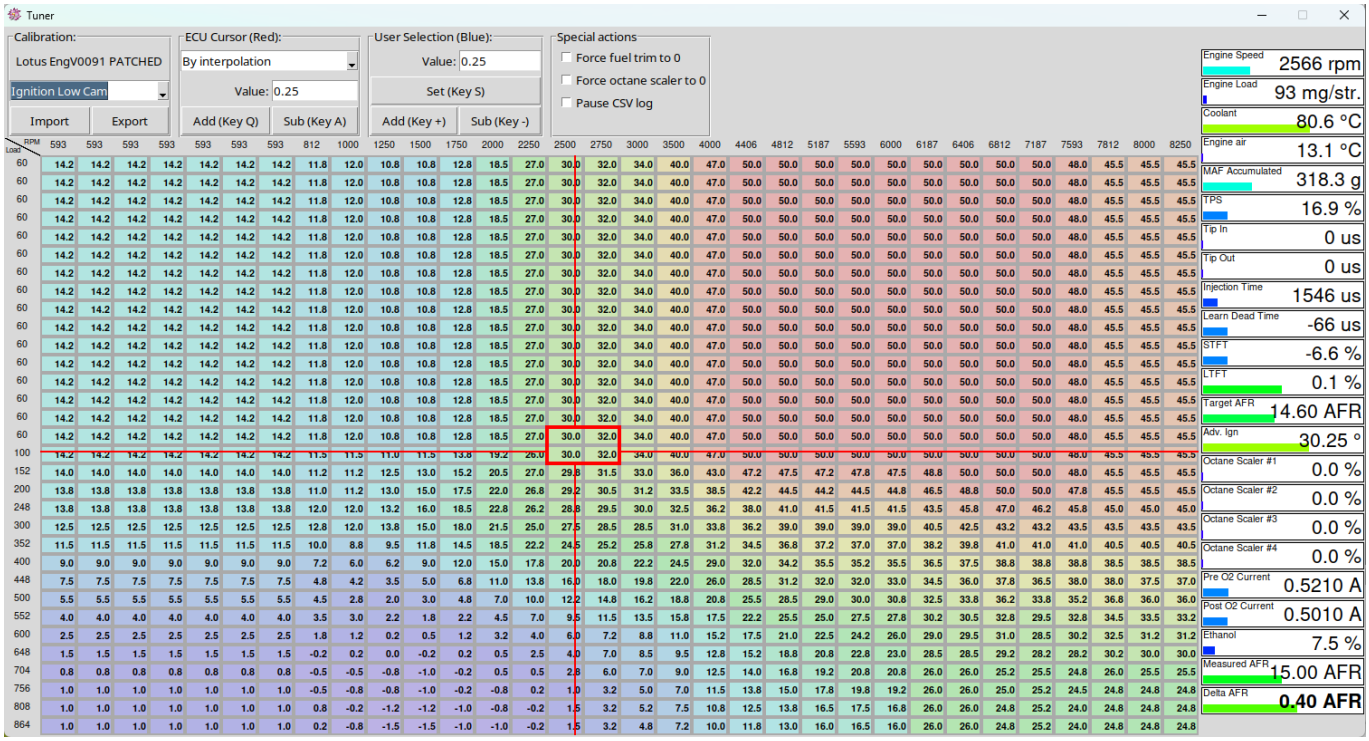


Figure 24: Live ignition map editing over CAN-Bus RAM access on a dyno.

12.9 Igniter Feedback (IGF)

The ECU uses the IGF (Igniter Feedback) signal from the ignition coil pack to confirm that each coil fired successfully.

Each ignition event generates three TPU-A interrupts per cylinder:

1. **Dwell start.** The TPU channel function status reaches 0xE, indicating the crank angle for dwell onset. The ISR programs the dwell duration and resets its phase counter.
2. **Fire.** The phase counter is below 2 (first two non-dwell interrupts). The ISR sets a pending-feedback bit for this cylinder. Before doing so, it checks whether any *other* cylinders' pending bits are still set — meaning those coils fired but their IGF confirmation never arrived. Those cylinders are flagged in a separate missed-feedback register.
3. **Post-fire.** The phase counter reaches 2 and is reset. The knock retard 1 recovery timer for this cylinder is processed (see Knock Detection), and the next ignition event is scheduled.

The IGF signal is received on TPU-B channel 15. When the interrupt fires, it clears the pending-feedback bit for the cylinder that most recently fired. If the bit is cleared before the next cylinder's fire phase, the coil is confirmed working.

Every 200 ms, `misfire_monitor_200ms` evaluates the missed-feedback register. Each cylinder's bit is fed into a diagnostic channel with confirm/clear thresholds. Once a fault is confirmed the corresponding DTC is set:

P0351 Ignition coil A primary/secondary circuit.

P0352 Ignition coil B primary/secondary circuit.

P0353 Ignition coil C primary/secondary circuit.

P0354 Ignition coil D primary/secondary circuit.

Both the pending-feedback and missed-feedback registers are cleared when the engine stops.

12.10 Combustion Misfire Detection

Separate from the IGF coil-failure check, the ECU detects actual combustion misfires by monitoring the time each cylinder takes to complete its power stroke. A misfiring cylinder produces a longer stroke period than its neighbours because the piston is not driven by combustion pressure.

Detection Pipeline

At every TDC event the crank-position ISR processes each cylinder in firing order (1, 3, 4, 2) through three successive stages that progressively isolate the misfire signal from noise:

1. **Smoothing.** A mild filter removes the effect of RPM acceleration between consecutive strokes of the same cylinder, giving a smoothed stroke time (`misfire_stroke_time_smooth`).
2. **Inter-cylinder difference.** The smoothed stroke time of the current cylinder is subtracted from the raw stroke time of the previous cylinder in the firing sequence (`misfire_stroke_cyl_diff`). This removes the absolute stroke duration and leaves only the relative variation between adjacent cylinders.
3. **Baseline correction.** A 16×4 learned table (`LEA_misfire_stroke_time`, one row per RPM band, one column per cylinder) captures the normal inter-cylinder variation for this specific engine. Subtracting the table value gives the residual deviation (`misfire_stroke_diff_1`). The table is adapted continuously during DFSO, slowly tracking natural stroke-time asymmetry between cylinders.

4. **Adjacent-cylinder normalisation.** The residual of the previous cylinder is subtracted from the current residual (`misfire_stroke_diff_2`). This cancels any engine-wide RPM disturbance that affects all cylinders equally, leaving a signal sensitive only to single-cylinder anomalies.

Two-Tier Thresholds

Two independent thresholds are applied at each TDC event:

Emission misfire The baseline-corrected residual (`misfire_stroke_diff_1`) is compared against `misfire_threshold`. Events are counted per cylinder over a rolling window of 2000 strokes; at the end of each window the counts are halved and stored in `LEA_misfire_count`. Sustained rates above the OBD limits set P0301–P0304.

Catalyst-damage misfire The normalised residual (`misfire_stroke_diff_2`) is compared against `misfire_cat_threshold`. A per-cylinder countdown timer (`misfire_cat_timer`) decrements each time the threshold is exceeded and resets to `misfire_cat_timer_max` otherwise. A separate 400-stroke window accumulates `misfire_cat_count`. The normalised signal is used here because single severe misfires must be detected even when global RPM is fluctuating.

Array Index Order

All misfire arrays follow firing order: index 0 = cylinder 1, index 1 = cylinder 3, index 2 = cylinder 4, index 3 = cylinder 2. The OBD dev-mode PIDs $0234–$0237 report `misfire_count[0..3]` in the same order (see Table 8).

13 Knock Detection

13.1 Overview

Understanding the knock noise baseline for your specific engine setup is essential before tuning ignition timing aggressively.

Hardware and Signal Processing

A piezoelectric knock sensor is mounted on the engine block. The signal is band-pass filtered and amplified by the L9119D knock IC, then read from QADC-B and compared to an exponentially-smoothed baseline (per cylinder). The peak ratio is computed as `knock_signal * 10 / (baseline + 1)`. When the ratio exceeds `CAL_knock_peak_threshold` (3D vs RPM and load), knock is detected and the excess above the threshold is recorded as a severity measure (`knock_peak_over_threshold`).

The sensor amplifier gain is configured per cylinder via SPI, using four independent maps (`CAL_knock_sensitivity_cylinder1...4`, each 3D vs RPM and load). When knock is not detected, the baseline tracks the signal via `CAL_knock_signal_reactivity`; when knock is detected, the baseline is raised by `CAL_knock_signal_fadeout_step` to prevent the same event from re-triggering.

Knock Window

The crank angle window during which the sensor signal is sampled is defined by two maps: `CAL_knock_window_start` and `CAL_knock_window_width` (both 3D vs RPM and load). A minimum width of 5° is enforced. The window parameters are computed per cylinder and loaded into the TPU for hardware-timed acquisition.

Enable Conditions

Knock retard is only active when all of the following are met:

- Engine load above `CAL_knock_retard_load_min` (2D vs speed, with hysteresis via `CAL_knock_load_margin`).
- Engine speed within `CAL_knock_retard1_engine_speed_min...max` (with hysteresis via `CAL_knock_speed_margin`).
- Coolant temperature above `CAL_knock_retard1_coolant_min`.
- VVT deviation within `CAL_knock_retard1_vvt_diff_max` (avoids false detection during cam phasing transients).
- TPS transient blanking timer expired — rapid throttle changes (`dt_tps_target_2` exceeding `CAL_knock_retard1_dt_tps_target_2_max`) blank detection for `CAL_knock_retard1_dt_tps_target_2_time`.

When any condition is lost, retard1 is cleared immediately.

Retard 1 — Immediate Retard

On knock detection, `knock_retard1` is increased by a value looked up from `CAL_knock_retard1_inc` (indexed by knock severity), clamped to `CAL_knock_retard1_limit`. This is tracked per cylinder.

Recovery is paced by `CAL_knock_retard1_time_between_step` (counted down in 50 ms increments): each time the timer reaches zero, the post-fire ISR decrements retard1 by `CAL_knock_retard1_dec` and reloads the timer. This repeats until retard1 reaches zero. If new knock is detected during recovery, retard1 is increased again and the timer restarts.

Retard 2 — Long-Term Octane Adaptation

Lotus calls this correction the *Octane Scaler*. Its purpose is to adapt timing to the octane rating of the fuel in the tank: the knock-safe map can therefore be thought of as a *low-octane* ignition table — the timing schedule the ECU falls back to when running on poor-quality fuel.

`LEA_knock_retard2[0...3]` is a per-cylinder learned factor (0–100 %) that blends the base ignition map toward the knock-safe map. It is persisted in EEPROM across power cycles.

When retard1 is active (knock occurring), `LEA_knock_retard2` increases proportionally to retard1 via `CAL_knock_retard2_corr_inc_coef`. When retard1 is zero (no knock), it slowly decays by `CAL_knock_retard2_corr_dec_step`. This makes the ECU progressively retard timing for cylinders that repeatedly knock, adapting to fuel quality and engine condition.

The actual retard2 applied is the learned fraction of the difference between the base ignition map and the knock-safe map. Retard2 learning requires load above a separate threshold (`CAL_knock_retard2_corr_load_min`).

13.2 Knock Retard 1 — Immediate Retard

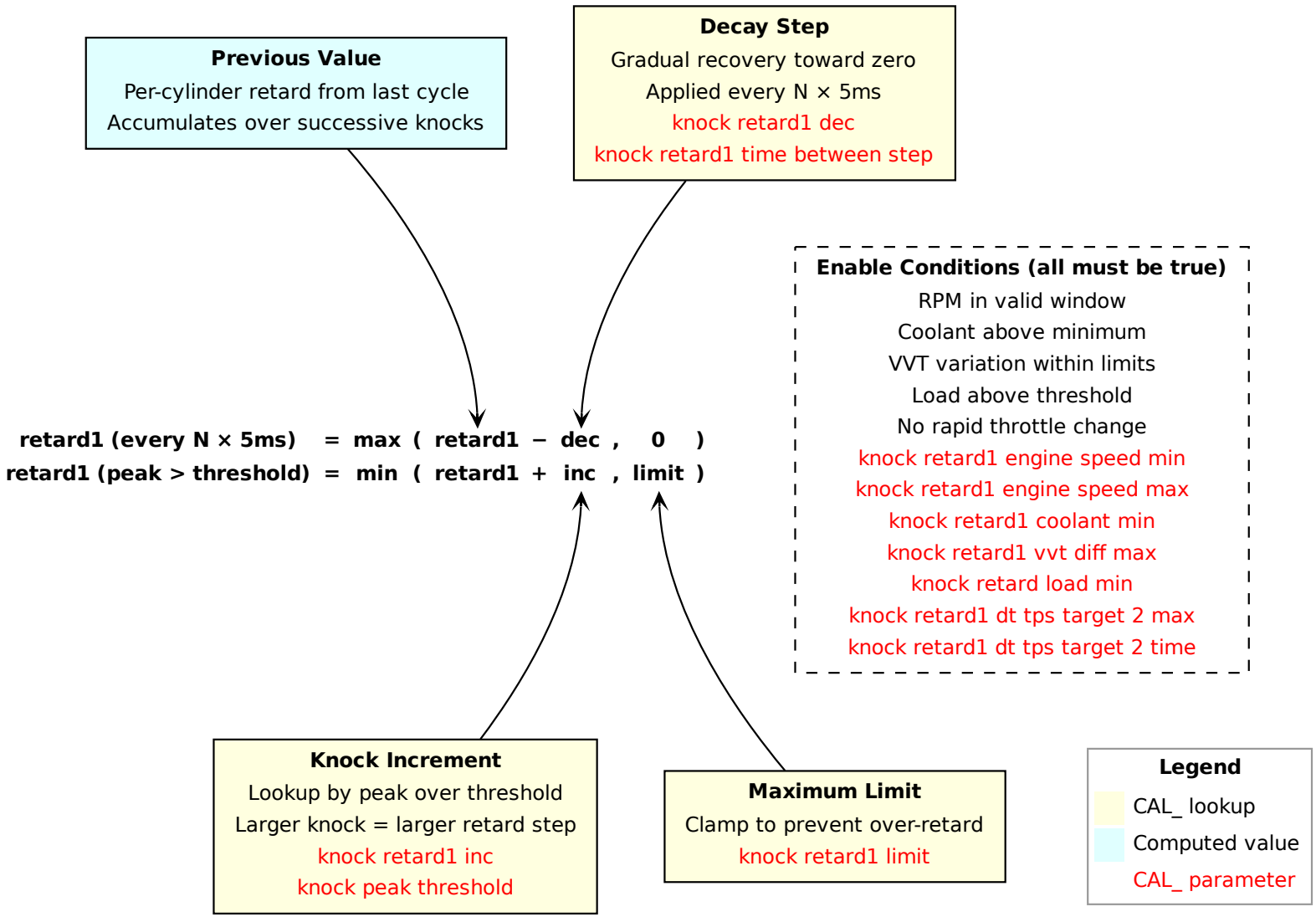


Figure 25: Knock retard 1 — immediate retard and recovery

13.3 Knock Retard 2 — Octane Learned Factor

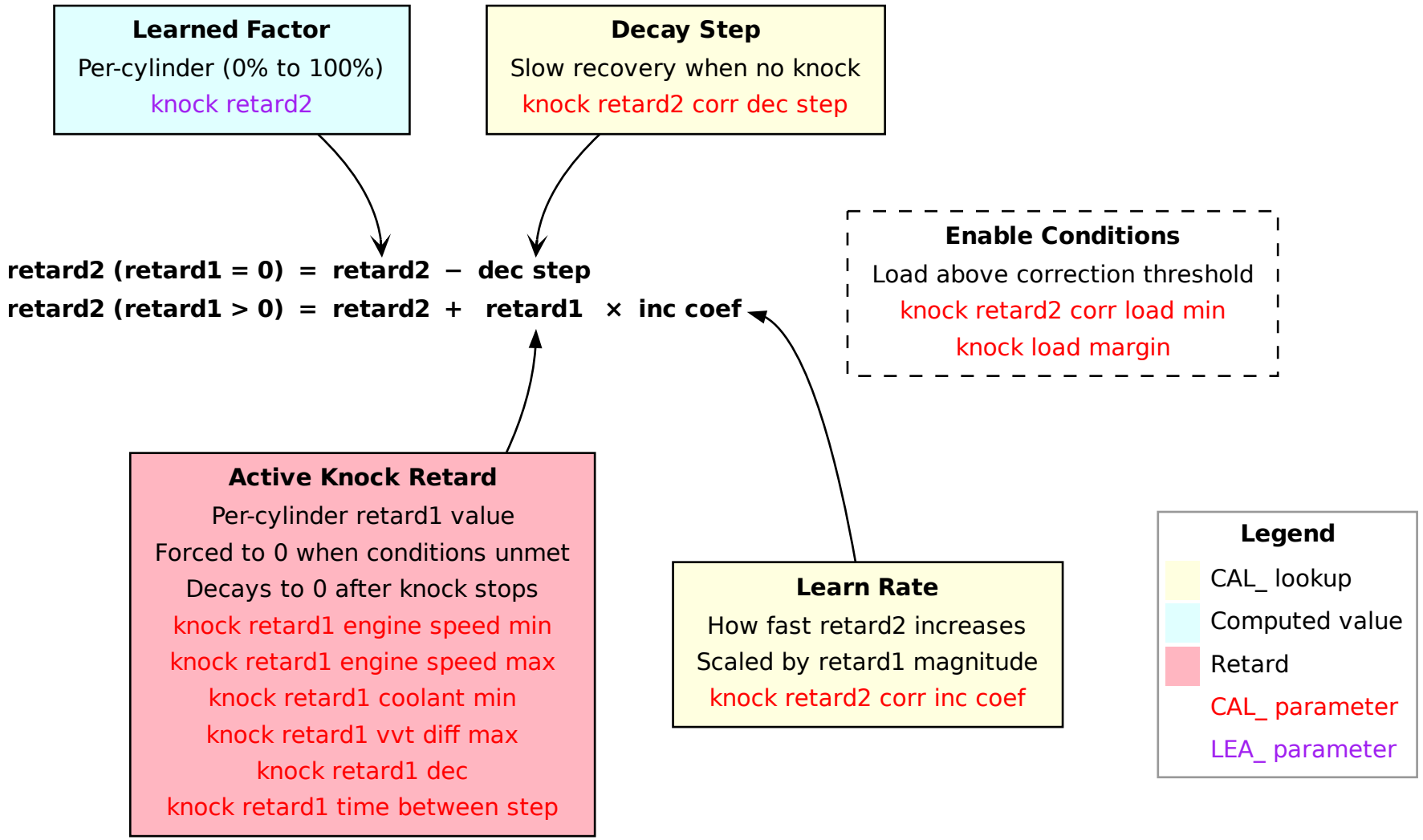


Figure 26: Knock retard 2 — octane learned factor

13.4 Tuning Tips

Calibrating `CAL_knock_peak_threshold` is a chicken-and-egg problem: you need to drive the car to measure the sensor signal level, but if the thresholds are wrong you risk either masking real knock or triggering false retard events that corrupt the data.

This problem is compounded on supercharged variants. The supercharger introduces broadband mechanical noise that the knock sensor picks up across the entire RPM range, leaving less margin to distinguish genuine detonation from background noise. A smaller supercharger pulley spins the blower faster and raises the noise floor further, making threshold calibration even more critical on high-boost builds.

A reliable approach is to eliminate any possibility of real knock during the calibration drive, then treat every above-threshold event in `knock_peak_over_threshold` as background noise to be characterised. This can be achieved by:

- Filling the tank with a high-ethanol blend or a very high octane fuel. Both give a large margin against detonation even under aggressive conditions.
- Using a conservative, knock-safe ignition map for the session — the goal is sensor characterisation, not performance.

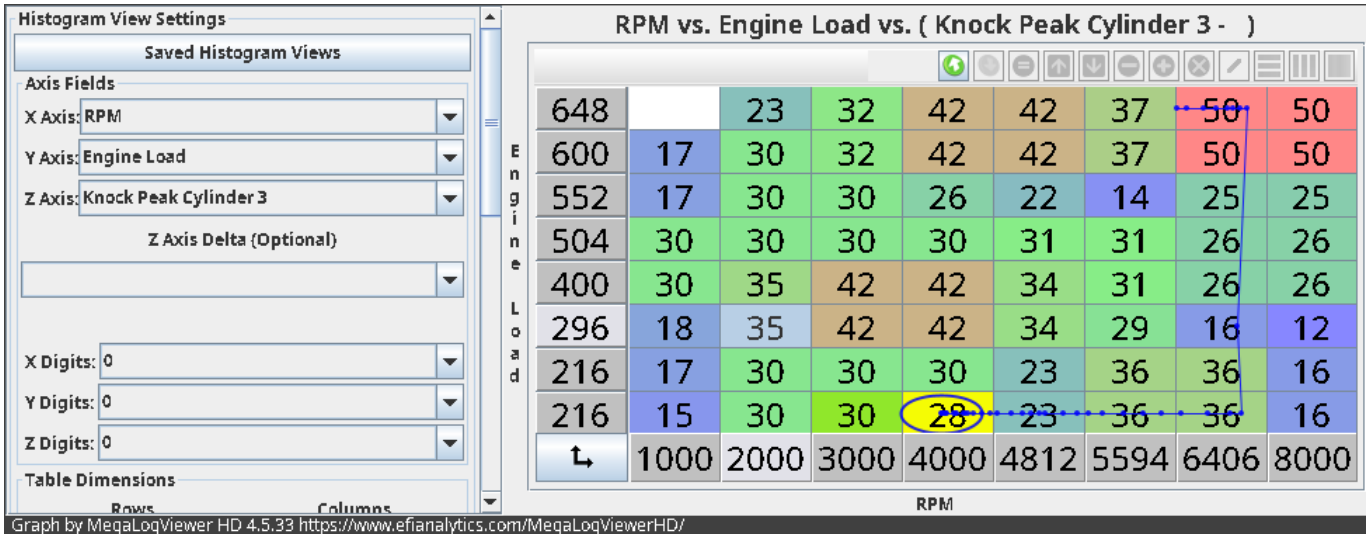
With knock ruled out by chemistry and timing, `knock_peak_over_threshold` can be monitored in real time. Two options exist to expose it during the drive:

- **Live-tuning access** (Section 20.1): poll the variable directly over CAN-Bus without any firmware modification. This is the simplest approach on unlocked ECUs.
- **Software patch**: redirect the peak-over-threshold value into an OBD-accessible location, allowing standard OBD logging tools to record it alongside all other channels.

Driving the full RPM and load range then yields a clean map of the sensor noise floor across all operating points — exactly the baseline needed to set `CAL_knock_peak_threshold` correctly: high enough to reject noise, low enough to catch real knock the moment it occurs.

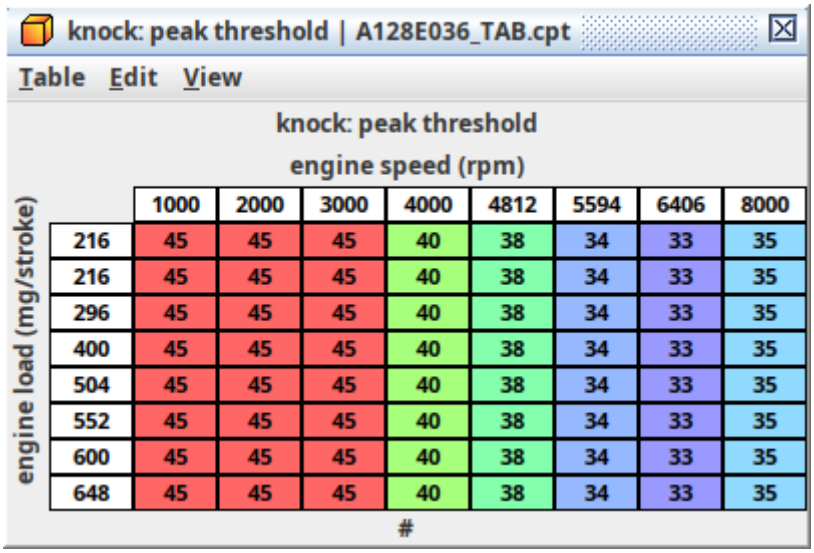
Note that `knock_peak_over_threshold` is a four-element array, one entry per cylinder. Significant differences between cylinders during the characterisation drive indicate that the sensor signal level varies from one cylinder to another — due to mechanical tolerances, sensor mounting position, or combustion chamber differences. In that case the per-cylinder sensitivity tables (`CAL_knock_sensitivity_cylinder1` through `CAL_knock_sensitivity_cylinder4`) should be adjusted individually to bring all four readings into the same range before the threshold table is finalised.

Figure 27 shows an example baseline for cylinder 3 logged on a Lotus Exige S Cup 260, visualised in MegaLogViewer HD using the Histogram/Table generator with the same RPM and load axes as `CAL_knock_peak_threshold`. Figure 28 shows the original `CAL_knock_peak_threshold` from the same car, as a starting point for comparison.



648		23	32	42	42	37	50	50
600	17	30	32	42	42	37	50	50
552	17	30	30	26	22	14	25	25
504	30	30	30	30	31	31	26	26
400	30	35	42	42	34	31	26	26
296	18	35	42	42	34	29	16	12
216	17	30	30	30	23	36	36	16
216	15	30	30	28	23	36	36	16
↳	1000	2000	3000	4000	4812	5594	6406	8000

Figure 27: Cylinder 3 noise floor baseline — MegaLogViewer HD (Lotus Exige S Cup 260).



	1000	2000	3000	4000	4812	5594	6406	8000
216	45	45	45	40	38	34	33	35
216	45	45	45	40	38	34	33	35
296	45	45	45	40	38	34	33	35
400	45	45	45	40	38	34	33	35
504	45	45	45	40	38	34	33	35
552	45	45	45	40	38	34	33	35
600	45	45	45	40	38	34	33	35
648	45	45	45	40	38	34	33	35

Figure 28: `CAL_knock_peak_threshold` — Lotus Exige S Cup 260 original calibration.

14 Idle Control

14.1 Overview

The idle control system operates in the airflow domain: it computes a target air mass flow (mg/s) rather than a throttle angle directly. The flow target is then converted to a TPS position via lookup tables, and added to the pedal demand in the throttle control stage.

Speed Target

The idle speed target is the sum of:

- **Base idle speed** — 2D vs coolant temperature (`CAL_idle_speed_base`). Cold engines idle higher.
- **Vehicle speed adjustment** — 2D vs car speed (`CAL_idle_speed_adj`). Raises idle when moving. Ramps down slowly via `CAL_idle_speed_time_between_decrement` when the condition clears.
- **AC compressor** — additive offset (`CAL_idle_speed_adj_ac_on`) when the AC is running.

The total is clamped to 1 520 rpm. The speed error (`engine_speed_idle_error`) is the difference between actual and target speed, used by the flow adjustments below.

Flow Target

The flow target is computed as the base flow plus a large number of signed adjustments:

- **adj1** (learned, `LEA_idle_flow_adj1`) — long-term learned correction persisted in EEPROM. Separate values for AC on/off. See learning section below.
- **adj2** — atmospheric pressure correction (2D vs atmo), subtracted from the total.
- **adj3** — battery voltage compensation (2D vs voltage, signed). Low voltage increases airflow to raise alternator output.
- **adj4 × adj4_adj** — feed-forward compensation based on the speed target itself (2D vs target idle), scaled by a 3D adjustment (coolant × accumulated air mass).
- **adj5** — proportional term (2D vs speed error, signed). Only active when the pedal is below `CAL_idle_flow_pps_max` and vehicle speed is below `CAL_idle_flow_adj5_max` (or engine speed is below target).
- **adj6** — vehicle speed compensation (2D vs car speed).
- **adj7** — dashpot (2D vs RPM). When engine speed exceeds the idle target by `CAL_idle_flow_adj7_enable`, this adds extra airflow to cushion the deceleration back to idle. Phased out linearly below `CAL_idle_flow_adj7_disable`.
- **adj8** — integrator. Accumulates correction based on speed error every `CAL_idle_flow_adj8_corr_time_between_step`, clamped by calibrated limits. Drives the long-term learning (adj1).
- **adj ac** — ramp-up offset when the AC compressor engages (`CAL_idle_flow_adj_ac`, 2D vs time since AC on).
- **adj fan** — separate offsets for low-speed and high-speed cooling fans (`CAL_idle_flow_adj_fan_low/high`, each 2D vs time since fan on).
- **Evap purge** — the estimated canister purge flow is subtracted, since purge air displaces throttle air.

Long-Term Learning (adj1)

The integrator (adj8) acts as the fast correction. When adj8 consistently deviates beyond `CAL_idle_flow_adj1_corr_max/min`, the learned value `LEA_idle_flow_adj1` is incremented or decremented by one unit. This slowly transfers the integrator's correction into the long-term learned value, so the integrator can remain near zero for the next idle event. Separate learned values are maintained for AC on (`LEA_idle_flow_adj1_ac_on`) and off. Learning requires sufficient engine run-time (`CAL_idle_flow_adj1_corr_maf_accumulated_min`) and is suspended during evap purge.

Flow to TPS Conversion

The flow target is converted to a throttle position for the motor controller:

- **Engine running** — lookup (`CAL_idle_tps`, 3D vs flow target and intake vacuum), clamped to `CAL_idle_tps_limit_h`.
- **Engine stopped** — lookup (`CAL_idle_tps_engine_stop`, 3D vs atmospheric pressure and coolant temperature). This pre-positions the throttle for starting.

The resulting `idle_tps_target` is then added to the pedal-demand TPS in the throttle control section.

Interaction with Ignition

When the idle flag is set (PPS below threshold and engine speed below the disable threshold), the ignition system switches from the running advance maps to dedicated idle advance maps with their own base, speed error correction, and rate-of-change damping.

14.2 Speed Target

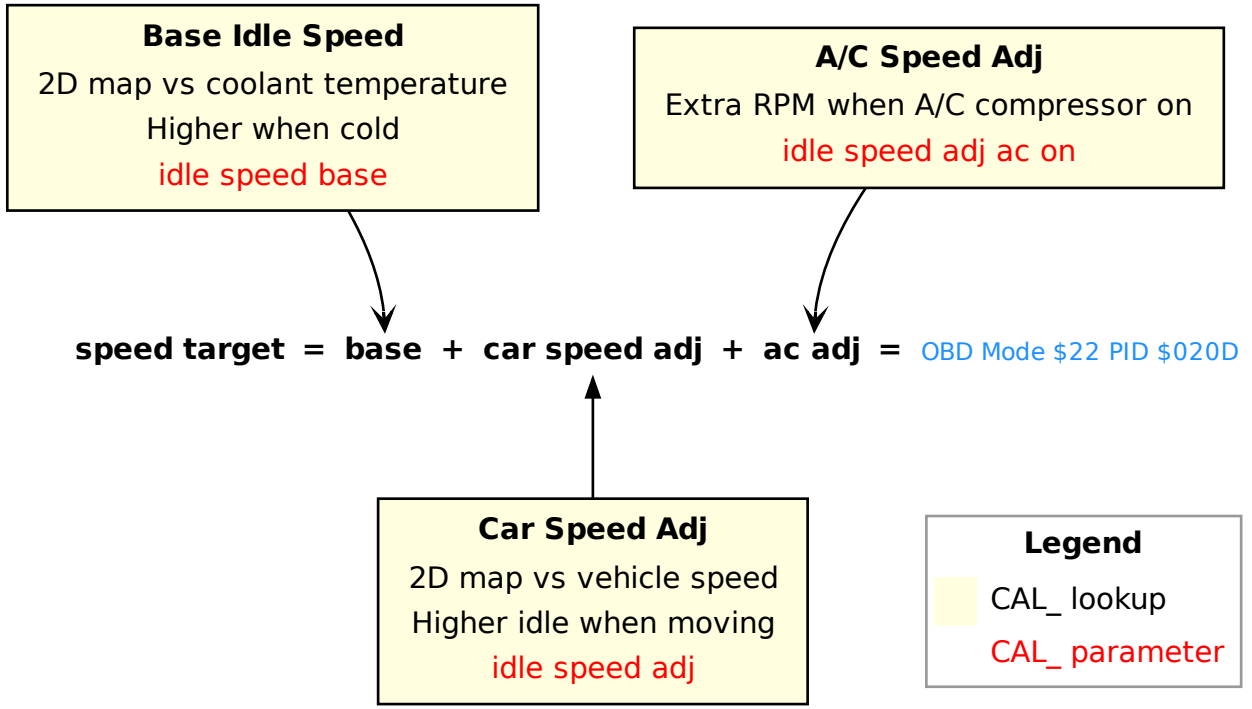


Figure 29: Idle speed target calculation

14.3 Flow Target

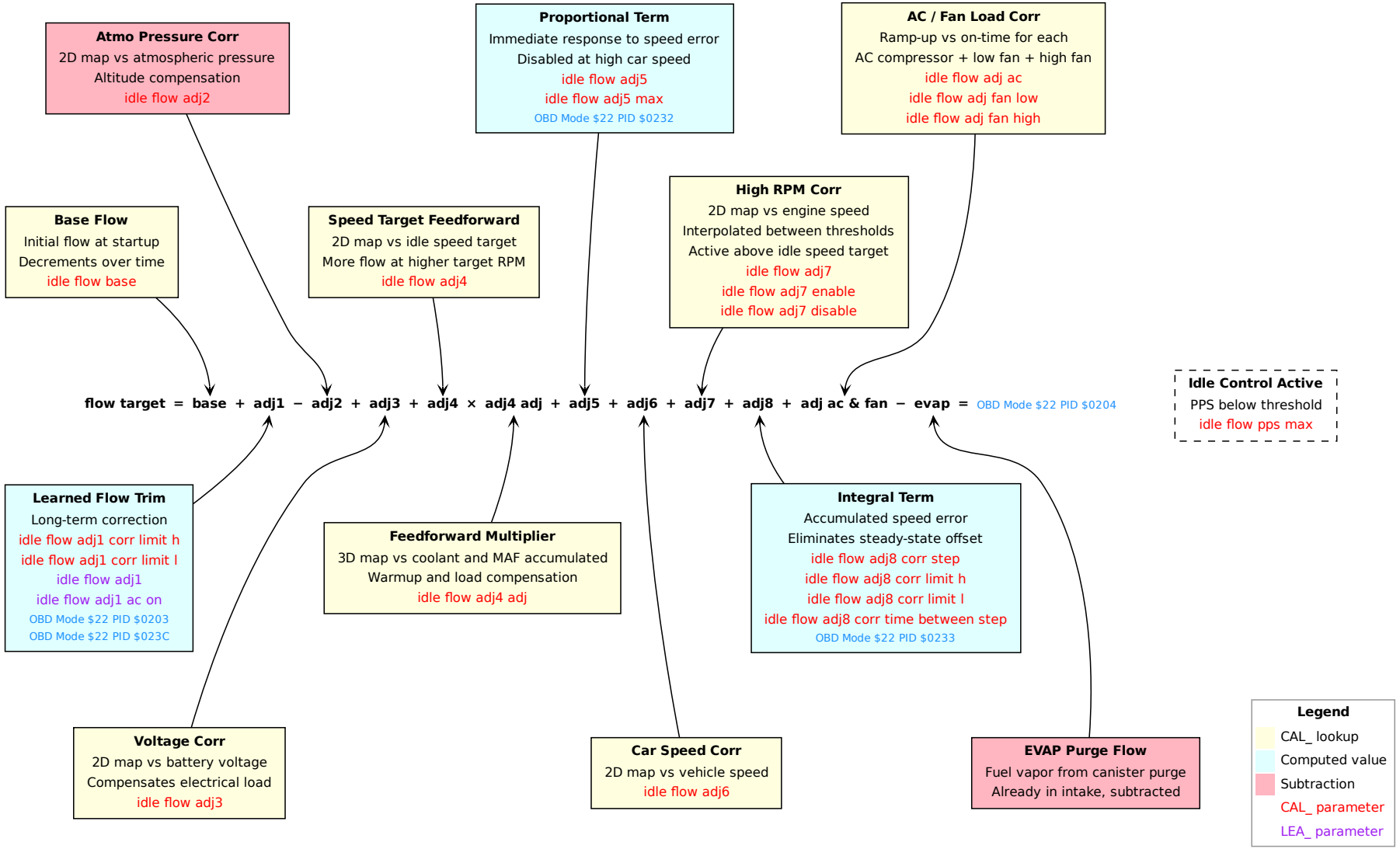


Figure 30: Idle flow target

15 Throttle Control

15.1 Overview

The throttle target is the sum of the pedal demand and the idle air contribution, clamped to calibration limits and ramped for smooth actuation.

15.2 Throttle Body Startup Sequence

At power-on, before the engine can be cranked, the ECU runs a state machine that verifies the throttle body is mechanically sound. The sequence is driven by the QADC interrupt (each ADC conversion cycle triggers one state machine step):

1. **Motor off (state 0).** The H-bridge PWM is set to zero and the motor enable is configured so the throttle blade is released. A delay counter of 100 ADC cycles is loaded, giving the return spring time to pull the blade to its rest position.
2. **ADC capture (state 1).** The counter from state 0 decrements each cycle. When it reaches zero the raw TPS1 ADC value is captured. This is the spring-rest voltage with no motor drive applied.
3. **Spring test / PID drive (state 2).** The PID controller begins driving the throttle toward an initial target of $\approx 0.71\,\text{V}$ (hardcoded). Once the position error falls below 29 mV, the setpoint is moved to fully closed ($\approx 0.59\,\text{V}$ raw ADC, hardcoded in program code). The motor must push the throttle closed against the return spring past the spring rest position ($\approx 18.5\%$), verifying motor authority. A stability check loop then begins: every 10 cycles a snapshot of the TPS voltage is taken, and 9 cycles later the snapshot is compared with the current voltage. While the PID is still driving the throttle closed, the position is changing and the check naturally fails. The loop repeats until:
   - *Pass:* the voltage has not moved more than $\pm 15\,\text{mV}$ since the last snapshot — the throttle has settled at the target. The state machine advances to calibration.
   - *Fail:* the timeout (1000 ADC cycles, hardcoded) expires before the position stabilizes. The `tps_state_test_failed` flag is set and the machine jumps directly to state 4 (active control), skipping calibration. This flag causes **P0638** (Throttle Actuator Control Range/Performance) to be set the next time the diagnostic monitor runs.
4. **Sensor calibration (state 3).** Using the motor-driven closed voltage from state 2:
   - For each sensor (TPS1, TPS2): the closed voltage is compared with the calibration default (`CAL_sensor_tps_{1,2}_offset`). If within $\pm 0.1\,\text{V}$, the learned offset (`LEA_sensor_tps_{1,2}_offset`) is blended 50% toward the measurement. Otherwise it is reset to the calibration default.
   - The corrected range and gain for both sensors are recomputed.
   - The motor is turned off and the machine advances to state 4.
5. **Active motor control (state 4).** Normal throttle control begins: the target is computed from PPS demand plus idle contribution, clamped and ramp-smoothed (see TPS Target below).

Fault Mode Behavior

During active control (state 4), if any fault flag is set — including the startup test failure flag, TPS sensor faults, or HC08 disagreement — the motor PWM is forced to zero, the H-bridge is disabled, and the throttle falls back to its spring-rest position. The HC08 is also notified via the forced-idle flag.

Related OBD Diagnostic Codes

P0122 TPS1 circuit low input (ADC below range).

P0123 TPS1 circuit high input (ADC above range).

P0222 TPS2 circuit low input (ADC below range).

P0223 TPS2 circuit high input (ADC above range).

P0638 Throttle actuator control range/performance — the throttle position does not track the target. Set when the position error (`tps_diff`) exceeds `CAL_obd_P0638_threshold` during stable conditions (no rapid TPS change). Also set immediately if the startup spring test fails.

P2104 Throttle actuator control system — forced idle. Set when the HC08 safety processor reports a forced-idle condition via `hc08_obd_flags`.

P2135 TPS1/TPS2 correlation — the two throttle position sensors disagree beyond `CAL_sensor_tps_match_max`. When correlation fails, the ECU selects the lower of the two sensors and limits maximum throttle opening.

P2173 Throttle actuator control system — high airflow detected. Set when the MAF-measured load diverges significantly from the Alpha-N estimate while the Alpha-N learned adjustment is at its maximum correction limit, indicating an air leak downstream of the throttle.

15.3 Engine-Stopped Sweep Test

State 4 contains an alternate branch for factory diagnostics. When `dev_tps_state_sweep_enable` is nonzero and the engine is not running, normal target calculation is bypassed. Instead, the throttle sweeps back and forth between 0% and 100% TPS in steps of 0.1%, with 1 ADC cycle between each step (all values hardcoded).

This feature is disabled by default (`tps_state_sweep_enable` is zero-initialized) and can only be activated by writing to its RAM address. It is likely a production-line test to verify throttle motor operation across the full range of travel.

15.4 Pedal Position Sensor (PPS) Calibration

Two independent PPS sensors provide redundant pedal demand measurement. Both are read via the QADC and converted to a pedal position value using a gain and a learned zero offset.

Continuous Offset Learning

Unlike the TPS, which calibrates its zero at startup, the PPS zero offset is learned continuously during normal operation. Every `CAL_sensor_pps_offset_time_between_step`, the ECU calls `learn_pps1_offset` and `learn_pps_2_offset`:

- If the current ADC reading is *below* the learned offset *and* within `CAL_sensor_pps_offset_diff_max` of the calibration default (`CAL_sensor_pps_1_offset`), the learned offset is slowly pulled down toward that minimum, at a rate controlled by `CAL_sensor_pps_offset_reactivity`. The result is clamped to `CAL_sensor_pps_1_offset_limit_l` (PPS1) or `CAL_sensor_pps_2_offset_limit_l` (PPS2) to prevent the zero from drifting below a safe floor.
- If the learned offset drifts further from `CAL_sensor_pps_1_offset` than `CAL_sensor_pps_offset_diff_max` allows, it is reset to the calibration default.

This means the ECU silently tracks the mechanical rest position of the pedal and compensates for wear and sensor drift over time, without any explicit calibration procedure.

EEPROM Persistence and Startup

The learned offsets (`LEA_sensor_pps_1_offset`, `LEA_sensor_pps_2_offset`) are persisted in EEPROM. At startup, if the EEPROM is valid (checked by CRC and model name string), the stored offsets are loaded

and increased by `CAL_sensor_pps_offset_diff_max`. This gives the learning algorithm headroom to track the true minimum downward from the first key-on cycle. If the EEPROM is blank or corrupt, both offsets are reset to the calibration defaults (`CAL_sensor_pps_1_offset`, `CAL_sensor_pps_2_offset`).

Fault Handling and `pps_min` Selection

Each QADC cycle the raw ADC voltages are checked against configurable range limits (`CAL_misc_pps_1_range_low` / `_high`, same for PPS2). The output `pps_min` used by the throttle target and DFSO logic is selected based on sensor health:

- **Both sensors healthy and correlating**: `pps_min` is the lesser of `pps_1` and `pps_2`. This ensures the ECU never commands more throttle than the driver intends.
- **One sensor hardware fault**: the healthy sensor's value is used as fallback. The faulted sensor's last-valid reading is frozen and used as backup.
- **Both sensors faulted**: `pps_min` is forced to zero (safe closed-throttle state).

In parallel, the two sensor readings are compared using the calibration-default gain (not the learned offset). If their difference exceeds `CAL_sensor_pps_match_max`, a correlation fault (`pps_correlation_state`) is raised. During a correlation fault, `pps_min` falls back to the frozen last-valid readings, and the HC08 safety processor is notified via the `flags_to_hc08` byte.

Related OBD Diagnostic Codes

P2122 PPS1 circuit low input (ADC below `CAL_misc_pps_1_range_low`).

P2123 PPS1 circuit high input (ADC above `CAL_misc_pps_1_range_high`).

P2127 PPS2 circuit low input (ADC below `CAL_misc_pps_2_range_low`).

P2128 PPS2 circuit high input (ADC above `CAL_misc_pps_2_range_high`).

P2138 PPS1/PPS2 correlation — the two pedal sensors disagree beyond `CAL_sensor_pps_match_max`.

15.5 TPS Target

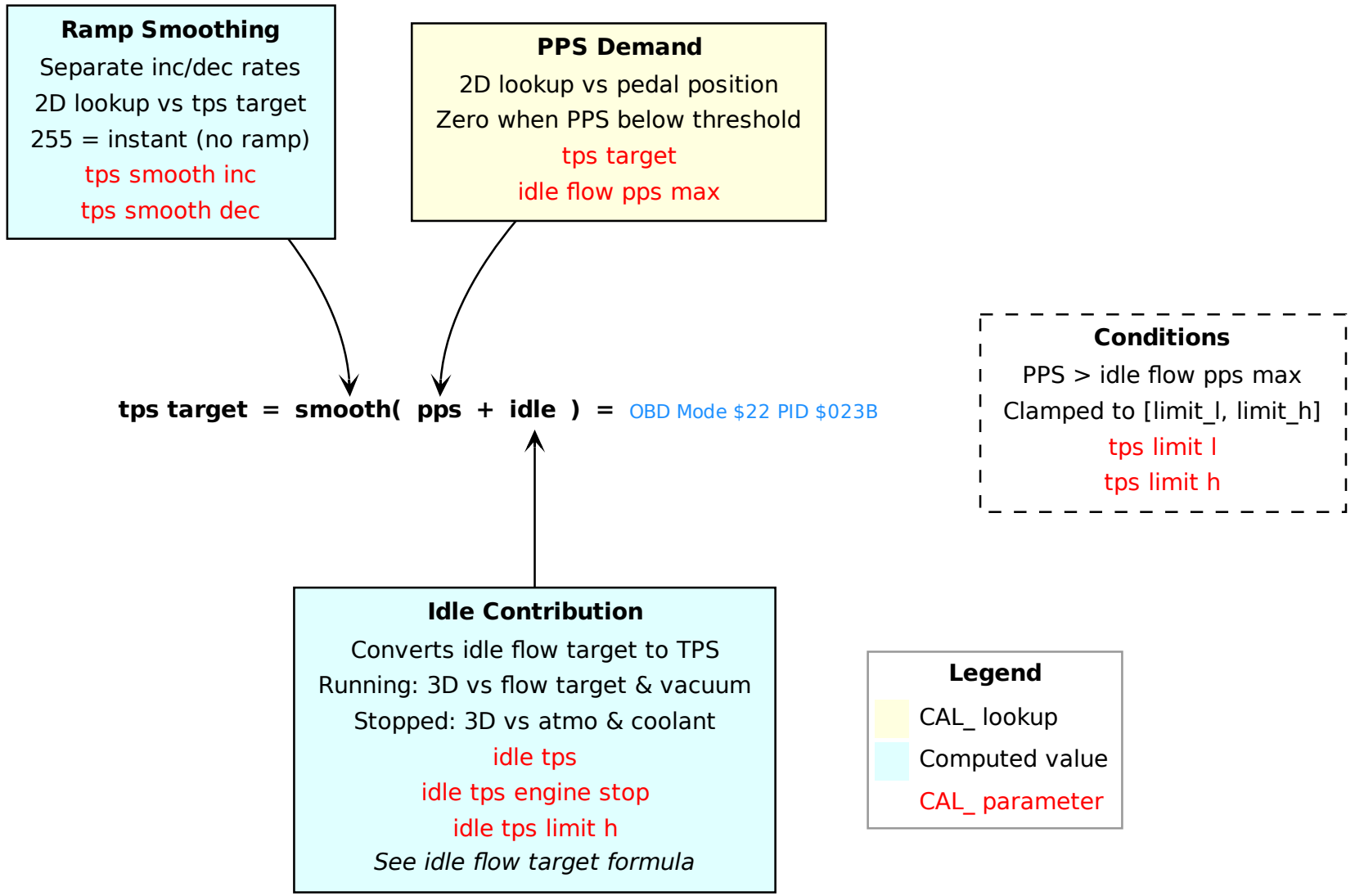


Figure 31: TPS target calculation

15.6 Tuning Tips

`CAL_tps_limit_h` sets the maximum throttle body opening. Lotus used this as a simple and effective way to reduce peak power on the 220 hp variant — limiting the throttle plate mechanically caps airflow without any other calibration change. If you are tuning a de-restricted car, verify this value is set to full opening before chasing fuelling or ignition changes, as an artificially low limit will prevent the engine from reaching its full air mass regardless of other calibration.

The HC08 safety processor also contains a copy of `CAL_tps_target`. Any modification to this table must also be applied to the HC08 calibration, otherwise the safety processor will detect a mismatch and trigger limp mode.

16 Variable Valve Timing & Lift

16.1 Overview

The 2ZZ-GE engine has two variable valve mechanisms: continuously variable intake cam timing and a two-stage valve lift switch (VVTL-i). Both are controlled by oil-pressure solenoids driven via PWM on TPU-B channels 7 (VVT) and 9 (VVL).

Variable Valve Timing (VVT)

VVT adjusts the intake camshaft phasing to optimize torque, emissions, and idle quality. The cam position is measured from the cam sensor interrupt by comparing the cam pulse timing to the crank reference, yielding `vvt_pos` in quarter-degree units.

The target advance is a 3D map (vs RPM and load), with separate maps for low cam (`CAL_vvt_adv_low_cam_base`) and high cam (`CAL_vvt_adv_high_cam_base`). A coolant temperature retard (`CAL_vvt_adv_adj`) is subtracted, and the result is clamped to a minimum of 5° and a maximum that depends on cam profile (40° low cam, 43° high cam). An override (`dev_vvt_angle`) allows direct target setting via OBD for tuning.

The target is smoothed with a slow ramp (1 unit per 50 ms step) before reaching the PI controller. The controller computes a proportional term `vvt_ctrl_p` (gain `CAL_vvt_ctrl_p_gain`) and an integral term `vvt_ctrl_i` (gain `CAL_vvt_ctrl_i_gain`) from `vvt_diff` (target minus actual position). The summed output `vvt_output` is converted to a PWM duty cycle (`vvt_duty_cycle`). The flag `vvt_target_reached` is set when the position error falls within tolerance.

VVT Startup Calibration

On each engine start, VVT remains inactive (0 % duty) for a minimum runtime (`vvt_runtime_min`, a 2D map vs coolant temperature at engine stop). During this period the ECU accumulates `vvt_pos` samples into `vvt_rest_pos_sum` and `vvt_rest_pos_count`, tracking the running minimum (`vvt_rest_pos_min`) and maximum (`vvt_rest_pos_max`). When the measurement window closes, `vvt_rest_pos_avg` is computed and `vvt_rest_pos_measured` is set. If the spread of the rest position exceeds `CAL_vvt_rest_pos_threshold`, `vvt_rest_pos_fault` is set and VVT control is inhibited for the entire drive cycle (fault P0016).

Variable Valve Lift (VVL)

VVL switches between two cam profiles: a low-lift profile optimized for efficiency at low RPM, and a high-lift profile that opens the valves further for high-RPM power. The switch is controlled by a single oil-pressure solenoid acting on a sliding pin mechanism.

Engagement requires coolant above `CAL_vvl_coolant_enable` and either of:

- Engine speed above `CAL_vvl_rpm_enable` (unconditional), or
- Engine speed above `CAL_vvl_rpm_high_load_enable` *and* load above `CAL_vvl_high_load_enable` (allows earlier engagement under high load).

Disengagement has matching thresholds with hysteresis (`CAL_vvl_disable_*`). When VVL is active, the ignition and VVT systems switch to their respective high-cam maps.

On the Elise, `CAL_vvl_rpm_high_load_enable` equals `CAL_vvl_rpm_enable`, so the high-load early engagement path is effectively disabled and the high-cam ignition map is never used (its values may be garbage). On the Exige S with supercharger, these thresholds differ, enabling early VVL engagement at high boost.

On 1ZZ-FE calibrations, which have no VVL hardware, all engagement thresholds are set to 9500 rpm — a value the engine can never reach — effectively disabling the system entirely.

16.2 VVT Target

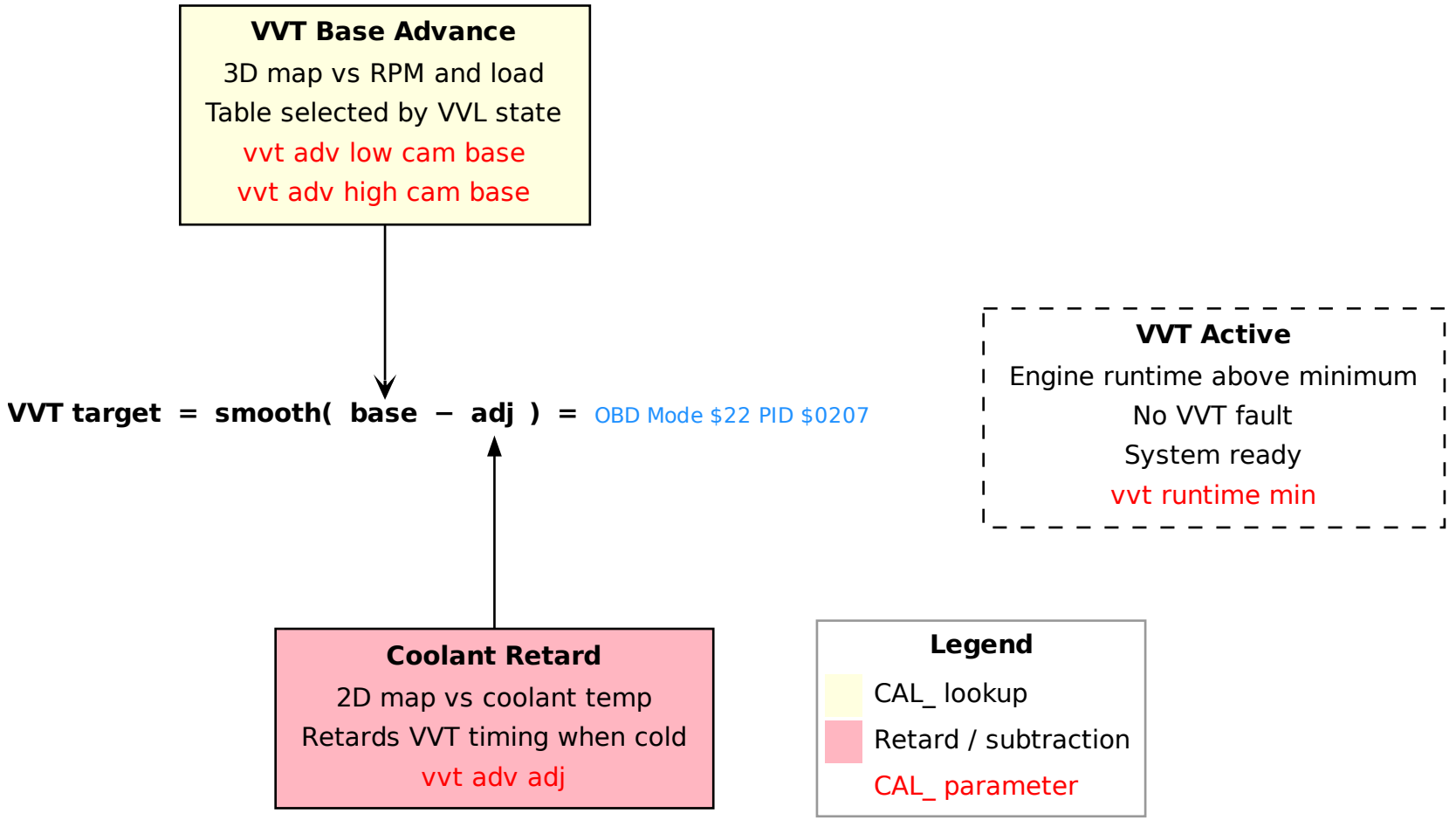


Figure 32: VVT target advance angle

16.3 VVL Engagement

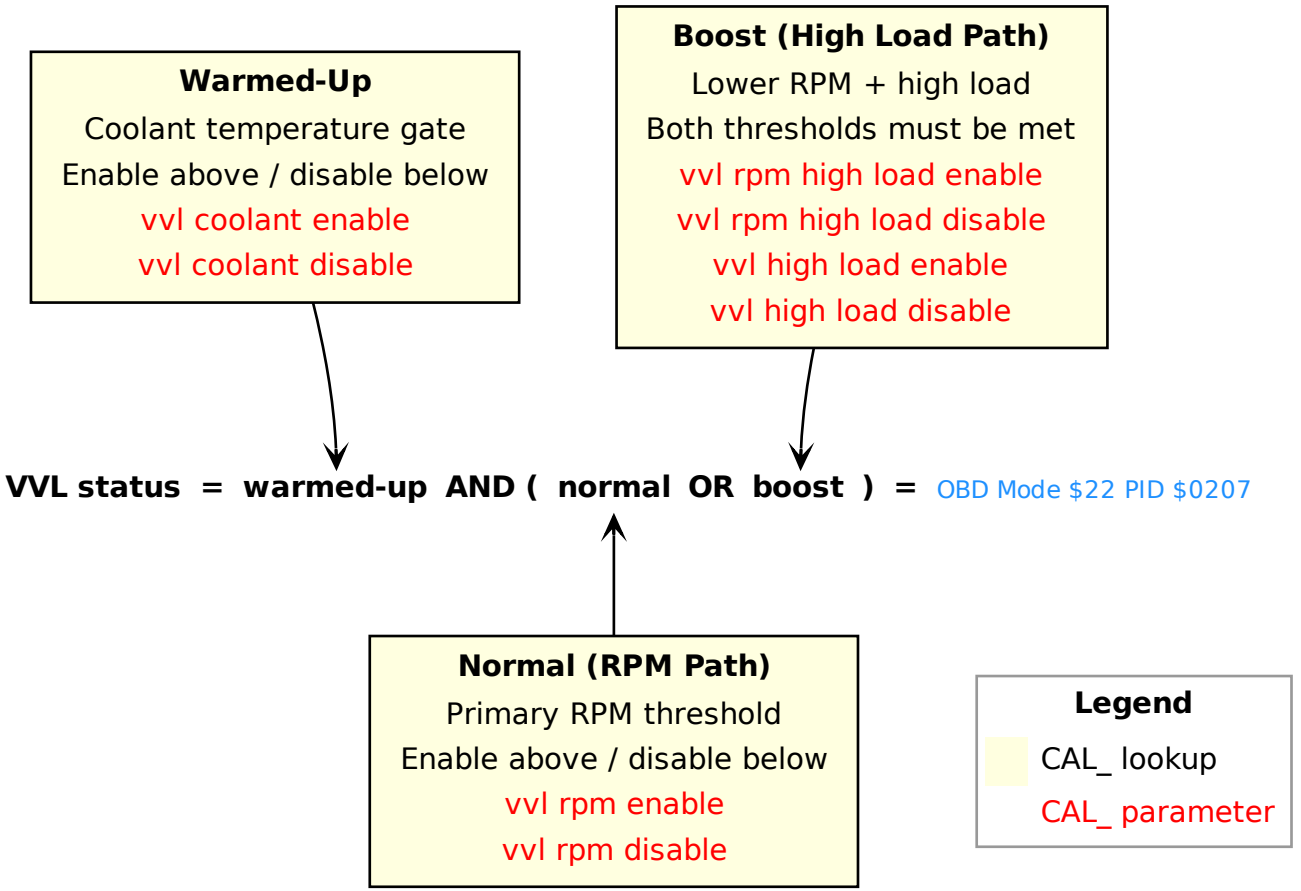


Figure 33: VVL engagement conditions

17 Traction Control

17.1 Overview

The traction control system reduces engine torque when the rear wheels spin faster than the fronts. It uses four wheel speed sensors (from the ABS ring teeth) and acts through both ignition retard and sequential fuel cut.

Wheel Speed and Slip

Each wheel speed is computed from its sensor period, the ABS ring tooth count (`CAL_tc_abs_ring_teeth_count`), and the tyre circumference (`CAL_tc_front_tyre_circumference`, `CAL_tc_rear_tyre_circumference`). The vehicle speed (`car_speed`) is derived from the faster rear wheel, smoothed by `CAL_tc_car_speed_reactivity`.

Slip is the percentage by which the faster rear wheel exceeds the faster front wheel. The slip error (`tc_slip_diff_smooth`) is the excess above the target, smoothed via `CAL_tc_slip_reactivity`.

Slip Target

The base slip target is a map (`CAL_tc_slip_target_base`, 3D vs front wheel speed and PPS). When large left-right speed differences are detected (cornering), an additive margin (`CAL_tc_slip_target_adj`) widens the target to avoid false intervention in turns.

Operating Modes

`CAL_tc_mode` selects the system behaviour:

- **Mode 0** — traction control disabled.
- **Mode 1** — button cycles TC on/off (long press toggles).
- **Mode 2** — Variable Traction Control. The button toggles between normal TC and VTC mode (`tc_flags` bit 7). In VTC mode, a rotary knob (`sensor_adc_tc_knob`) directly sets the slip target: counter-clockwise uses the base map, mid-range scales the target proportionally to allow more wheelspin, fully clockwise disables TC intervention entirely. When the engine is off, the knob programs the launch control rev limit (`LEA_tc_launchcontrol_revlimit`, 2 000–8 500 rpm, persisted in EEPROM).

Torque Reduction

When slip exceeds the target and all enable conditions are met (front speed in range, engine speed above minimum, not in DFSO, TC enabled), the system applies two complementary interventions:

- **Ignition retard** — `ign_adv_adj_by_tc` is a 0–100 % multiplicative factor on ignition advance. It combines a proportional term (`tc_retard_adj2`, from slip error scaled by `CAL_tc_slip_to_retard_ratio`) with a slow-ramping integral term (`tc_retard_adj1`, incremented every `CAL_tc_time_between_step`). The total is clamped by `CAL_tc_ign_adv_adj_limit`.
- **Fuel cut** — `tc_fuelcut` sets a sequential injection skip pattern (looked up from `CAL_tc_fuelcut` vs slip error). Individual cylinder injector ISRs use this as a modulo counter: when the combustion event counter reaches the threshold, that cylinder's injection is skipped and the injector fires a minimal 200 μs pulse instead.

When the slip error drops below the target, retard1 ramps back down and fuel cut is disabled. The TC warning lamp on the dashboard blinks while traction control is actively intervening.

17.2 Slip

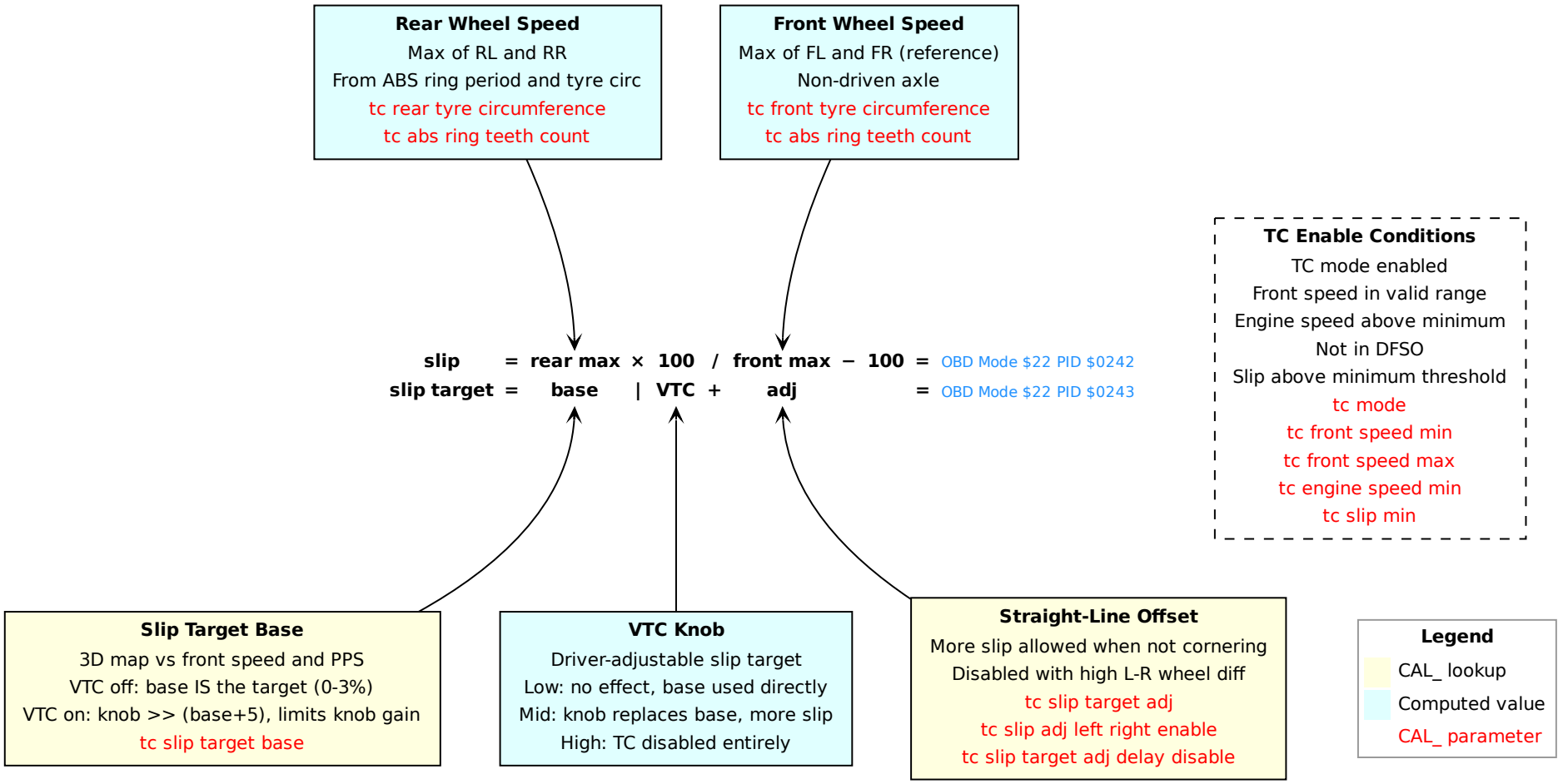


Figure 34: Traction control — slip calculation and target

18 Cooling System

18.1 Overview

The ECU controls three cooling-related actuators via the L9822E octal serial solenoid driver: a two-speed radiator fan, an AC compressor clutch, and a coolant recirculation pump. All three interact with the idle control system through dedicated airflow adjustment tables.

18.2 Radiator Fan

There are two physical fan units. Speed is controlled by switching their wiring configuration: in low-speed mode the two fans are connected in **series**, so each receives half the supply voltage; in high-speed mode they are switched to **parallel**, giving each fan the full supply voltage. Each mode is controlled independently with separate enable/disable temperature thresholds (hysteresis pairs) and a minimum run-time timer to prevent rapid cycling.

Low-Speed Fan

- **Engine running**: engages when coolant exceeds `CAL_fan_low_enable`, disengages below `CAL_fan_low_disable`.
- **Engine stopped**: separate thresholds `CAL_fan_low_stop_enable` / `CAL_fan_low_stop_disable` keep the fan running after shutdown to dissipate heat soak.
- `CAL_fan_low_stop_time_min` enforces a minimum off-time: a counter increments every 5 ms from the moment the fan turns off and is reset to zero each time the fan runs. Re-engagement is only allowed once that counter reaches the threshold, preventing rapid cycling.

High-Speed Fan

- **Normal thresholds**: engages when coolant exceeds `CAL_fan_high_enable`, disengages below `CAL_fan_high_disable`.
- **AC thresholds**: when the AC compressor is running a separate (typically lower) pair applies — `CAL_fan_high_ac_enable` / `CAL_fan_high_ac_disable` — ensuring adequate condenser cooling even at moderate coolant temperatures.
- **External AC fan request**: `sensor_adc_ac_fan_request` can activate the high fan independently of coolant temperature. When that signal drops, the fan continues running for `CAL_fan_high_stop_user_delay` seconds before switching off.
- `CAL_fan_high_stop_time_min` enforces a minimum off-time: a counter increments every 5 ms from the moment the fan turns off and is reset to zero each time the fan runs. Re-engagement is only allowed once that counter reaches the threshold, preventing rapid cycling.

Car Speed Cutoff

Both fans are disabled above `CAL_fan_car_speed_disable`. At speed, ram air provides sufficient cooling and the fans would be ineffective against the airflow anyway.

Related OBD Diagnostic Codes

P0480 Fan 1 (low-speed) control circuit fault.

P0481 Fan 2 (high-speed) control circuit fault.

18.3 AC Compressor Clutch

The AC compressor clutch is engaged via the user button, subject to a set of protective inhibit conditions. Any one of the following will prevent or cut engagement:

- **WOT cutoff**: AC is cut when PPS exceeds `CAL_ac_pps_disable`, and re-enabled once TPS drops below `CAL_ac_tps_enable`. `CAL_ac_pps_disable_time_min` enforces a minimum cutoff duration: the AC is held off for at least this long after the cutoff triggers, even if the throttle closes quickly.
- **Coolant overtemperature**: above `CAL_ac_coolant_disable`, re-enabled below `CAL_ac_coolant_enable`.
- **Engine overspeed**: above `CAL_ac_engine_speed_disable`, re-enabled below `CAL_ac_engine_speed_enable`.
- **Car speed**: above `CAL_ac_car_speed_disable`.
- **Cold start**: engine runtime below `CAL_ac_runtime_min`.

When all conditions allow, two timers prevent rapid on/off cycling of the compressor clutch:

- `CAL_ac_user_enable_time_min`: minimum time the AC must remain on. The clutch cannot disengage until this timer expires after the button is released.
- `CAL_ac_user_disable_time_min`: minimum time the AC must remain off. The clutch cannot engage until this timer expires after the button is pressed.

18.4 Coolant Recirculation Pump

The recirculation pump runs *only when the engine is stopped*. In conditions of heat soak, after stopping a hot engine, the pump circulates coolant through the heater circuit to limit the potential for localised boiling within the cylinder head. It engages when coolant exceeds `CAL_misc_recirculation_pump_stop_enable` and disengages below `CAL_misc_recirculation_pump_stop_disable`. As soon as the engine starts, the pump is forced off. It can also be controlled directly via OBD service mode 0x2F for diagnostic purposes.

18.5 Idle Flow Adjustments

All three actuators place a significant additional load on the engine at idle. The ECU compensates through additive tables, each 2D vs how long the actuator has been active (on-time). The profile is typically a peak at engagement to cover the initial surge demand, decaying back to zero as the idle controller takes over:

- `CAL_idle_flow_adj_ac` — AC compressor load compensation.
- `CAL_idle_flow_adj_fan_low` — low-speed fan load compensation.
- `CAL_idle_flow_adj_fan_high` — high-speed fan load compensation.

19 Evaporative Emission Control

19.1 Overview

The evaporative emission control system (EVAP) prevents fuel vapors from the tank from escaping to the atmosphere. Vapors are captured in a charcoal canister and periodically purged into the intake manifold, where they participate in combustion. Because the fuel content of the purge air is unknown, the ECU must learn it online and compensate the injection accordingly.

Purge Valve

The canister purge valve is a PWM-controlled solenoid. The duty cycle (`evap_purge_command`) is the sum of a minimum (`CAL_evap_duty_cycle_limit_l`) and a 3D adjustment (`CAL_evap_purge`, 3D vs pressure drop and vacuum), clamped between low and high limits. During the first few seconds after purge starts, a gentler initial duty cycle (`CAL_evap_duty_cycle_initial`) is used.

Pressure Drop Ramp

Rather than opening the purge valve abruptly, the system ramps a virtual "pressure drop" value (`evap_pressure_drop`) that governs the purge intensity. Every 100 ms:

- When purge is enabled, the pressure drop ramps up by `evap_pressure_drop_inc` (adapts to MAF flow and concentration).
- When purge is disabled (idle flow too low, injection time too short, or STFT unstable), it ramps down by a fixed decrement.
- The maximum is clamped by the lesser of a vacuum-based limit (`CAL_evap_pressure_drop_max`, 2D vs vacuum) and a load-based limit (prevents excessive enleanment). An additional cap applies during idle.

Concentration Learning

The fuel concentration of the purge air (`evap_concentration`, 0–100 %) is unknown at startup and must be learned. The ECU observes how closed-loop STFT shifts when purge is active:

- If STFT goes negative (mixture too rich from purge fuel), the concentration estimate increases.
- If STFT goes positive (mixture too lean), the concentration estimate decreases.

The adjustment rate (`evap_concentration_adj`) is proportional to the STFT deviation scaled by the fuel load and engine speed. Learning is gated by STFT stability and only active while purge flow is nonzero.

Flow Estimation and Injection Compensation

From the pressure drop, the ECU estimates the total purge air flow (`evap_flow`, in mg/s) and the fuel fraction (`evap_fuel_flow = evap_flow` × `concentration / 100`). This is converted to a per-stroke fuel mass (`evap_fuel_load`) and subtracted from the required fuel load in the injection calculation. The purge air flow is also subtracted from the idle flow target, since purge air displaces throttle air.

State Machine

The EVAP system uses a 5-state machine:

- **State 0** — inactive. Waits for enable conditions.
- **State 1** — purging. Pressure drop ramps up, concentration learning active.

- **State 2** — STFT learning. Purge is paused while the closed-loop system re-centers (resets STFT, allows LTFT learning).
- **State 3** — DFSO pause. Purge is suspended during fuel cut and held until a recovery delay expires.
- **State 4** — leak test. System integrity check for OBD compliance (P0441/P0446 monitors).

Enable Conditions

Purge requires: closed-loop fuel control active, STFT within $\pm 199\,\mu$s, engine started for at least `CAL_evap_engine_start_delay`, no related DTCs (P0441, P0444, P0445), and `CAL_evap_purge_mode` $\neq 2$ (mode 2 disables purging). `CAL_evap_purge_mode = 0` enables normal operation; mode 1 forces purge on for testing.

19.2 Federal Leak Test

The ECU performs a vacuum-decay leak test to detect fuel vapor leaks in the EVAP system, as required by OBD-II regulations. The test runs automatically when conditions allow and uses a 6-state sequence (`evap_leak_state`), executed every 100 ms:

Pre-Conditions

The leak test requires: vehicle stationary, engine idling in closed-loop, fuel level between `CAL_evap_leak_fuel_level_min` and `CAL_evap_leak_fuel_level_max`, coolant above `CAL_evap_leak_coolant_min`, intake air below `CAL_evap_leak_engine_air_max`, air temperature at engine stop below `CAL_evap_leak_engine_air_stop_max`, atmospheric pressure above `CAL_evap_leak_atmo_pressure_min`, and no active EVAP-related DTCs. A cooldown timer (`CAL_evap_leak_cooldown`) prevents re-entry after a P0442 fault. Setting `CAL_evap_leak_force` to nonzero forces the test on next idle.

Test Sequence

0. **Initial delay.** Waits for `CAL_evap_leak_initial_delay` after entering idle.
1. **Vent valve closed.** The canister vent valve is closed (sealing the system), then waits `CAL_evap_leak_settle_time` to let the tank pressure settle.
2. **Baseline measurement.** Over `CAL_evap_leak_baseline_time`, the ECU records the minimum and maximum tank pressure to characterize the natural pressure variation ("tank breathing" from fuel sloshing and temperature). The pressure range and fuel level are used to compute a reference threshold for the gross leak check.
3. **Purge with gross leak check (P0455).** Purge is resumed with a limited pressure drop (`evap_pressure_drop_max_leak_test`), which is dynamically adjusted based on the observed vacuum. Over `CAL_evap_leak_purge_time`, if the tank pressure stays above `CAL_evap_leak_gross_pressure`, a large leak is indicated (P0455). If vacuum exceeds `CAL_evap_leak_vacuum_min`, the system is sealed and the test continues. The vent valve is reopened once the purge command exceeds `CAL_evap_leak_purge_min` (or `CAL_evap_leak_purge_retry_min` after a prior P0442).
4. **Vacuum decay setup.** Purge is stopped and the system waits for the pressure drop to reach zero. The ECU then records the reference pressure and looks up the P0442 and P0456 leak thresholds from the fuel level.
5. **Final measurement.** Over `CAL_evap_leak_P0442_time` and then `CAL_evap_leak_P0456_time`, the ECU measures how quickly the vacuum decays by comparing the current tank pressure to the reference, with a correction for the baseline pressure variation measured in step 2. The result (`evap_leak_result`) is compared against two fuel-level-dependent thresholds:
   - If the result exceeds the P0442 threshold over multiple test cycles: small leak (P0442).
   - If the result exceeds the P0456 threshold over multiple test cycles: very small leak (P0456).

   The final result is stored in `LEA_evap_leak_result` and persisted in EEPROM for OBD reporting.

20 Data-Logging

20.1 Live Tuning Access

The ECU firmware includes a development interface (`can_a_mb15_recv_dev`) that allows reading and writing arbitrary memory addresses over CAN-Bus. Since the calibration data is copied from flash into RAM at startup, this makes it possible to edit maps while the engine is running. All changes are temporary and lost when the ECU is powered off.

Unlock

The interface is only active when the ECU is "unlocked". At startup, the ECU checks `CAL_ecu_unlock_magic` for a 4-byte magic value. If it matches, `dev_unlocked` is set to true and the CAN mailbox 15 receive interrupt is enabled. Most white dashboard cars (pre-2008) shipped unlocked, but the black dashboard versions have this access locked. It can be re-enabled with a JTAG/BDM programmer or a modified CRP file.

Protocol

The interface uses CAN-A mailbox 15 for receiving commands and mailboxes 0 and 1 for responses. Commands are distinguished by CAN ID:

- **Read**: 1, 2, or 4 bytes from a given RAM address, or N bytes for bulk reads.
- **Write**: 1, 2, or 4 bytes to a given address, with a multi-frame mode for larger writes.
- **List**: Query the `dev_varptr_list` — returns the base address of the table in flash.

The list query simply provides a starting point for discovering variable RAM addresses using plain reads.

Variable Pointer List

The `dev_varptr_list` is a null-terminated constant table stored in flash at `0x7884C` (this version of the firmware). Each entry is a `struct_varptr` (6 bytes):

- `ptr` (4 bytes): RAM address of the variable.
- `size` (2 bytes): variable width — 1, 2, or 4 bytes.

A `ptr` of zero marks the end of the list. The entry count varies between firmware versions, so tools must walk the list until the null terminator rather than relying on a fixed count.

This table is similar across all Lotus ECU families from the K4 to the T6, which is why tooling written for one variant often works on another.

Example: reading `engine_speed_2`

`engine_speed_2` is a `u16` located at index 31 of the list. To read it:

1. **Query the list.** Send the list command; the ECU responds with the base address `0x7884C`.
2. **Read entry 31.** Entry 31 is at flash address $\texttt{0x7884C} + 31 \times 6 = \texttt{0x78952}$. Read 4 bytes to get `ptr` (the RAM address of `engine_speed_2`) and 2 more for `size` (= 2).
3. **Read the value.** Send a 2-byte read command for the retrieved `ptr`. The ECU returns the current engine speed as a raw `u16` (e.g. `0x0BB8` = 3000 rpm).

The index of a variable in the list is consistent across firmware revisions; its RAM address is not. This is why tools should always resolve addresses through the list at connection time rather than hardcoding them.

In practice a tool reads the entries it needs once at startup, caches the RAM addresses, and then polls those addresses directly for the rest of the session.

20.2 Snapshot Logging

The ECU has a built-in burst logging mechanism (`can_a_mb14_recv_log`) that captures a snapshot of internal variables and streams them as a sequence of CAN frames. Unlike the live tuning interface, this feature does not require the ECU to be unlocked.

Trigger

A CAN message on ID 0x80 (mailbox 14) triggers the snapshot. The command byte selects the logging mode, which determines how many frames are sent:

- **Mode 5/6/7**: 12 frames (96 bytes)
- **Mode 2/4**: 36 frames (288 bytes)
- **Mode 1/3**: 44 frames (352 bytes)

Data Collection

The `log_copy` function fills the `log_data` buffer by walking `log_varptr_list`, a fixed table of 233 entries (`struct_varptr`: pointer + size). Each variable's raw bytes are copied sequentially into the buffer.

In addition, 8 user-selectable variables are appended. These are read from `dev_varptr_list` by index (stored in a configurable index array), with each value zero-extended or truncated to 16 bits.

Transmission

The snapshot is sent as a burst of 8-byte CAN frames on mailbox 2. The first frame goes out on CAN ID 0x200, and each subsequent frame increments the ID by 4 (0x204, 0x208, ...). The MB2 transmit-complete interrupt (`can_a_mb02_send_log`) automatically sends the next frame until the `log_canid` array reaches a zero terminator.

20.3 Tuning Tips

If you want to be a better tuner than average there is no magic: you need to know what is going on inside the engine and inside the ECU. The more data you have, the better your decisions will be. Before touching any map, invest time in your logging strategy.

Interface Selection

On cars built from 2008 onwards the OBD interface runs over CAN-Bus and provides a good refresh rate, making it the most convenient option for logging standard PIDs.

On pre-2008 cars the OBD interface uses K-Line. Although all the important data is still accessible, the refresh rate is significantly lower and makes time-based analysis much harder. For these cars, use the raw RAM access interface (Section 20.1) to poll variables directly at a higher rate. Upgrading a 2006–2007 car to the 2008 CAN-Bus specification is also an option worth considering.

What to Log

As a minimum, always log engine speed, load, coolant temperature, both STFT and LTFT corrections, and the octane scalers. Add throttle position and intake air temperature whenever transient or thermal behaviour is under investigation. On supercharged cars, include boost pressure via raw RAM access — it is not available on the standard OBD interface without a software patch.

21 Protocols

21.1 TPMS

The ECU communicates with an external TPMS controller over CAN-B, a dedicated bus running at approximately 47.6 kbit/s (separate from the main 500 kbit/s CAN-A bus used for OBD and cluster communication). TPMS support is optional and enabled by `CAL_tpms_use_tpms`.

CAN-B Message Layout

The ECU sends commands on three CAN IDs:

- **0x220** (`can_b_tpms_init`): Initialization frame sent at startup and before each session start/stop.
- **0x240** (`can_b_tpms_send`): Command frames with service byte 0x97. Supports multi-frame transfers for payloads longer than 5 bytes (first frame 0xC0 — count, continuation frames 0x80 — remaining).
- **0x266** (`can_b_tpms_status`): Status acknowledgment sent after receiving diagnostic responses.

The TPMS controller sends data on three CAN IDs, received by the ECU on CAN-B mailboxes 5, 7, and 9:

- **Pressure data** (MB5): Contains sensor status bits and four tire pressures (`tpms_pressure_fl`, `_fr`, `_rl`, `_rr`) in `u8_pressure_40mbar`. Status flags indicate warnings or sensor errors; errored sensors have their pressure zeroed.
- **Diagnostic responses** (MB7): Multi-frame responses to ECU queries, using a simple framing protocol.
- **Sensor data** (MB9): Individual sensor readings by command byte (0x27 = sensor data by index, 0x2F = sensor ID, 0x2D = pairing info, 0x2E = status flags, 0x30 = additional data).

Startup Sequence

At power-on, `tpms_process` runs a 9-state sequence with a 50-second timeout:

0. Send initialization frame.
1. Query the TPMS controller's stored VIN (service 0x21, sub 0x2F).
2. Compare the returned VIN against `CAL_ecu_generic_VIN`. If it matches, the controller is already configured and the sequence ends.
3. Write VIN and tire target pressures to the controller (service 0x3B, sub 0x2F): `CAL_tpms_front_pressure` and `CAL_tpms_rear_pressure`.
4. Wait for acknowledgment.
5. Send configuration with warning thresholds (service 0x3B, sub 0x40): `CAL_tpms_front_threshold` (applied to both front tires) and `CAL_tpms_rear_threshold` (applied to both rear tires), along with hardcoded timing and hysteresis parameters.
6. Wait for acknowledgment.
7. Send enable command (service 0x3B, sub 0x47).
8. Wait for acknowledgment.

Each wait state retries with a countdown timer and falls back to state 0 for a configurable number of retries before giving up.

OBD Session

The TPMS session handler (`tpms_session_handler_100ms`) is triggered by OBD mode 0x13. It sends periodic keep-alive messages (service 0x3E), reconfigures the controller, and queries status. The tire pressures and `tpms_flags` are available through OBD for external tools, and are forwarded to the instrument cluster for the TPMS warning display.

21.2 Instrument Cluster

The ECU sends engine data to the instrument cluster on CAN-A at 500 kbit/s. Two CAN IDs are used: 0x400 for the main data frame and 0x401 for text messages.

Data Frame (CAN ID 0x400)

The function `send_cluster_data` assembles an 8-byte `struct_data2cluster` and transmits it on CAN-A mailbox 4:

Byte	Type	Field
0	u8_speed_kph	`speed_display` — augmented speed for speedometer
1	u8_speed_kph	`speed_odo` — true speed for odometer
2–3	u16_rspeed_rpm	`rpm` — engine RPM
4	u8_factor_1/255	`fuel_level` — fuel gauge
5	u8_temp_5/8-40c	`coolant` — coolant temperature
6	uint8_t	`lights_flags[0]` — warning lights (see below)
7	uint8_t	`lights_flags[1]` — region / coolant warning

Table 6: Cluster CAN frame (ID 0x400) layout

Speed Augmentation

The cluster receives two separate speed values. The `speed_odo` field always contains the true `car_speed` and is used by the cluster for odometer counting. The `speed_display` field is inflated for legal compliance (speedometer must never read below the actual speed):

- Below 130 kph: `speed_display` = `car_speed` × 107/100
- 130–155 kph: interpolated formula that gradually reduces the augmentation factor from 7% down toward 2%.
- Above 155 kph: `speed_display` = `car_speed` × 102/100

An OBD override (mode 0x2F) can substitute a fixed calibration value for the displayed speed.

RPM

Under normal conditions, `engine_speed_2` is sent. When launch control is active, `LEA_tc_launchcontrol_revlimit` is sent instead, causing the tachometer to show the launch RPM target. An OBD override can substitute a test RPM value.

Fuel Level

The raw `fuel_level` sensor reading (14 = empty, 170 = full) is rescaled to 0–255 for the cluster gauge.

Shift Lights

A 4-state machine controls the shift light progression relative to the current `rev_limit`. The lookup table `CAL_misc_shift_lights_before_revlimit` provides an RPM step per gear. The three thresholds are:

- **State 0 → 1** (one red light): `rev_limit` − 8 × `step`
- **State 1 → 2** (two red lights): `rev_limit` − 6 × `step`
- **State 2 → 3** (three rapidly flashing): `rev_limit` − 4 × `step`

In state 3, the code XORs the three light bits each cycle, causing all three lights to flash rapidly. Each downward transition has a hysteresis offset (`CAL_misc_shift_lights_margin`) to prevent flickering.

Warning Lights

The `lights_flags` bytes encode the following indicators:

Byte	Bits	Indicator	Source
0	[2:0]	Shift lights	0=off, 1=one red, 3=two red, 7=three flashing
0	3	MIL	`obd_mil_flags` bit 3
0	4	Oil pressure	`sensor_adc_oil_pressure` < 0x200
0	5	TC intervention	`tc_flags` bit 5 (engine running)
0	6	TPMS warning	`tpms_flags` (low pressure or fault)
0	7	Temperature unit	VIN[11] = 'A'/'L': display coolant temp in °F
1	4	Coolant warning	`coolant` > `CAL_misc_coolant_warning_max`
1	5	VIN region	VIN[11] ≠ 'A'/'B'/'L'/'M'

Table 7: Cluster warning light bit encoding

The MIL and oil pressure indicators can be overridden through OBD mode 0x2F for instrument cluster testing.

Text Messages (CAN ID 0x401)

The function `send_cluster_text` sends 8-byte frames on the same mailbox via a TX-complete interrupt. The first byte is a flag (0 = no text, 1 = text present), followed by a 7-character ASCII string displayed on the cluster.

TPMS warnings are rotated through the display with a countdown timer, cycling through the strings "LF LOW", "RF LOW", "LR LOW", "RR LOW" for individual tire warnings. A TPMS system fault alternates between "TPMS" and "FAULT".

22 OBD Diagnostics

The T4e ECU supports OBD diagnostics over CAN-A (ISO 15765-4, 500 kbit/s) on 2008-onward vehicles. In addition to the standard OBD-II services (modes $01–$09), the ECU implements four manufacturer-specific services.

22.1 Mode $11 — Reset Learned Tables

Mode $11 resets all adaptive corrections to their default values in RAM and replies with service ID 0x51. The handler `obd_mode_0x11_reset_learn_table()` calls `reset_learn_tables()`, which performs the following resets:

- **LTFT:** `LEA_ltft_zone1_adj` $\leftarrow 0\,\mu s$; `LEA_ltft_zone2_adj` and `LEA_ltft_zone3_adj` $\leftarrow 128$ ($= 0\,\%$ with the $-50\,\%$ offset).
- **Idle learn:** `LEA_idle_flow_adj1` and `LEA_idle_flow_adj1_ac_on` $\leftarrow 0$.
- **Knock / octane:** `LEA_knock_retard2[0..3]` $\leftarrow 0$.
- **Alpha-N adjustment:** `LEA_load_alphaN_adj[256]` $\leftarrow 100$ ($= 100\,\%$, no correction); its X/Y axes restored from `CAL_` defaults. `LEA_ecu_engine_speed_byte_coefficient` and `LEA_ecu_engine_speed_byte_offset` also restored from `CAL_`.
- **Misfire stroke-time correction:** 16×4 table (`LEA_misfire_stroke_time`, per load-band per cylinder) copied from calibration flash defaults; associated sample counter zeroed.

The reset takes effect in RAM immediately; values are written to EEPROM on the next normal shutdown flush.

Note: This is distinct from the "9999" VIN trick in Mode $3B (see Section 22.4). The "9999" trick corrupts the first 30 bytes of `LEA_base` with invalid data, causing the EEPROM integrity check to fail on the next power-up. The ECU then reinitialises the *entire* EEPROM from defaults, resetting all `LEA_` variables — a more thorough reset than Mode $11, which only resets the specific adaptive variables listed above.

22.2 Mode $22 — Read Data by Identifier

Mode $22 reads live or static values using a 2-byte PID. The ECU replies with service ID 0x62. The handler `obd_mode_0x22_performance_data()` implements two PID blocks:

- **Live data** (high byte 0x02, $0200–$024A): real-time engine variables listed in Table 8.
- **Performance logs** (high byte 0x03, $0300–$0339): EEPROM-backed counters and event records (`LEA_perf_*` variables) described at the end of this subsection.

PID	Variable	Description
Supported PIDs capability (same role as mode $01 PID $00/$20/$40)		
$0200	(bitmask)	Supported PIDs for $0201–$0220
$0220	(bitmask)	Supported PIDs for $0221–$0240
$0240	(bitmask)	Supported PIDs for $0241–$0260
EVAP system		
$0201	evap_leak_state	EVAP system leak status (raw byte)
$0212	evap_pressure	EVAP purge vacuum pressure, **0.1 mbar/LSB**
$022F	evap_concentration_2	EVAP vapour concentration, raw 16-bit ×2
$0247	evap_leak_result	EVAP leak check result, **0.1 mbar/LSB**
Fueling / injection		
$0205	inj_time_final_1	Injection pulse time, **1 µs/LSB** (displayed in ms)
$0213	afr_target	Air/fuel ratio target, **0.01 AFR/LSB**
$0214	L9822E_outputs	Output relay flags (fuel pump, fans, A/C…), raw bit field
$0226	load_2	Mass air per stroke, **4 mg/LSB**
$022A	load_5	Engine load, **1 %/LSB**
$022E	LEA_ltft_zone1_adj	LTFT Zone 1 / fuel learn dead time B1, **1 µs/LSB**
$0248	LEA_ltft_zone2_adj	LTFT Zone 2 B1, **0.39 %/LSB − 50 %**
$0249	LEA_ltft_zone3_adj	LTFT Zone 3 B1, **0.39 %/LSB − 50 %**
$024A	LEA_ltft_zone1_adj	LTFT Zone 1 B2 stub (same as $022E, B2 not fitted)
$023A	fuel_learn_timer	Fuel learn timer, **0.1 s/LSB**
Knock / octane		
$0231	knock_retard1[0..3]	Knock retard cyl. 1/3/4/2 (`ign_retard1`), **0.25°/LSB**
$0218	LEA_knock_retard2[0]	Octane scaler cyl. 1 (`ign_retard2`), **0–100 %**
$0219	LEA_knock_retard2[1]	Octane scaler cyl. 3, **0–100 %**
$021A	LEA_knock_retard2[2]	Octane scaler cyl. 4, **0–100 %**
$021B	LEA_knock_retard2[3]	Octane scaler cyl. 2, **0–100 %**
VVT / VVL		
$0207	vvt_target_smooth	VVT target cam position (inlet), **0.25°/LSB**
$0208	vvt_pos	VVT actual cam position B1 (inlet), **0.25°/LSB**
$022B	vvt_diff	Cam angle error B1, **0.25°/LSB**
$0209	vvl_is_high_cam	VVL status (0 = low cam, 1 = high cam)
Throttle / pedal		
$023B	tps_target_smooth	Throttle target position, **0.098 %/LSB**
$0245	tps_max	Throttle position, **0.098 %/LSB**
$0246	pps_min	Pedal position, **0.098 %/LSB**
Idle		
$020D	idle_speed_target	Idle speed target, **5 RPM/LSB + 600 RPM**
$022C	engine_speed_idle_error	Idle speed error, **1 RPM/LSB**
$0203	LEA_idle_flow_adj1	Idle learn, **9.77×10^{-5} g/s/LSB**
$023C	LEA_idle_flow_adj1_ac_on	Idle learn with A/C on, **9.77×10^{-5} g/s/LSB**
$0204	idle_flow_target	Idle air output, **0.1 g/s/LSB**
$0206	idle_status	Idle status flags (raw byte)
$0232	idle_flow_adj5_or_zero	Idle control proportional term, **0.1 g/s/LSB**
$0233	idle_flow_adj8	Idle control integral term, **9.77×10^{-5} g/s/LSB**
Traction control / wheel speeds		
$023F	wheel_speed_fl	Wheel speed LF, **0.01 km/h/LSB**
$0241	wheel_speed_fr	Wheel speed RF, **0.01 km/h/LSB**

. . . continued

PID	Variable	Description
$023D	wheel_speed_rl	Wheel speed LR, **0.01 km/h/LSB**
$023E	wheel_speed_rr	Wheel speed RR, **0.01 km/h/LSB**
$0242	tc_slip	TC slip, **1 %/LSB**
$0243	tc_slip_target	TC target slip, **1 %/LSB**
Misfire		
$0234	misfire_count[0]	Misfire count cyl. 1, unsigned 16-bit
$0235	misfire_count[1]	Misfire count cyl. 3
$0236	misfire_count[2]	Misfire count cyl. 4
$0237	misfire_count[3]	Misfire count cyl. 2
$0238	misfire_max_result	Max misfire result (emissions), unsigned 16-bit
$0239	misfire_max_result_cat_damage	Max misfire result (cat damage), unsigned 16-bit
Catalyst monitor		
$020A	cat_diag_pre_o2_sw	Cat monitor pre-cat O2 switch B1, raw count
$020B	cat_diag_pre_o2_max_sw	Cat monitor pre-cat O2 max switch, raw count
$0244	cat_diag_pre_o2_timer	Pre-cat O2 diagnostic timer, **0.005 s/LSB**
TPMS		
$0215	tpms_pressure_fl/fr/rl/rr	Tyre pressures FL/FR/RL/RR, **0.04 bar/LSB** per byte
$0216	tpms_flags	TPMS status and fault flags, raw 16-bit
Temperatures / timers		
$0225	intake_air_smooth	Ambient air temperature, **0.625 °C/LSB − 40 °C**
$0227	coolant_stop	Coolant temperature at engine-off, **0.625 °C/LSB − 40 °C**
$0228	maf_accumulated_1	Accumulated mass air, **1 mg/LSB**
$0229	shutdown_delay_2	ECU shutdown timer, **0.005 s/LSB**
$022D	ecu_runtime	ECU on timer, **0.1 s/LSB**
Miscellaneous		
$020C	car_gear_current	Current gear (raw byte)
ECU identity (static, hardcoded values)		
$020E	(bootloader)	Serial number
$020F	(bootloader)	Hardware number
$0210	(bootloader)	Crypto flag
$0211	(bootloader)	ECU type (ASCII `"T4e "`)
$021C–$021F	CAL_ecu_model_name[0..15]	ECU identity, 4 bytes per PID
$0221–$0224	CAL_ecu_model_name[16..31]	ECU identity continued

Table 8: Mode $22 live data PIDs (0x02xx)

Performance Log PIDs (0x03xx)

The 0x03xx block reads `LEA_perf_*` variables recorded during normal driving. The block follows the same supported-PIDs convention as the live-data block.

PID	Variable	Description
Supported PIDs capability		
$0300	(bitmask)	Supported PIDs for $0301–$0320
$0320	(bitmask)	Supported PIDs for $0321–$0340
Time histograms (32-bit counters, seconds)		
$0301–$0308	LEA_perf_time_at_TPS[0..7]	Time spent at each of 8 TPS buckets
$0309–$0310	LEA_perf_time_at_RPM[0..7]	Time spent at each of 8 RPM buckets
$0311–$0318	LEA_perf_time_at_KMH[0..7]	Time spent at each of 8 vehicle speed buckets
$031A–$031D	LEA_perf_time_at_coolant_temp[0..3]	Time spent in each of 4 coolant temp bands
Peak engine speed events (5 entries)		
$031E	LEA_perf_max_engine_speed[0]	Peak engine speed #1, 16-bit RPM
$031F	LEA_perf_max_engine_speed[1]	Peak engine speed #2
$0321–$0323	LEA_perf_max_engine_speed[2..4]	Peak engine speeds #3–5
$0324	LEA_perf_max_engine_speed_5_coolant_temp	Coolant temp at peak #5
$0325	LEA_perf_max_engine_speed_5_run_timer	Engine run time at peak #5, 32-bit
$0326	LEA_perf_max_engine_speed_4_coolant_temp	Coolant temp at peak #4
$0327	LEA_perf_max_engine_speed_4_run_timer	Engine run time at peak #4, 32-bit
$0328	LEA_perf_max_engine_speed_3_coolant_temp	Coolant temp at peak #3
$0329	LEA_perf_max_engine_speed_3_run_timer	Engine run time at peak #3, 32-bit
$032A	LEA_perf_max_engine_speed_2_coolant_temp	Coolant temp at peak #2
$032B	LEA_perf_max_engine_speed_2_run_timer	Engine run time at peak #2, 32-bit
$032C	LEA_perf_max_engine_speed_1_coolant_temp	Coolant temp at peak #1
$032E	LEA_perf_max_engine_speed_1_run_timer	Engine run time at peak #1, 32-bit
Peak vehicle speed (5 entries)		
$032F–$0333	LEA_perf_max_vehicle_speed[0..4]	Five peak vehicle speeds, 1 byte each
Standing starts		
$0334–$0335	LEA_perf_fastest_standing_start[0..1]	Fastest standing start time
$0336–$0337	LEA_perf_last_standing_start[0..1]	Last standing start time
$0339	LEA_perf_number_of_standing_starts	Total standing start count, 16-bit
Engine run time		
$0338	LEA_perf_engine_run_timer	Total engine run time, 32-bit

Table 9: Mode $22 performance log PIDs (0x03xx)

22.3 Mode $2F — Input/Output Control

Mode $2F activates individual ECU outputs for bench testing and diagnostics. The ECU replies with service ID 0x6F. All T4e PIDs have high byte 0x01; the handler `obd_mode_0x2F_test()` dispatches commands into an `obd_mode_0x2F_state` machine updated every 5 ms.

Like mode $01 PIDs $00/$20/$40, PIDs $0100, $0120, $0140, and $0160 are supported-PIDs capability responses (4 bytes each); they list which actuator PIDs in the following 32 positions are available and do not activate any output. All other PIDs are actuator commands. Most actuators require the engine to be stopped (`engine_speed_1 == 0`) before the request is accepted; exceptions are noted below.

PID	Output	Notes
Injectors (TPU-B, engine off required)		
$0101	Injector 1 (TPU3B CH1)	pulse-mode via TLE6220GP
$0102	Injector 2 (TPU3B CH2)	
$0103	Injector 3 (TPU3B CH3)	
$0104	Injector 4 (TPU3B CH4)	
Ignition coils (TPU-A, engine off required)		
$0161	Coils 1 + 4 (TPU3A CH1 + CH4)	paired firing (1-4 TDC pair)
$0162	Coils 2 + 3 (TPU3A CH2 + CH3)	paired firing (2-3 TDC pair)
$0163	Coil 3 only (TPU3A CH3)	
$0164	Coil 4 only (TPU3A CH4)	
Cam / valvetrain (engine off required)		
$0121	VVT output	timed, reverts on timeout
$0122	VVL output	timed, reverts on timeout
Fuel system		
$0141	Fuel pump	engine off required
$0143	Lambda (O2) heaters (all sensors)	engine off required
$0127	EVAP purge valve	**no engine-off requirement**
$0149	EVAP canister close valve	engine off required
Cooling / air		
$014A	Cooling fan 1	engine off required
$014B	Cooling fan 2	engine off required
$0144	Coolant circulation pump	engine off required
$0142	Intake air valve (ACIS)	no engine-off; requires timer != 0
Cluster / warning lights (engine off required)		
$0125	Tachometer enable	timed display override
$0126	Speedometer enable	timed display override
$0147	MIL	
$0148	Oil pressure warning lamp	
Other (engine off required)		
$0146	A/C clutch	

Table 10: Mode $2F actuator PIDs

22.4 Mode $3B — VIN Programming

Mode $3B writes chunks of the VIN stored in `LEA_ecu_VIN` (EEPROM-backed). The handler `obd_mode_0x3B_VIN()` replies with service ID 0x7B. No security seed/key challenge is required.

Each request carries a 1-byte index followed by up to 4 data bytes. The index selects which region of the 17-character VIN to overwrite:

Index	Target	Notes
0	*(ignored)*	No response sent
1	`LEA_ecu_VIN[3..6]`	4 bytes written
2	`LEA_ecu_VIN[7..10]`	4 bytes written
3	`LEA_ecu_VIN[11..14]`	4 bytes written
4	`LEA_ecu_VIN[15..16]`	2 bytes written
5	*(TPMS)*	Triggers `tpms_session_handler_100ms()` with state set from request byte

Table 11: Mode $3B VIN programming index map

If VIN[13..16] = “9999”, the first 30 bytes of `LEA_base` are set to `'1'`, corrupting the EEPROM so the next boot reinitialises all `LEA_` variables to their standard values.

VIN characters at positions 0–2 (manufacturer identifier “SCC”) are not writable via this service.